World Models in Action

4. Predicting in representation space

This chapter covers

- Spatial and spatiotemporal JEPA prediction, from I-JEPA to V-JEPA
- Cross-modal prediction into language embeddings
- Collapse, predictive sufficiency, and the statistical geometry of embeddings
- LeJEPA/SIGReg and VISReg as distributional regularizers
- Action-conditioned prediction, planning, memory, hierarchy, and predictive-agent design

4.1. Compatibility, energy and prediction

A predictive model can estimate an observation, represent a distribution over possible outcomes, or assign a score to a candidate configuration. These descriptions are related, but they are not interchangeable. A joint-embedding predictive architecture does not try to reproduce every detail of an observation. It learns a representation in which a selected target is compatible with the information available in a context.

The most useful starting point is therefore not an encoder architecture. It is the notion of **compatibility**.

4.1.1. A score over candidate targets

Let x denote information available to a model and y a candidate target. A model can assign a scalar score

$$E_\theta(x, y) \in \mathbb{R}.$$

Adopt the convention that lower values indicate greater compatibility. Given x, an inference procedure can select a target by solving

$$\hat{y}(x) = \arg\min_{y \in \mathcal{Y}} E_\theta(x, y).$$

This is a ranking or optimization rule. It identifies which candidate the model prefers, but it does not specify a probability for that candidate.

A probability density can be defined from an energy by choosing a Gibbs distribution,

$$p_\theta(y \mid x) = \frac{\exp[-E_\theta(x, y)]}{Z_\theta(x)}, \qquad Z_\theta(x) = \int_{\mathcal{Y}} \exp[-E_\theta(x, y')]\, d\nu(y'),$$

where ν is a specified base measure on the target space. The normalizer $Z_\theta(x)$, called the partition function, makes the density integrate to one. For a high-dimensional continuous target, computing this integral can be intractable. An energy-based model may therefore use its score directly for ranking or optimization without evaluating a normalized density.

The distinction matters in representation learning. A feature discrepancy can be used as a training loss or compatibility score without defining a calibrated distribution over future observations. A planner can minimize a scalar objective without that objective being a negative log probability. Probability requires a normalization convention and a base measure; optimization requires a scalar criterion and a search domain.

4.1.2. Introducing a latent configuration

A context and target may be compatible for several different reasons. There may be hidden factors, alternative explanations or multiple plausible completions. Introduce a latent variable z to represent a configuration that is not directly observed:

$$E_\theta(x, y, z).$$

There are at least two ways to eliminate the latent variable from this expression. The first is to select the lowest-energy configuration,

$$F_{\min}(x, y) = \inf_z E_\theta(x, y, z).$$

This asks whether there exists a latent configuration that makes the pair compatible. It is a minimization over z.

The second is to aggregate over latent configurations,

$$F_{\mathrm{marg}}(x, y) = -\log \int_z \exp[-E_\theta(x, y, z)] \, d\nu(z).$$

This is a negative log marginal score, up to any normalization terms needed to define the relevant conditional distribution. It combines contributions from all latent configurations rather than retaining only the minimum. If many configurations have similar energy, their combined volume can matter. The minimum-energy and marginal constructions therefore need not rank two candidate targets in the same way.

A latent variable is useful when context alone does not determine one target. A video prefix, for example, may be compatible with several continuations. Different values of z can select different continuations without requiring the model to average them into one deterministic prediction.

The latent also creates a shortcut. If it has enough capacity to encode the target representation itself, the predictor can ignore the context and transmit the answer through z. The model can then achieve low energy without learning a relation between context and target. The latent must be constrained by its dimension, distribution, inference mechanism or another design choice. The exact constraint determines how many alternatives can be represented and how information is divided between context and latent state.

LeCun's JEPA formulation treats this restriction as a central design requirement: the latent should account for uncertainty that cannot be explained by context, not provide an unrestricted channel for copying the target. This is a conceptual requirement in the position paper, not a claim that it defines one universal mutual-information penalty. (LeCun, 2022)

4.1.3. Joint-embedding prediction

A joint-embedding predictive architecture moves the comparison from observation space to a learned representation space.

Let

$$s_x = f_\theta(x), \qquad s_y = f_{\bar{\theta}}(y)$$

be representations of the context and target. A predictor maps the context representation, and possibly a query q, to a target representation:

$$\hat{s}_y = g_\phi(s_x, q).$$

The query specifies which target is requested. In an image model it may identify spatial positions. In a video model it may identify spatiotemporal positions. In an action-conditioned model it must also include the action or control sequence whose consequence is being predicted.

A feature-space compatibility score is

$$E_{\theta,\bar{\theta},\phi}(x, y, q) = d\big(g_\phi(f_\theta(x), q), f_{\bar{\theta}}(y)\big),$$

where d measures the discrepancy between predicted and target features. For squared Euclidean distance,

$$d(u, v) = \|u - v\|_2^2.$$

Training lowers this discrepancy for context-target-query triples sampled from the data distribution. The target representation determines which discrepancies count. A difference

removed by $f_{\bar{\theta}}$ contributes little or nothing to the objective; a difference retained by the target encoder remains visible to the loss.

This is the essential change from pixel reconstruction. A pixel-level objective treats discrepancies in the observation as the quantities to explain. A latent prediction objective treats discrepancies in a learned representation as the quantities to explain. Neither choice is universally correct. Fine contact geometry or precise texture may require detailed observation-level information. Object identity, spatial layout or motion may be represented more compactly. Feature prediction changes what is supervised; it does not establish in advance that the representation is sufficient for an agent.

A latent vector is not semantic merely because it is internal. Its invariances are determined by the data, target encoder, mask distribution, temporal sampling, architecture and objective. The representation is a modelling choice. Its usefulness must be evaluated against the distinctions required by downstream predictions and decisions.

4.1.4. Invariance and alternative futures are different operations

A context need not determine a unique target. There are two conceptually different ways to handle multiple outcomes.

The first is **invariance in the target representation**. Suppose observations $y_1, \dots, y_m$ differ in details that are irrelevant to the task. If the target encoder maps them to the same representation,

$$f_{\bar{\theta}}(y_1) = \cdots = f_{\bar{\theta}}(y_m),$$

then the objective treats them as equivalent targets. A predictor can match that shared representation without choosing among the observation-level differences.

This is appropriate only when those differences are genuinely irrelevant. If the encoder removes object position, contact state or gripper aperture when the task depends on those variables, the invariance is destructive. The training objective has no direct reason to recover information that the target representation has removed.

The second operation is a **latent variable that selects among alternatives**. For example,

$$\hat{s}_y = g_\phi(f_\theta(x), z).$$

For a fixed context x, different values of z can produce different predicted representations. The latent preserves distinctions between possible outcomes but allows the model to select among them. A vehicle approaching a fork may turn left or right. The two futures remain different, but the current image does not determine which one will occur.

These mechanisms are not interchangeable. Invariance says that two outcomes should count as the same for the task. A latent variable says that the outcomes remain different, but the available context does not determine which one should be predicted. The first removes a distinction; the second represents an unresolved choice.

4.1.5. The latent must not become a copy of the answer

Suppose the latent z can carry a vector with the same information as the target representation s_y, and the predictor is expressive enough to ignore the context and output z. It can choose $z = s_y$, giving

$$d(g_\phi(s_x, z), s_y) = 0$$

without learning how the context relates to the target.

The objective would be minimized for the wrong reason. The predictor would not be estimating the target from context; the latent would be transmitting the target directly.

A useful architecture must therefore decide which information is permitted to enter the latent variable and how that information is selected. Its capacity may be limited, its distribution may be constrained, or its values may be inferred through a separate mechanism. A model that needs several plausible futures must preserve enough capacity to represent them, while preventing the latent from making every target trivially predictable.

This is a design tension rather than a universal recipe. A discrete latent with a small codebook, a low-dimensional continuous latent and a stochastic latent with a prior make different assumptions. Their suitability depends on the structure and number of alternatives the task requires.

4.1.6. Prediction loss, compatibility energy and task cost

Three scalar quantities appear repeatedly in predictive agent systems.

Prediction loss compares the model output with a training target. A generic feature-prediction objective is

$$\mathcal{L}_{\text{pred}} = \mathbb{E}_{(x,y,q)\sim p_{\text{data}}} \left[d\big(g_\phi(f_\theta(x), q), f_{\bar{\theta}}(y)\big) \right].$$

It is defined by the chosen representation, target sampling distribution and discrepancy function. It answers whether the predictor matches the selected target features.

Compatibility energy scores a candidate configuration under a learned model. It may have the same mathematical form as a prediction discrepancy, but its interpretation depends on how the score is trained and used. It is not necessarily a calibrated probability.

Task cost evaluates a predicted state or trajectory according to an agent's objective. It may penalize distance from a goal, collision, excessive force, energy use or discomfort. It answers whether a predicted outcome is desirable.

A common scalar notation does not make these quantities interchangeable. A small prediction loss does not imply that a future is desirable. A low compatibility energy does not imply safety. A task cost need not equal the discrepancy used to train the representation.

This distinction becomes essential when an action-conditioned model is placed inside a planner. The model predicts what may happen under a candidate action. A separate cost evaluates the predicted outcome. The planner searches for actions that reduce that cost. A predictive representation supplies part of this process; it does not specify the complete decision problem.

4.1.7. What does the target represent?

A target is not necessarily a future observation. The word *prediction* refers to estimating a withheld quantity from available information. The target's temporal and informational relation to the context must be stated.

- A **spatial target** is a hidden region or set of patches from the same image. It asks what can be inferred about a withheld part of the observation.
- A **spatiotemporal target** is a masked region of a video clip. It asks which features are compatible with the visible space-time context.
- A **future target** is sampled from a later observation or state. Its prediction horizon depends on temporal stride and sampling.
- An **action-conditioned target** is predicted after specifying an intervention. The action is part of the condition, not simply a label attached to the observed outcome.
- A **cross-modal target** belongs to a different modality. A visual context may be used to predict a text representation, for example.

These are different conditional prediction problems. The phrase *predictive representation* includes all of them, but it does not make them equivalent. A masked image objective does not become a physical dynamics model because its target is hidden. A video objective does not become causal because the target occurs later in time. An action-conditioned objective requires the action to be represented in the model input and aligned with the transition being predicted. (Assran et al., 2023; Bardes et al., 2024; Assran et al., 2025; Chen et al., 2025/2026)

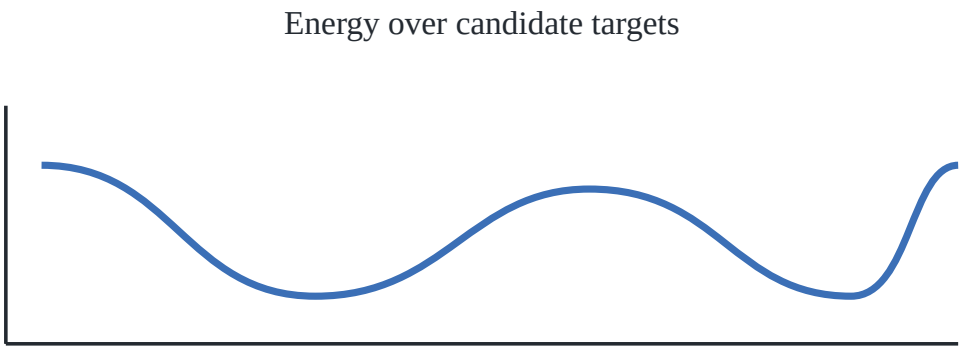


Figure 4 1. Energy landscape with multiple compatible target modes

Figure 4.1. Energy across candidate targets. For a fixed context, several candidate targets may occupy low-energy regions. The energy ranks compatibility; a probability interpretation requires a normalization and base measure.

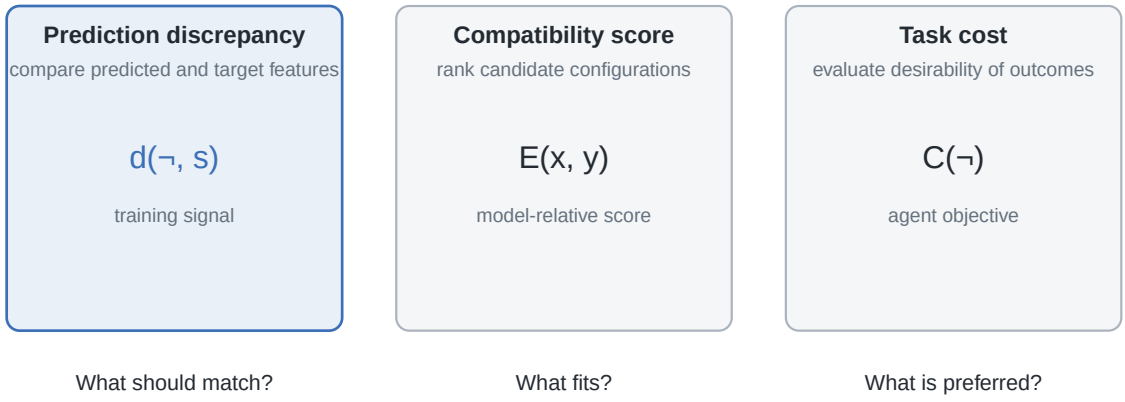


Figure 4 2. Prediction discrepancy, compatibility score and task cost

Figure 4.2. Three distinct scalar objectives. A feature discrepancy is a training signal, an energy scores compatibility, and a task cost evaluates desirability. One system may use all three, but they are not interchangeable.

The next section defines the context, target and query variables more carefully. This is necessary before discussing the optimization objective, the information it can preserve, or the degenerate solutions it admits.

4.2. The prediction problem in representation space

The expression ``predict a representation'' is compact, but it hides several modelling decisions. A method must specify what information is available, what is withheld, what the target encoder observes, and which variable tells the predictor what it is expected to estimate. These decisions define the actual statistical problem. The loss function alone does not.

4.2.1. Context, target and query

Let X denote the information available to the model and Y the target information it is asked to predict. The variables need not be individual frames. X may be a crop, a set of visible image patches, a video prefix or a history of observations. Y may be a hidden image block, selected video tokens, a later observation, or a representation from another modality.

A query Q specifies which part of the target is requested. For an image, it may contain spatial positions. For video, it may contain spatial and temporal positions. For an action-conditioned model, the conditioning information must also include the action or action sequence. The query is part of the problem definition, not just an implementation convenience.

A general representation-prediction system can be written as

$$S_X = f_\theta(X), \qquad S_Y = f_{\bar{\theta}}(Y), \qquad \widehat{S}_Y = g_\phi(S_X, Q).$$

The training objective is an expectation over the joint distribution of context, target and query:

$$\mathcal{L}_{\text{pred}}(\theta, \bar{\theta}, \phi) = \mathbb{E}_{(X,Y,Q)\sim p_{\text{data}}} \left[d\big(g_\phi(f_\theta(X), Q), f_{\bar{\theta}}(Y)\big) \right].$$

The distribution $p_{\text{data}}(X, Y, Q)$ is as important as the function d. It determines which contexts occur, which targets are sampled, and which information the model must use to make a prediction. Changing the mask sampler changes the conditional distribution the predictor faces. Changing temporal stride changes the prediction horizon. Changing the action distribution changes the interventions represented in the training data.

4.2.2. A feature-space target is not the same as a cropped input

The context and target branches do not necessarily receive symmetric inputs. In I-JEPA, for example, the context encoder processes the selected context tokens, while the target encoder processes the full image before the target positions are selected from its feature map. The model therefore has access to a target feature at a location that is hidden from the context branch. It does not simply encode an isolated target crop through an otherwise identical pathway.

This detail matters because a target encoder defines the features being predicted. The

representation may contain spatial context from outside the target block. The target is not equivalent to an independently encoded patch unless the architecture explicitly makes it so. (Assran et al., 2023)

The same caution applies to video. A visible-context encoder and an unmasked target encoder create different information paths. The predictor is asked to infer target features from the context pathway, not to copy the target encoder's input. Positional information tells it where the target lies in the image or clip. The masking pattern determines whether the prediction requires local continuity, global scene context, temporal context or some combination.

A figure of a JEPA should therefore show the information flow, not just three boxes labelled encoder, predictor and target. In particular, it should make clear which pixels or tokens each branch receives and where target positions are selected.

4.2.3. The mask defines a conditional distribution

The target sampler determines what is unknown. Suppose M denotes a random mask and Y_M the selected target under that mask. The predictor is trained against a family of conditional distributions

$$p(Y_M \mid X_{\neg M}, M),$$

where $X_{\neg M}$ denotes the visible context. A mask selecting a small region near visible content may make the target predictable from local texture or continuity. A large, distant target may require broader context. A target spanning later times may require temporal information. If the context removes a variable essential to the target, uncertainty remains even for an ideal predictor.

There is no mask geometry that is optimal independently of the intended prediction problem. A method that makes targets too easy can reward shortcuts. A method that makes targets too ambiguous can produce weak or noisy learning signals. The target distribution is part of the inductive bias.

This is why image, video and dense-feature objectives should not be described as though they were the same prediction task at different resolutions. I-JEPA's spatial blocks test relations between visible and withheld image regions. The original V-JEPA uses masked spatiotemporal features, which can be spatially or temporally withheld. V-JEPA 2.1 changes the target supervision again by including dense features from visible as well as masked tokens and by applying supervision at intermediate depths. That choice asks the representation to preserve spatially structured information across the token grid. It is not the same as adding an action input. (Bardes et al., 2024; Assran et al., 2025; Mur-Labadia et al., 2026)

4.2.4. The deterministic squared-error predictor

A simple predictor minimizes squared distance in the target representation space. For a fixed context $X = x$ and query $Q = q$, let the random target feature be $H = f_{\bar{\theta}}(Y)$. The optimal deterministic prediction is

$$a^*(x, q) = \arg\min_a \mathbb{E}\left[\|H - a\|_2^2 \mid X = x, Q = q\right].$$

Assume the conditional second moment exists and define

$$\mu(x, q) = \mathbb{E}[H \mid X = x, Q = q].$$

Add and subtract $\mu(x, q)$:

$$\begin{aligned}\mathbb{E}[\|H - a\|_2^2 \mid x, q] &= \mathbb{E}[\|H - \mu + \mu - a\|_2^2 \mid x, q] \\ &= \mathbb{E}[\|H - \mu\|_2^2 \mid x, q] + \|\mu - a\|_2^2 \\ &\quad + 2\,\mathbb{E}[(H - \mu)^\top(\mu - a) \mid x, q].\end{aligned}$$

The cross term is zero because $\mathbb{E}[H - \mu \mid x, q$=0]. Hence

$$\mathbb{E}[\|H - a\|_2^2 \mid x, q] = \mathbb{E}[\|H - \mu\|_2^2 \mid x, q] + \|a - \mu\|_2^2.$$

The first term does not depend on a, so the unique minimizer is

$$a^*(x, q) = \mathbb{E}[H \mid X = x, Q = q].$$

This is a property of squared-error regression. It is not a pathology unique to JEPA.

4.2.5. What happens when the target is multimodal?

A conditional mean can fail to correspond to any single plausible outcome. Consider a one-dimensional target with two equally likely values, -1 and $+1$, given the same context. The conditional mean is zero, even though zero is never observed. A deterministic squared-error predictor returns the point between the two modes.

In an image or video representation, the same phenomenon can arise when one context is compatible with distinct futures. A single predicted feature may average across alternatives. Whether this is harmful depends on the geometry of the target space and the downstream task. For a representation-learning objective, a mean feature may still be useful. For a planner that must distinguish mutually exclusive outcomes, a single averaged future can be misleading.

A system that needs multiple alternatives must represent them explicitly, for example through a latent variable, a distribution over future features, an ensemble or multiple hypotheses. Merely naming the model a world model does not supply uncertainty. The conditional distribution and the prediction objective must support it.

This observation will matter when introducing the latent variable z in energy-based prediction. It can index alternative compatible futures, but it must not become an unconstrained channel through which the target is copied.

4.2.6. When is a predictor necessary?

A predictor is useful when the target contains information that is not directly available in the context, or when the model must transform context information into a target-specific representation. A future feature conditioned on a history and an action is an obvious example. A masked region whose location is specified by a query is another.

However, a separate predictor network is not a universal requirement of joint-embedding learning. For symmetric views that are intended to represent the same content, a loss can directly bring the view embeddings toward a shared centre. LeJEPA uses a multi-view construction in which a mean of global-view embeddings serves as the centre and local and global view embeddings are drawn toward it. Its basic recipe does not require the EMA teacher and predictor arrangement used by I-JEPA or V-JEPA. (Balestriero and LeCun, 2025)

The choice depends on information asymmetry:

- If the task is to make two views of the same underlying sample agree, direct view agreement may be appropriate.
- If the target is a hidden part specified by a location or query, a predictor can map context features to that target.
- If the target is a future that depends on a selected action, the predictor must receive the action and the model must be trained on aligned action-observation transitions.
- If the target is linguistic, the predictor maps visual context to a text representation; that is a cross-modal prediction problem, not a physical dynamics model.

The predictor should be justified by the conditional problem, not included merely because a previous JEPA implementation has one.

A linguistic target: VL-JEPA

The target representation need not be visual. In VL-JEPA, a visual context and a text query are used to predict an embedding of a target text response. The primary predictive objective operates in the continuous text-embedding space rather than requiring autoregressive token generation at

every training step. A language decoder can be applied when an explicit text answer is required.

This is another instance of the same conditional structure: the visual context and query define what is available, and the text embedding defines the target. It also makes the limit of the abstraction clear. Predicting a linguistic representation from an image is cross-modal prediction; it is not, by itself, an action-conditioned model of physical dynamics.

4.2.7. Target updates are a separate design choice

The notation $f_{\bar{\theta}}$ does not imply a particular update rule. Several possibilities exist:

1. An EMA target encoder, updated from the trainable encoder after each optimization step.
2. A target branch that receives no gradient but is updated by another rule.
3. A frozen encoder used as a fixed feature target.
4. Shared or symmetric branches for multi-view objectives.
5. A predictor-free objective in which the view embeddings are compared directly.

For an EMA target,

$$\bar{\theta}_{t+1} = m_t \bar{\theta}_t + (1 - m_t)\theta_{t+1}.$$

This update makes the target a smoothed history of the online parameters. It does not define the meaning of JEPA and is not a general proof that every collapse solution is impossible. The target-update rule, the loss, the mask sampler and the architecture form a method-specific training system.

It is important not to conflate the EMA used to update a target encoder with the stop-gradient applied to a tensor in a regularizer. Both affect gradient flow, but they serve different purposes. VISReg, for example, detaches the scale statistic in the shape normalization to separate scale and shape gradients. That is not the same mechanism as using an EMA teacher.

4.2.8. What a low prediction loss establishes

A low value of $\mathcal{L}_{\text{pred}}$ establishes that the predictor matches the selected target representation under the training distribution, the chosen mask or query sampler, the normalization and the optimization procedure. It does not establish, without additional evidence, that:

- the target representation preserves every variable relevant to a downstream task;
- the representation is a sufficient state for physical prediction;
- action effects are identified from passive observations;

- predictions remain accurate under recursive rollout;
- the model captures multiple futures or calibrated uncertainty;
- a planner will find desirable or safe actions;
- the result generalizes to unobserved action-state combinations.

The next section examines the simplest failure permitted by an unconstrained agreement objective: every input can receive the same representation and achieve zero prediction error.

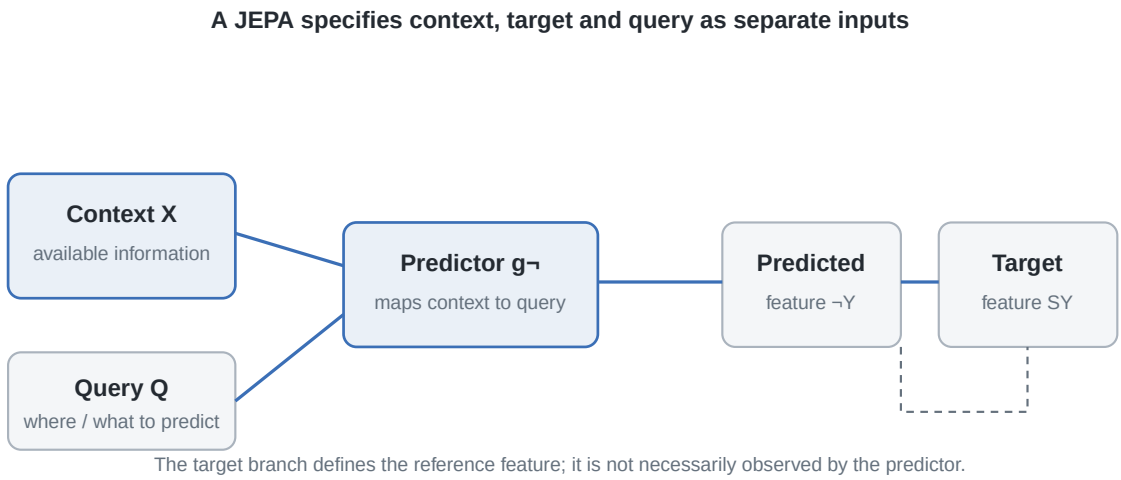


Figure 4 3. General JEPA context, query, predictor and target information flow

Figure 4.3. The context, target and query define the prediction problem. The predictor receives context and a query, while a target pathway supplies the reference feature. The target branch is not necessarily an input to the predictor.

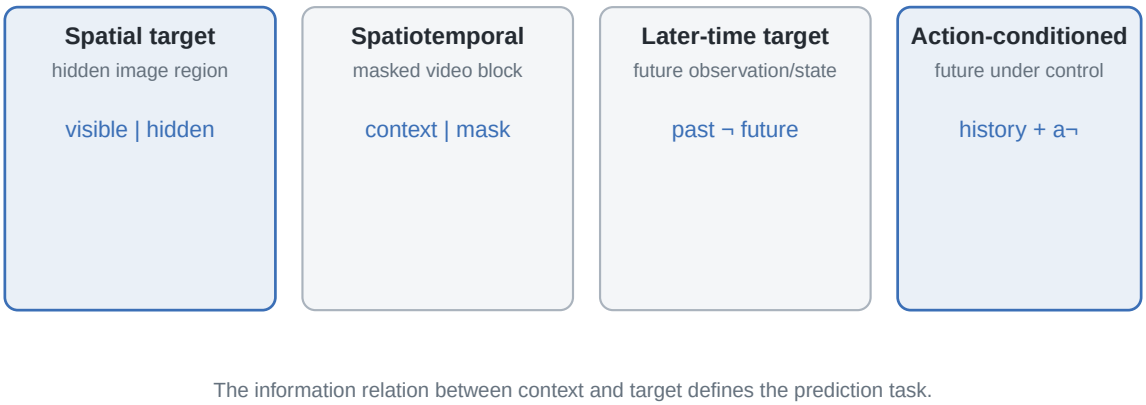


Figure 4 4. Spatial, spatiotemporal, future and action-conditioned targets

Figure 4.4. Four different conditional prediction problems. A target may be spatially hidden, spatiotemporally masked, later in time, or conditioned on an action. The target's relation to the available context defines the prediction task.

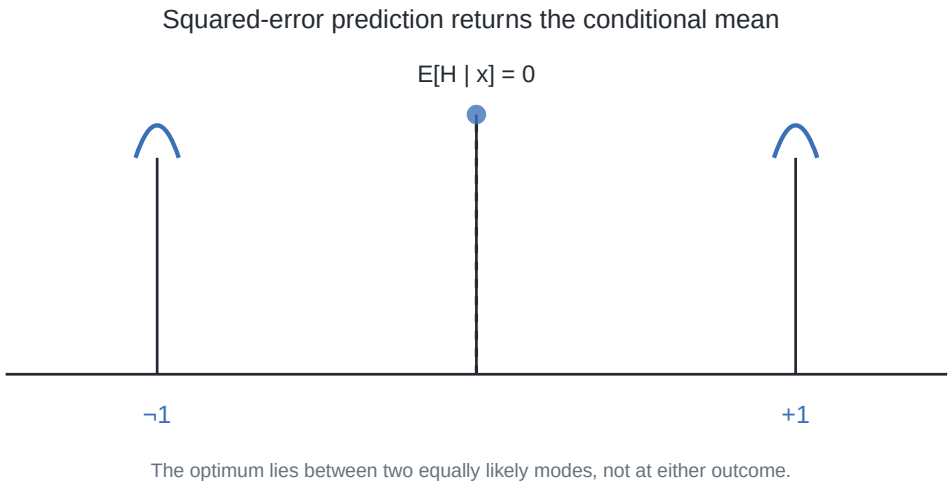


Figure 4 5. Two-mode target distribution and conditional mean

Figure 4.5. The conditional mean can lie between plausible outcomes. For two equally likely target modes, the squared-error optimum is their conditional mean. That mean need not itself be a plausible physical outcome.

4.3. I-JEPA: predicting hidden image features from spatial context

I-JEPA makes joint-embedding prediction concrete without introducing time, actions or a planner. An image is divided into patch tokens. The model receives a large region as context and predicts features at several other regions. The target is not an RGB crop. It is the output of a target encoder at selected spatial positions.

The distinction matters: the target encoder is not simply applied to an isolated hidden crop. It processes the image, produces a spatial feature map, and the method selects features at the target positions. The context encoder, by contrast, receives only visible context tokens. One branch constructs target features with access to the image; the other must infer those features from restricted context. (Assran et al., 2023)

4.3.1. From an image to a grid of features

Let an image be $x \in \mathbb{R}^{H \times W \times 3}$. A patch embedding operation partitions it into a grid and maps each patch to a vector. For patch size $P \times P$, the grid contains

$$\[N_h = H/P, \qquad N_w = W/P, \qquad N = N_h N_w\]$$

positions, assuming divisibility. The patch embedding produces tokens

$$\[e(x) = (e_1, \ldots, e_N), \qquad e_i \in \mathbb{R}^d.\]$$

Positional encodings identify where each token came from. A context encoder transforms selected visible tokens. A target encoder transforms the full image. If C is the context-position set and Y the target-position set, write

$$\[Z_C = f_\theta(e(x)_C, C), \qquad H_Y = f_{\bar{\theta}}(e(x))_Y.\]$$

The subscript Y means that target features are selected from the target encoder's output at those positions. It does not mean that the target encoder only saw the target crop. The feature at a target position may incorporate information from a larger spatial neighbourhood through the target encoder.

The predictor is trained to estimate those contextualized target features from the restricted context pathway. This is a conditional feature-estimation problem, not patchwise pixel completion.

4.3.2. The target is selected in feature space

A mask is sampled over the patch grid. One large context block determines which tokens are visible to the context encoder. Several smaller target blocks determine which target positions must be predicted. The target encoder still processes the image; the target mask selects which of its features contribute to the loss.

The default I-JEPA recipe uses one large context block and four target blocks. The context block is sampled with a large area, typically 85% to 100% of the image area. Each target block is sampled at a smaller scale, typically 15% to 20%, with a range of aspect ratios. Context positions that overlap target positions are removed from the context set. These values describe this implementation, not a universal definition of JEPA.

Why use multiple target blocks? A single target may encourage a narrow completion strategy. Several spatially separated targets require the context representation to support predictions at several locations. The predictor must use the visible image as a whole rather than only extrapolate one local boundary. Still, no mask geometry is universally best. A target adjacent to visible context may be predictable from local continuity; a distant target may require broader scene information.

The mask is part of the objective because it specifies what is unknown. It is not a neutral augmentation that can be changed without changing the conditional distribution faced by the predictor.

4.3.3. The predictor receives a location query

The context encoder produces features for visible tokens. The predictor must estimate features at target positions, but those positions are not visible as image tokens. I-JEPA supplies them as queries.

For each target block, the predictor receives context features together with learned mask tokens. A positional embedding is added to each mask token to indicate its target location. The predictor processes the visible context and target queries jointly and outputs a vector at each requested position:

$$\hat{H}_{Y_m} = g_\phi(Z_C, \mathrm{pos}(Y_m)).$$

The learned mask token provides a common starting vector for a missing position. The positional embedding distinguishes one requested location from another. Without the position, identical mask tokens would not tell the predictor which target feature it is being asked to estimate.

The predictor does not generate an image patch. It outputs a feature vector in the target encoder's representation space. The target position specifies *where* a representation is requested; the context specifies *what information is available* to estimate it.

The target blocks are prediction queries. A useful diagram should show one context pathway feeding several position-marked query sets, rather than depicting the method as a single crop-to-crop translation.

4.3.4. The training objective

Let $Y_1, \ldots, Y_M$ denote the sampled target blocks, with $M = 4$ in the default recipe. Let H_{Y_m} be target features and $\hat{H}_{Y_m}$ their predictions. The paper defines the loss as the average discrepancy over target blocks:

$$\mathcal{L}_{\text{I-JEPA}} = \frac{1}{M} \sum_{m=1}^{M} D(\hat{H}_{Y_m}, H_{Y_m}) = \frac{1}{M} \sum_{m=1}^{M} \sum_{j \in Y_m} \|\hat{h}_j - h_j\|_2^2.$$

Here D is the sum of squared Euclidean discrepancies over the target tokens in one block. The outer average gives equal weight to the sampled target blocks. Within each block, the displayed paper equation sums over target positions; it does not show an additional division by the number of positions or feature dimensions. This matters when comparing numerical loss magnitudes across implementations, because sum-versus-mean reductions change the scale of the objective even if they use identical predictions and targets. (Assran et al., 2023)

There is no pixel-level reconstruction term. A prediction is correct when it matches the target

encoder's feature, even if no RGB patch is produced. The target encoder therefore determines which visual differences matter. If its features are insensitive to illumination, predicting the target does not require reproducing illumination changes. If the target features retain object boundaries or spatial arrangement, those properties remain visible to the loss.

This does not mean that the model automatically learns semantic information or discards every unpredictable detail. It means only that the error is evaluated in a learned feature space. The contents and invariances of that space must be measured.

4.3.5. Why use a target encoder?

The target features should not move in exactly the same way as the predictor's output during the same gradient update. If both sides of a feature-matching objective are updated freely, they can reduce their discrepancy by changing together, including toward a constant representation. I-JEPA uses a slowly moving target encoder to provide a more stable regression target.

The context encoder and predictor are updated by gradient descent. The target encoder is updated by an exponential moving average:

$$\bar{\theta}_{t+1} = m_t \bar{\theta}_t + (1 - m_t)\theta_{t+1}.$$

For fixed m, repeated substitution gives

$$\bar{\theta}_t = m^t \bar{\theta}_0 + (1 - m) \sum_{k=0}^{t-1} m^k \theta_{t-k}.$$

The target parameters are a smoothed history of the online parameters. This reduces how quickly the regression target changes as the online encoder is optimized.

Three operations must be kept distinct:

- **Stop-gradient** blocks a derivative along a particular computational path.
- **EMA** updates target parameters using a weighted history of online parameters.
- **Prediction loss** compares predicted and target features.

They work together in I-JEPA but are not synonyms. EMA is part of this method's recipe, not a universal requirement for all JEPAs, and not a theorem that collapse is impossible under every implementation.

4.3.6. The mask and target encoder jointly specify the task

The context mask controls what the predictor can observe. It sets the visible fraction, distance

between visible and withheld regions, number of target blocks and their sizes. The target encoder controls what must be predicted at those positions. Two methods can use the same mask and differ because their target representations preserve different features. Conversely, two methods can share a target encoder and learn different relations because their masks sample different conditional problems.

Consider an image of a table with several objects. A nearby target block may be predictable from texture continuity. A target containing another object may require information about the broader scene. A target containing a small object near a boundary may require spatial precision that a global pooled representation would not preserve. I-JEPA does not guarantee that one of these properties will be learned. It creates a training signal in which the predictor estimates target features for sampled regions.

The target-block distribution should therefore be read as part of the model specification. The architecture is not only a pair of encoders and a predictor. It also includes a rule for deciding which information is visible and which target locations are queried.

4.3.7. What the experiments establish

The I-JEPA paper evaluates representations through downstream protocols including image classification, low-shot classification, object counting and depth estimation. These experiments test whether features learned from spatial prediction are useful beyond the pretraining loss. They are not measurements of future-state prediction or action-conditioned dynamics.

The paper also compares target-mask designs. Its multi-block sampling performs better than alternatives such as rasterized, single-block or random masking in the reported low-shot ImageNet setting. The conclusion is not that four blocks are universally optimal. It is that target geometry materially affects what the objective teaches.

Evaluation protocol matters. A frozen encoder with a linear classifier tests which distinctions are accessible to a linear readout. A denser or task-specific head can extract different information. Neither is a direct test of whether the representation preserves every variable required for a physical transition model.

I-JEPA is a clean image-representation result and a demonstration of spatial feature prediction. It is not an action-conditioned world model. It receives no intervention as input, and its target is sampled from an image rather than a controlled future trajectory.

4.3.8. What I-JEPA does not tell us

No time variable. The target and context belong to one image. The method is not trained to distinguish motion histories or predict the result of a later action.

No action variable. The predictor receives spatial query positions, not a chosen control input. It cannot answer what would happen under different candidate actions unless an extension supplies an action condition.

Target-relative information. The predictor matches features that the target encoder emits. A feature not retained by that encoder is not directly supervised.

No rollout requirement. The method is optimized for masked feature prediction, not recursive application of its own predictions. Good masked-region performance does not establish long-horizon consistency.

No planning objective. The loss does not specify which future state is desirable, how candidate actions should be compared, or how safety constraints should be enforced.

These points delimit the problem I-JEPA was designed to solve; they are not defects in that paper. Video prediction adds time to the target. Action-conditioned models add the intervention that distinguishes one possible future from another.

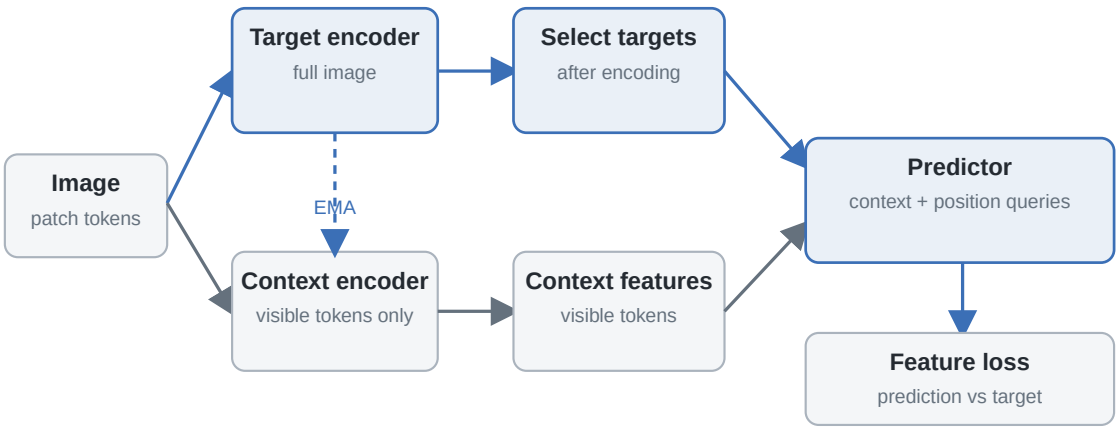


Figure 4 6. I-JEPA context and target encoder information paths

Figure 4.6. I-JEPA's two information paths. The target encoder processes the full image before target features are selected; the context encoder processes visible tokens only. Position-marked queries specify where the predictor must return features.

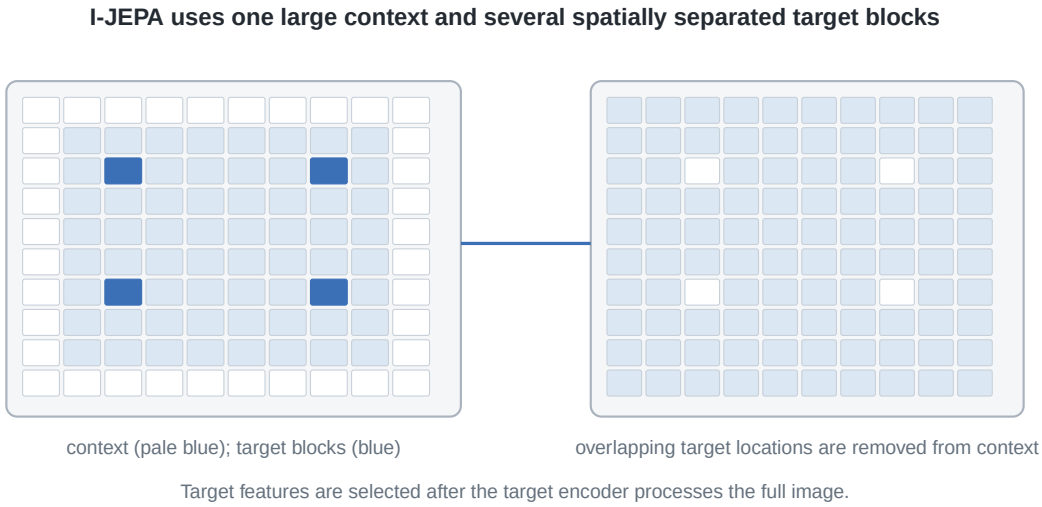


Figure 4 7. I-JEPA large context block and multiple target blocks

Figure 4.7. I-JEPA's context and target-block geometry. A large context block and multiple target blocks define the spatial prediction problem. Target positions overlapping the sampled context are removed from the context input.

The next step is to add time. A video representation can be indexed by both spatial and temporal position, so the target sampler can ask the model to predict features withheld from a sequence rather than from one image.

4.4. V-JEPA: predicting spatiotemporal features from video

An image contains spatial relationships, but it does not directly reveal how those relationships change. A single frame may show an object at a particular position without specifying whether it is moving, accelerating, being pushed or resting. Video adds temporal evidence. The challenge is to define a prediction problem that makes this evidence useful without requiring the model to reconstruct every pixel of every frame.

V-JEPA extends feature prediction from a spatial grid to a spatiotemporal grid. Its target positions have both spatial and temporal coordinates. The model observes part of a video clip and predicts target features at selected positions. This still does not make the objective action-conditioned. The video supplies evidence about what tends to happen; the model is not necessarily told which intervention caused the observed transition. (Bardes et al., 2024)

4.4.1. A video is a volume of tokens

Let a clip contain T frames:

$$\[X = (x_1, x_2, \ldots, x_T).\]$$

A video tokenizer converts the clip into spatiotemporal tokens. A token location can be indexed by

$$r = (t, h, w),$$

where t denotes temporal position and h, w spatial position. In the original V-JEPA configuration, the video tokenizer uses tubelets spanning two consecutive frames and a 16×16 spatial patch. A token is therefore a small spatiotemporal volume, not simply a patch from one independently processed frame.

The context encoder receives a selected subset of tokens. The target encoder computes target features for the clip, and the predictor estimates target features at selected spatiotemporal positions:

$$Z_C = f_\theta(X_C, C), \qquad H_Y = f_{\bar{\theta}}(X)_Y,$$

$$\widehat{H}_Y = g_\phi(Z_C, Q_Y),$$

where C is the visible set and Q_Y describes the requested target positions. The query includes time as well as space. A model can be asked to predict a feature at a later time, at a different spatial location, or both.

As in I-JEPA, the target features come from the target encoder's output at selected positions. The target is not an RGB crop. The objective is to predict a learned feature that summarizes the target pathway's representation of the clip.

4.4.2. Why temporal masking is different from image masking

Adjacent video frames are highly redundant. An object may move only a few pixels between consecutive frames. If the model sees nearly all of a frame or nearby frames, it may infer the target from trivial local continuity. A task that is too easy can reward a shortcut rather than a representation that captures useful temporal relations.

V-JEPA samples spatially contiguous regions and extends them across the temporal dimension of the clip. Its target masks therefore select spatiotemporal regions rather than independent random holes in individual frames. The paper uses two families of target masks. A short-range mask is the union of eight randomly sampled target blocks, each covering 15% of a frame; a long-range mask is the union of two blocks, each covering 70% of a frame. The aspect ratio of each sampled block lies between 0.75 and 1.5. The unions produce an average masking ratio of approximately 90%. The values 15% and 70% describe the spatial scale of the sampled blocks, not the fraction of the complete video removed by the final union. Both mask families extend each selected spatial block through the clip's full temporal dimension. (Bardes et al., 2024)

The encoder paths also matter. The visible complement is applied at the input of the context encoder by dropping masked tokens. The target encoder processes the complete clip; target positions are selected from its contextualized output. Thus, the model does not create a target by encoding an isolated RGB crop.

The geometry of a target tube matters. If the same spatial region is hidden across time, the model cannot simply inspect the corresponding patch in another nearby frame. It must use the rest of the clip to predict the target features. If a target is withheld at only one time but visible at neighbouring times, temporal interpolation may be sufficient. Both are valid prediction problems, but they test different conditional information.

V-JEPA keeps the core I-JEPA insight and changes the coordinate system of prediction. A target is no longer described only by its location in an image. Its time coordinate is also part of the query. The training distribution can emphasize spatial context, temporal context or their interaction.

4.4.3. The feature target and the loss

Let H_Y be target features at selected spatiotemporal positions and $\widehat{H}_Y$ the corresponding predictions. The original V-JEPA paper uses an L_1 feature-prediction loss:

$$\mathcal{L}_{\text{V-JEPA}} = \frac{1}{|Y|d} \sum_{r \in Y} \left\| \widehat{h}_r - h_r \right\|_1.$$

This differs from the squared-error form often used to introduce generic feature prediction. The discrepancy function changes the statistical estimator approached by an ideal deterministic predictor.

For a scalar random target H, the constant a minimizing expected absolute deviation,

$$a^* = \arg\min_a \mathbb{E}[|H - a| \mid C = c],$$

is a conditional median of H given the context. The constant minimizing expected squared error is the conditional mean. In multiple dimensions, an L_1 vector loss is typically applied coordinate-wise; its minimizers are coordinate-wise conditional medians, subject to possible non-uniqueness. This property should not be used to claim that V-JEPA explicitly models a full multimodal distribution. It describes the pointwise estimator favoured by the chosen loss under its assumptions.

The paper motivates its L_1 choice in the context of feature prediction and target-update dynamics. The predictor estimates target features from the context representation, while the target encoder is updated as a moving average. The EMA target provides a slowly changing feature target while the

online encoder and predictor are trained. This arrangement is specific to the original V-JEPA recipe.

4.4.4. Target updates and gradient paths

The original V-JEPA retains an online context encoder and an EMA target encoder. A stop-gradient is applied to target features in the prediction loss, so gradients from feature regression update the context encoder and predictor rather than directly updating the target branch. The target parameters change through the momentum update.

The training step can be described as follows:

1. Sample a clip and a spatiotemporal target mask.
2. Construct visible context tokens and target positions.
3. Encode visible tokens with the online context encoder.
4. Encode the clip with the target encoder and select target features.
5. Supply target positions as queries to the predictor.
6. Compute the L_1 discrepancy between predicted and target features.
7. Update the online encoder and predictor by gradient descent.
8. Update the target encoder using EMA.

This sequence makes the information paths explicit. The target branch supplies a feature target, not a gradient path through the prediction loss. The EMA update is a separate parameter update. The phrase ``stop-gradient teacher'' can obscure the distinction between detaching the target tensor in the loss and changing target parameters through a moving average.

Neither mechanism by itself proves that collapse is impossible for every objective and architecture. The result depends on the complete training system, including mask construction, data, optimizer, normalization, target update and predictor.

4.4.5. What time contributes to the representation

Video feature prediction can make temporal relations useful to the training objective. If a target feature depends on motion, persistence or an interaction visible in context, a predictor that ignores those relations may incur larger error. The model can therefore learn features that support motion understanding and temporal anticipation.

But temporal evidence is not the same as physical state. A feature that predicts the next visible frame need not encode velocity explicitly. A feature that supports action anticipation need not encode force, contact or mass. A feature that predicts what usually follows an observed action may

reflect correlations in the dataset rather than the effect of an intervention selected by the learner.

Consider a clip of a hand approaching a cup. The next observed frame may show the cup being grasped. A video predictor can learn that this visual pattern is often followed by a particular feature configuration. Without an explicit action variable and suitable intervention data, however, it does not necessarily learn what would happen if the hand moved away, approached from another direction or pushed the cup instead. Observed sequences mix environment dynamics with the policy that generated them.

The claim that video prediction can learn useful temporal structure follows from the objective. The stronger claim that video prediction alone teaches a causal model of how the world responds to actions does not.

4.4.6. Representation evaluation is not target prediction

A video encoder may produce a grid of feature vectors, but that does not mean every useful property is immediately available to a linear classifier. The original V-JEPA work evaluates frozen representations through downstream protocols, including video classification. One practical finding is that attentive pooling can improve substantially over simple average pooling for the unnormalized token features. A learnable query uses cross-attention to extract a task-specific representation from the feature grid.

This has a broader implication. The pretraining objective determines which relationships the model is optimized to preserve, but a downstream readout determines how those relationships are accessed. A linear probe tests whether a distinction is linearly accessible from frozen features. An attentive probe tests whether a small trainable attention mechanism can combine token features to expose the distinction. They are different tests.

The evaluation protocol must be stated alongside reported scores. A result obtained using an attentive head is not a pure measure of linear separability. A fine-tuned score is not a frozen-feature score. A feature-prediction loss is not a classification metric. These evidence types should not be mixed.

The original V-JEPA paper reports strong frozen-feature performance on video understanding benchmarks using large-scale video pretraining. These results establish that spatiotemporal feature prediction can yield useful visual representations under the tested protocols. They do not show that the model can plan under candidate actions or remain accurate over recursive rollouts.

4.4.7. What V-JEPA does not condition on

The phrase *future prediction* is sometimes used loosely in discussions of video models. A masked target can occur later than visible context, but temporal ordering is not enough to establish a

controlled transition model.

The original paper explicitly compares three masking regimes: random tubes, causal multi-block masks, and multi-block masks sampled across the entire clip. In the causal variant, the context is restricted to the first six or twelve frames of a sixteen-frame clip before spatiotemporal blocks are removed. The standard multi-block setting, by contrast, can use context from the whole clip, including frames later than a target position. In the reported frozen evaluation, the full-clip multi-block setting outperforms the causal-prefix alternatives on Kinetics-400, Something-Something-v2 and ImageNet-1K. The result is important for interpretation: the principal V-JEPA recipe is a strong masked-video representation objective, not simply a causal next-frame predictor. Temporal ordering exists in the data, but the mask does not impose a past-only information boundary. (Bardes et al., 2024)

To be action-conditioned, the predictor must receive a variable representing the candidate action or intervention. A controlled latent transition might be written as

$$\hat{s}_{t+1} = g_\phi(s_{\le t}, a_t),$$

where a_t identifies the action whose consequence is being predicted. If the action is absent, the model estimates what tends to occur under the trajectory distribution in its data. It may infer likely actions from visual motion, but that is not the same as conditioning on a chosen action.

This distinction matters when a planner compares alternatives. To evaluate actions a and a', the model must be able to produce different predictions when the action changes while the observed context is held fixed. An action-free model has no explicit input with which to represent this counterfactual difference.

The original V-JEPA is therefore a video representation-learning method, not the complete action-conditioned system used for physical planning. The next extension adds that interface as a separate post-training stage rather than simply renaming the original video encoder.

4.4.8. From video features to controlled dynamics

I-JEPA asks which features at hidden image locations can be predicted from visible spatial context. V-JEPA asks which features at hidden space-time locations can be predicted from visible video context. The model gains temporal evidence, and its target grid can express motion and interactions over a clip.

Neither method is required to distinguish outcomes under a specified action. That requires the model to receive actions, train on trajectories with aligned observations and actions, and be evaluated on action-conditioned predictions. The distinction is not terminological. It changes the conditional distribution the model must estimate.

V-JEPA 2 makes this separation explicit. It first scales action-free video pretraining, then adds a distinct action-conditioned predictor trained on robot trajectory data. V-JEPA 2.1 later strengthens dense feature prediction, but its robot-planning experiments again use a separately trained action-conditioned predictor. These stages will be treated separately rather than combined into one undifferentiated ``V-JEPA family".

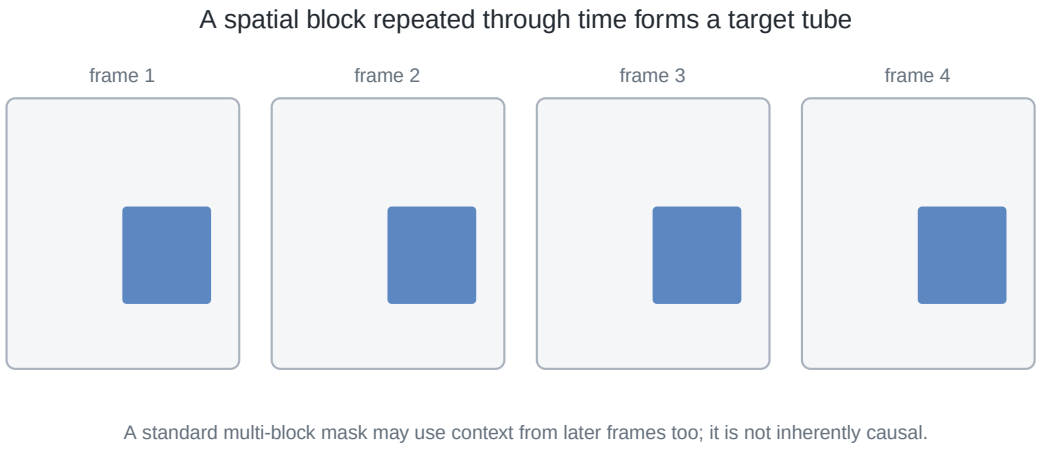


Figure 4 8. V-JEPA spatiotemporal target tube

Figure 4.8. A spatial mask extended through time. The same spatial selection can be withheld across several frames. Because visible context may include later frames, this standard multi-block mask is not inherently causal.

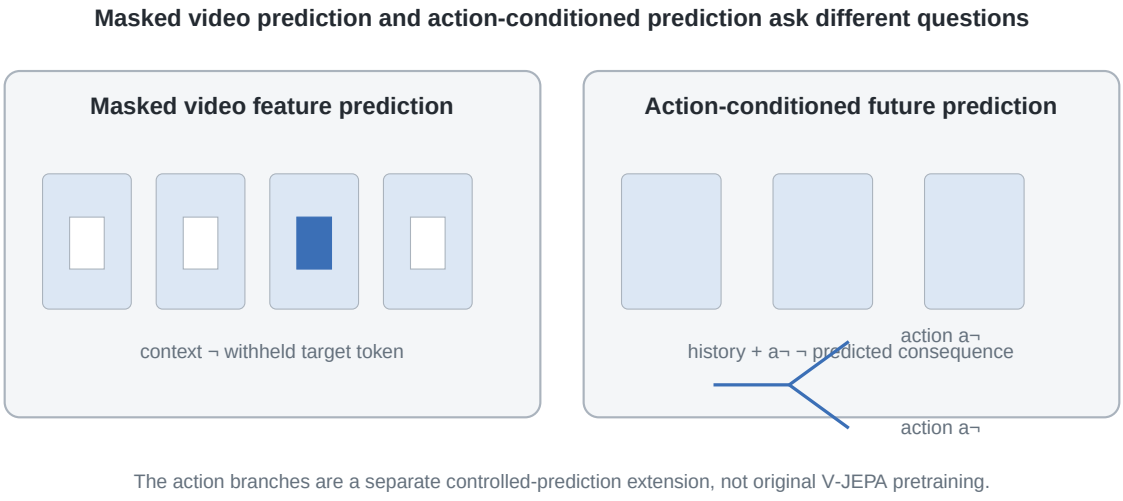


Figure 4 9. Masked video prediction and action-conditioned prediction

Figure 4.9. Masked video prediction versus action-conditioned prediction. Masked video

prediction asks for withheld features without an action input. A controlled predictor instead conditions alternative futures on candidate actions, a separate extension from original V-JEPA.

The next section examines what the target representation is allowed to forget, and how the geometry of the resulting features affects downstream estimation.

4.5. Why agreement alone is not enough

An objective that asks two representations to agree does not necessarily require either representation to contain information. The distinction is elementary, but it determines the design of non-contrastive self-supervised learning and joint-embedding prediction.

The collapse problem should be derived from the objective itself. It is not useful to describe it merely as an optimization instability or a nuisance that a particular architecture happens to avoid.

4.5.1. The constant solution

Let X and X' be two views of the same observation. Consider an encoder f_θ and a simple agreement loss,

$$\mathcal{L}(\theta) = \mathbb{E}\left[\|f_\theta(X) - f_\theta(X')\|_2^2\right].$$

Suppose the encoder ignores its input and returns the same vector c for every possible observation:

$$f_\theta(x) = c \qquad \text{for all } x.$$

Then

$$\mathcal{L}(\theta) = \mathbb{E}[\|c - c\|_2^2] = 0.$$

Because a squared distance cannot be negative, zero is a global minimum. The model has satisfied the objective perfectly while discarding all information about the input.

The argument does not depend on the particular value of c. It can be the zero vector or any constant vector. It also does not depend on the use of two augmented image views. Any objective that only rewards agreement between outputs can admit this solution when both outputs are allowed to become the same constant.

The same problem occurs in feature prediction. If the context and target encoders both map every input to c, and the predictor outputs c, then

$$\|g_\phi(f_\theta(x), q) - f_{\bar{\theta}}(y)\|_2^2 = \|c - c\|_2^2 = 0.$$

The feature prediction is perfect in the narrow sense measured by the objective. It is useless as a representation.

This is an identifiability problem: the training criterion does not distinguish an informative solution from a constant one. An optimizer cannot infer a requirement that the loss does not express.

4.5.2. Agreement and information are separate properties

The constant solution exposes two logically different requirements.

The first is **consistency**. Two views or branches that should describe the same underlying content should produce compatible representations.

The second is **non-degeneracy**. A representation should vary with the input in ways that preserve distinctions needed by the learning problem.

Consistency does not imply non-degeneracy. If every input maps to the same vector, all views agree. Non-degeneracy alone does not imply consistency either. Two branches can each produce varied embeddings while assigning different meanings to corresponding samples.

A useful objective therefore needs both kinds of pressure. In an abstract two-branch formulation,

$$\mathcal{L}(Z, Z') = \lambda\, s(Z, Z') + r(Z) + r(Z'),$$

where s measures agreement between corresponding views and r constrains the geometry of one branch independently. The expression is a decomposition of responsibilities, not a claim that one particular choice of r is universally sufficient.

This separation is the central motivation behind variance and distributional regularization. The cross-view term says which pairs should agree. The per-branch term says which embedding distributions are admissible.

4.5.3. Complete collapse and dimensional collapse

Complete collapse occurs when every input is mapped to the same vector. If a batch of centred embeddings is represented by a matrix $Z \in \mathbb{R}^{B \times D}$, then its covariance matrix is zero:

$$\hat{\Sigma}_Z = \frac{1}{B-1} Z^\top Z = 0$$

when the rows of Z are centred and identical before centring.

A less obvious failure occurs when the representation varies but occupies only a low-dimensional

subspace. Suppose the embedding dimension is D, but the centred embeddings have rank $r \ll D$. Then the covariance spectrum has at most r nonzero eigenvalues. The representation uses only a small fraction of its nominal dimensionality.

This is often called dimensional collapse or rank collapse. It does not mean every coordinate is constant. Several coordinates may vary, but they may be redundant linear combinations of the same few underlying directions.

A coordinate-wise variance floor can prevent individual dimensions from becoming constant, yet still allow correlated or low-rank structure. Conversely, penalizing covariance alone does not guarantee that every dimension has sufficient spread. The two requirements address different degeneracies. (Bardes et al., 2022; Zbontar et al., 2021)

4.5.4. The information-theoretic view

For a deterministic encoder $Z = f_\theta(X)$, complete collapse implies that Z is independent of X:

$$\[I(X; Z) = 0.\]$$

Here $I(X; Z)$ denotes mutual information. A constant representation contains no information about which input was observed. A low-rank representation may still contain information, but it can discard distinctions that matter to a downstream prediction problem.

It is tempting to say that the goal should be to maximize $I(X; Z)$ without qualification. That is not sufficient either. A representation that preserves every nuisance detail may be unnecessarily difficult to learn or use. Some transformations of the input should be treated as irrelevant for a particular task. The real requirement is task-relative: preserve information needed for the predictions and decisions that matter, while making the chosen invariances explicit.

This is why the target encoder matters. If it maps two observations to the same target feature, the predictive loss gives no reason to distinguish them. The objective cannot recover a distinction that the target representation has deliberately removed. That removal may be desirable for one task and destructive for another.

4.5.5. Why different anti-collapse mechanisms exist

Several families of methods modify the optimization problem so that the constant solution is no longer an attractive optimum or is avoided by the training dynamics.

Negative comparisons. Contrastive objectives compare a positive pair against negative examples. When every sample maps to the same normalized vector, positive and negative similarities become equal, so the model cannot minimize the contrastive objective by collapsing all samples. The mechanism depends on the negative distribution and on how false negatives, batch size and

sampling are handled.

Balanced assignments. Clustering-based approaches assign embeddings to learned prototypes. A balancing constraint discourages every sample from selecting the same prototype. This shifts the comparison from sample-to-sample similarities to sample-to-prototype assignments, and introduces its own assumptions about assignment balance.

Asymmetric target branches. Methods such as BYOL and I-JEPA use asymmetric training paths. A trainable online branch is matched to a target branch that does not receive the same gradient update. EMA updates can make the target change slowly. Stop-gradient changes the gradient path. A predictor can add a transformation between the online representation and the target. These choices can stabilize optimization, but one should not infer a universal theorem from their empirical success.

Moment constraints. Variance and covariance penalties add explicit constraints on batch statistics. A variance floor discourages constant coordinates; a covariance penalty discourages pairwise linear redundancy. They are direct properties of the batch distribution rather than implicit consequences of branch architecture.

Distributional constraints. Instead of matching only a few moments, a regularizer can compare the projected or full embedding distribution with a target distribution. This constrains a broader set of distributional properties, at the cost of introducing a target geometry and an approximation procedure.

These families should not be treated as interchangeable tricks. They make different mathematical objects part of the objective. Negative samples constrain relations among examples; balanced assignments constrain prototype usage; EMA changes the target dynamics; variance, covariance and distributional penalties constrain statistics of the embedding batch. (Chen et al., 2020; Caron et al., 2020; Grill et al., 2020; Assran et al., 2023; Balestriero and LeCun, 2025)

4.5.6. A moving target is not a proof of non-collapse

Consider an online parameter vector θ_t and an EMA target parameter vector ξ_t:

$$\xi_{t+1} = m\xi_t + (1-m)\theta_{t+1}, \qquad 0 \le m < 1.$$

Repeated substitution shows that the target is an exponentially weighted history of online parameters:

$$\xi_t = m^t\xi_0 + (1-m)\sum_{j=0}^{t-1} m^j\theta_{t-j}.$$

The target therefore changes more slowly than the online network, depending on the momentum schedule. This can stabilize a teacher-student optimization process. It does not, by itself, remove every constant solution from the mathematical objective. Whether the training system avoids collapse depends on the full objective, network, data, normalization and optimization dynamics.

The same care is needed with stop-gradient. It specifies which computational path receives a derivative. It is not a semantic constraint and does not guarantee that the representation contains the information required by an arbitrary task.

LeJEPA provides a useful counterexample to the idea that every JEPA needs an EMA teacher and predictor. Its basic multi-view objective directly compares embeddings to the mean of global views while regularizing their distribution. Conversely, action-conditioned prediction can require a predictor because the target depends on actions or future information unavailable in the context. The presence or absence of a predictor should follow from the information structure of the problem.

4.5.7. Non-collapse is necessary, not sufficient

A representation with nonzero variance can still omit object position, velocity, contact state or another variable needed by an agent. A representation with full-rank covariance can still be arranged so that the task-relevant variable is difficult for a downstream estimator to access. A Gaussian distribution can have the desired statistics without being a sufficient state for physical prediction.

The first objective is therefore modest: rule out degenerate solutions admitted by the training criterion. The next question is more demanding. Among non-degenerate representations, which ones make relevant downstream predictions easier, and under what assumptions can that statement be proved?

The answer requires treating the embedding distribution as a geometric and statistical object rather than judging it only by whether it has collapsed.

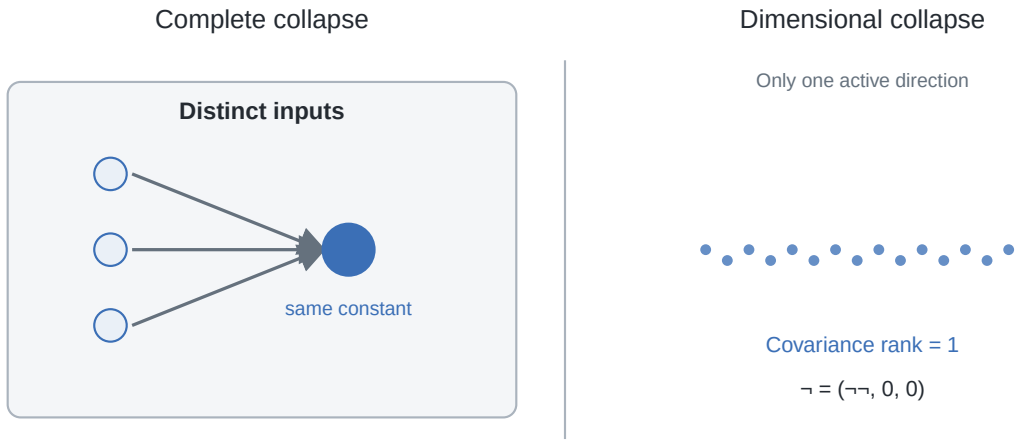


Figure 4 10. Complete collapse and rank-deficient representation

Figure 4.10. Complete collapse and dimensional collapse. A constant representation destroys all input information; a rank-deficient representation varies only along a subset of directions. Both can satisfy weak agreement objectives.

4.6. Predictive sufficiency and the geometry of a representation

Avoiding a constant representation does not establish that the model has learned a useful state. A representation can vary across observations and still merge two situations that have different consequences. The question is therefore not whether the embedding contains variation. It is whether it preserves the distinctions needed for the predictions and decisions under consideration.

4.6.1. A task-relative definition of a useful state

Let X denote the available history of observations, A an action or action sequence, Y a future quantity relevant to a task, and

$$\[Z = f_\theta(X)\]$$

the learned representation. The representation is sufficient for the chosen prediction problem when conditioning on Z and A preserves the conditional distribution of Y that was available from X and A:

$$\[P(Y \mid X, A) = P(Y \mid Z, A).\]$$

This equality is task-relative. The target Y, action domain, time horizon and data distribution must be specified. A representation sufficient for classifying an image need not be sufficient for

predicting the motion of an object. A representation sufficient for short-horizon prediction may fail at a longer horizon. A representation that predicts passive video may fail to predict the effect of a control input.

The action appears on both sides of the equality for a reason. A state representation used for control should retain enough information that the future distribution can be predicted when the action is known. If two histories map to the same Z but respond differently to the same action, the representation has merged states that the controller must distinguish.

4.6.2. Predictive equivalence between histories

The condition can be understood through an equivalence relation on histories. Fix a family of targets and a set of admissible actions. Define two histories x_1 and x_2 to be predictively equivalent when

$$x_1 \sim x_2 \iff P(Y \mid X = x_1, A = a) = P(Y \mid X = x_2, A = a)$$

for every relevant action a and target Y in the task specification.

A minimal sufficient predictive state would assign the same representation to histories in the same equivalence class and different representations when the relevant conditional future distributions differ. In practice, a neural representation may preserve more than the minimum needed, and the equivalence relation is only defined relative to the selected tasks and data.

This perspective clarifies why a representation should not be called a physical state simply because it is temporally predictive. A learned feature may support some future predictions without preserving every hidden variable, and the relevant variables can change with the action space and objective. (LeCun, 2022)

4.6.3. The target encoder defines which distinctions count

Suppose the target encoder maps two possible target observations to the same feature:

$$f_{\bar{\theta}}(y_1) = f_{\bar{\theta}}(y_2).$$

A predictor matching this target representation has no direct loss-based reason to distinguish y_1 from y_2. The two targets are equivalent under the feature space used for training. This can be an intentional invariance, for example when a change in illumination should not affect the representation. It can also be a failure if the difference corresponds to object position, contact, velocity or another variable that matters downstream.

The target representation is thus not a neutral measurement instrument. It defines the geometry in

which prediction errors are measured. The learned invariances of that target become the invariances of the predictive objective.

The same applies to dense and pooled representations. A globally pooled vector can be adequate for a scene-level classification target while discarding precise spatial correspondence. A dense token representation can preserve spatial structure, but its predictive objective may still fail to encode the variable required for a particular control task. Dense prediction changes the information being supervised; it does not certify control sufficiency.

4.6.4. Probes measure accessibility under a chosen decoder

A frozen representation is often evaluated by training a downstream predictor on top of it. The choice of predictor determines which notion of accessibility is measured.

A linear probe asks whether the target can be estimated by a linear function of the representation. It is useful because it isolates the geometry exposed by a simple readout. A nonlinear probe asks whether a more expressive function can extract the information. A nearest-neighbour or kernel probe asks whether local geometry supports useful predictions. A planning evaluation asks whether a model and a planner can use predicted states to select successful actions in an environment.

These tests answer different questions. A strong linear probe indicates that certain information is readily accessible to a linear readout. A weak linear probe does not prove that information is absent; a nonlinear readout may recover it. Strong representation benchmarks do not establish that the latent state is Markov, causal or sufficient for control. Planning success is evidence about the complete model-planner-environment system, not an encoder in isolation.

A useful evaluation therefore matches the readout to the claim. Representation quality, target-feature prediction, action-conditioned prediction and closed-loop control should not be collapsed into one leaderboard or one metric. (Balestriero and LeCun, 2025; Assran et al., 2025)

4.6.5. Covariance as a first view of embedding geometry

Let a batch of B embeddings in D dimensions be arranged as rows of a matrix

$$Z = \begin{bmatrix} z_1^\top \\ \vdots \\ z_B^\top \end{bmatrix} \in \mathbb{R}^{B \times D}.$$

Let $\bar{z}$ be the batch mean and Z_c the centred matrix. The empirical covariance is

$$\widehat{\Sigma} = \frac{1}{B-1} Z_c^\top Z_c.$$

The covariance matrix describes coordinate-wise variance and pairwise linear dependence. Its eigenvectors give principal directions of variation; its eigenvalues quantify the variance along those directions.

If the eigenvalues are all equal, the covariance is isotropic:

$$\widehat{\Sigma} = \sigma^2 I_D.$$

If a small number of eigenvalues dominate while others are close to zero, the embedding cloud is anisotropic or approximately low rank. The model may use only a small set of directions, even when every coordinate has some nonzero marginal variance.

Covariance is useful because it provides a tractable description of second-order geometry. It is not a description of the full distribution. Two distributions can share the same mean and covariance while differing in multimodality, tail behaviour or support. Zero covariance does not imply statistical independence, and a full-rank covariance matrix does not imply that the representation preserves the right semantic variables.

This limitation motivates the distinction between moment regularization and distributional regularization. Before introducing the latter, however, it is useful to understand why covariance geometry matters even for a simple downstream estimator.

4.6.6. Why the spectrum matters

Consider two embedding distributions with the same total variance,

$$\operatorname{tr}(\Sigma) = \sum_{k=1}^{D} \lambda_k.$$

An isotropic distribution spreads this variance equally across directions. An anisotropic distribution concentrates it unevenly. The total amount of variation can be identical while the weakest direction is much narrower in the anisotropic case.

A downstream estimator that relies on a narrow direction may be poorly conditioned. Regularization can shrink its coefficient strongly because the data provide little variation along that direction. Without regularization, small eigenvalues can amplify estimator variance. These are not merely visual differences between point clouds. They affect the statistical stability of downstream prediction.

The next section derives this dependence for ridge regression and ordinary least squares. The derivation is deliberately limited to those estimators and their assumptions. It is not a proof that

isotropic covariance is optimal for every nonlinear task or every controller.

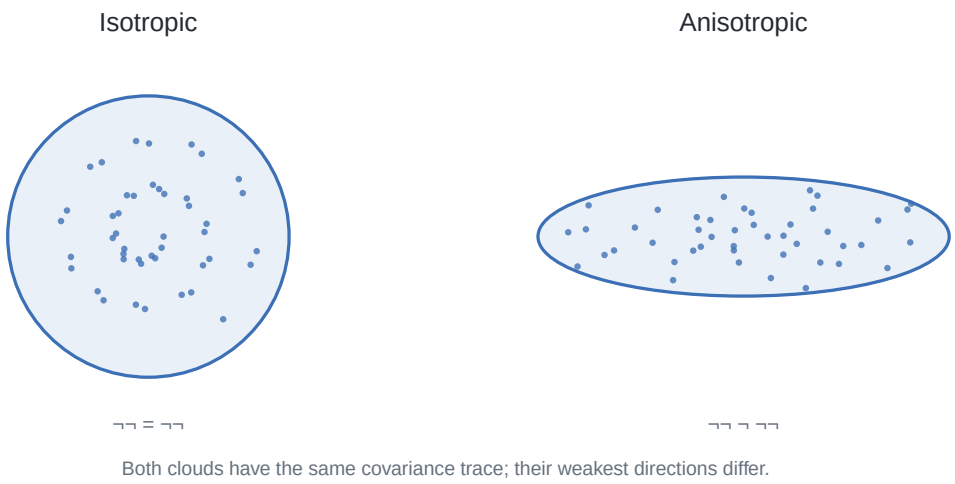


Figure 4 11. Isotropic and anisotropic embedding clouds with equal covariance trace

Figure 4.11. Equal trace does not imply equal embedding geometry. An isotropic and an anisotropic cloud can have the same covariance trace. The anisotropic representation has weak directions that can make downstream estimation sensitive.

4.7. Linear downstream risk and the covariance spectrum

A representation is often evaluated by freezing its encoder and fitting a small predictor on top of the resulting embeddings. This procedure isolates one question: how accessible is the information in the representation to a restricted downstream estimator? The simplest case is linear regression. Its mathematics makes the effect of embedding geometry explicit.

The aim of this section is not to prove that linear probes characterize every downstream task. It is to derive one concrete reason why a representation's covariance spectrum matters.

4.7.1. The frozen-feature problem

Suppose a frozen encoder maps observations to K-dimensional features. For a downstream dataset, arrange N feature vectors as rows of

$$\[X \in \mathbb{R}^{N \times K},\]$$

and collect the scalar targets in $y \in \mathbb{R}^{N}$. A linear probe predicts

$$\[\hat{y} = X\beta.\]$$

With a quadratic penalty on the coefficient vector, ridge regression solves

$$\widehat{\beta}_\lambda = \arg\min_{\beta \in \mathbb{R}^K} \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right\}, \qquad \lambda > 0.$$

Differentiating with respect to β and setting the gradient to zero gives

$$-2X^\top (y - X\widehat{\beta}_\lambda) + 2\lambda \widehat{\beta}_\lambda = 0.$$

Therefore

$$\widehat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y.$$

The learned coefficient vector depends on the geometry of the feature matrix through $X^\top X$. If the features are centred, this matrix is proportional to their empirical covariance. Its eigenvalues determine how much sample variation is available along each principal direction.

4.7.2. Ridge bias depends on the weak directions

Assume a linear downstream relationship

$$y = X\beta_{\text{true}} + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0.$$

Conditioning on X,

$$\begin{aligned} \mathbb{E}[\widehat{\beta}_\lambda \mid X] &= (X^\top X + \lambda I)^{-1} X^\top \mathbb{E}[y \mid X] \\ &= (X^\top X + \lambda I)^{-1} X^\top X \beta_{\text{true}}. \end{aligned}$$

Subtracting the true coefficient vector gives

$$\mathbf{Bias}(\widehat{\beta}_\lambda \mid X) = -\lambda (X^\top X + \lambda I)^{-1} \beta_{\text{true}}.$$

Let the eigendecomposition of $X^\top X$ be

$$X^\top X = Q \Lambda Q^\top, \qquad \Lambda = \mathbf{diag}(\lambda_1, \dots, \lambda_K),$$

with positive eigenvalues. In this basis, ridge shrinks the coefficient in direction q_k by the factor

$$\frac{\lambda_k}{\lambda_k + \lambda}.$$

Equivalently, the magnitude of the bias component in that direction is multiplied by

$$\frac{\lambda}{\lambda_k + \lambda}.$$

A direction with a large eigenvalue is well represented in the sample: the features vary substantially along it. Ridge therefore shrinks that coefficient less. A direction with a small eigenvalue is weakly supported by the data. Ridge shrinks its coefficient more strongly.

This observation permits a controlled comparison of two feature geometries. Hold the total covariance trace fixed, but allow its eigenvalues to be distributed differently. Let $\bar{\lambda}$ denote the mean eigenvalue. In the isotropic case, every eigenvalue equals $\bar{\lambda}$. In the anisotropic case, at least one eigenvalue is smaller than $\bar{\lambda}$.

Choose a downstream task whose true coefficient vector aligns with the eigenvector associated with the smallest anisotropic eigenvalue, denoted $\lambda_{\min}$. For this task,

$$\| \mathbf{Bias}_{\text{aniso}} \| = \frac{\lambda}{\lambda_{\min} + \lambda} \|\beta_{\text{true}}\|,$$

whereas for the isotropic geometry,

$$\| \mathbf{Bias}_{\text{iso}} \| = \frac{\lambda}{\bar{\lambda} + \lambda} \|\beta_{\text{true}}\|.$$

Because $\lambda_{\min} < \bar{\lambda}$,

$$\| \mathbf{Bias}_{\text{aniso}} \| > \| \mathbf{Bias}_{\text{iso}} \|.$$

The result is a worst-case statement. For any anisotropic covariance with the same total variance, there exists a downstream linear task aligned with its weakest eigen-direction for which ridge incurs greater bias than under the isotropic covariance. It does not say that every task aligned with every direction is worse. [Balestriero and LeCun, 2025]

4.7.3. OLS variance and the inverse spectrum

Ridge introduces bias by shrinking coefficients. To isolate estimator variance, set $\lambda = 0$. The resulting ordinary least-squares estimator is

$$\widehat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y,$$

assuming X has full column rank. Under the classical homoskedastic linear model with

$$\mathrm{Cov}(\varepsilon \mid X) = \sigma^2 I,$$

its conditional covariance is

$$\mathrm{Cov}(\widehat{\beta}_{\mathrm{OLS}} \mid X) = \sigma^2 (X^\top X)^{-1}.$$

The trace summarizes the total variance across coefficient directions:

$$\operatorname{tr} \mathrm{Cov}(\widehat{\beta}_{\mathrm{OLS}} \mid X) = \sigma^2 \operatorname{tr}\left((X^\top X)^{-1}\right) = \sigma^2 \sum_{k=1}^{K} \frac{1}{\lambda_k}.$$

A small eigenvalue becomes a large reciprocal. Consequently, a weak feature direction makes the fitted coefficient sensitive to changes in the sampled data.

Now hold total feature energy fixed:

$$\sum_{k=1}^{K} \lambda_k = T.$$

The function $f(u) = 1/u$ is strictly convex for $u > 0$. Jensen's inequality gives

$$\frac{1}{K} \sum_{k=1}^{K} \frac{1}{\lambda_k} \geq \frac{1}{\frac{1}{K} \sum_{k=1}^{K} \lambda_k} = \frac{K}{T}.$$

Thus,

$$\sum_{k=1}^{K} \frac{1}{\lambda_k} \geq \frac{K^2}{T},$$

with equality if and only if all eigenvalues are equal. Under the fixed-trace and full-rank assumptions, the isotropic covariance spectrum uniquely minimizes the trace of the OLS coefficient covariance. [Balestriero and LeCun, 2025]

This derivation explains why anisotropy can increase estimator variance even if the total amount of feature variance is unchanged. Concentrating variance into a few directions forces the remaining directions to become narrower. The inverse spectrum penalizes those narrow directions heavily.

4.7.4. What the linear result establishes

The two calculations isolate complementary effects:

- With positive ridge regularization, a task aligned with a weak covariance direction experiences stronger coefficient shrinkage and larger bias.
- With ordinary least squares and homoskedastic noise, the sum of coefficient variances is proportional to the sum of inverse eigenvalues and is minimized by an isotropic spectrum at fixed total variance.

These results rely on the linear model, the specified regularizer, the design assumptions and the comparison at fixed total variance. They describe the estimator's coefficients, not every possible measure of prediction error for an arbitrary nonlinear learner. In particular, they do not establish that a Gaussian feature distribution is optimal for all downstream tasks.

The linear analysis does establish a concrete design pressure: if a foundation representation must support unknown linear readouts, severe anisotropy can make some readouts poorly conditioned. Equalizing second-order variation removes this weakest-direction disadvantage under the model just analysed.

However, matching covariance does not determine the entire feature distribution. Two distributions can have identical covariance and very different local density structure. A nonlinear predictor that estimates a target by averaging nearby embeddings can be affected by that structure. The next section derives how the density gradient enters local prediction bias, and why the Fisher-information functional appears in the LeJEPA analysis.

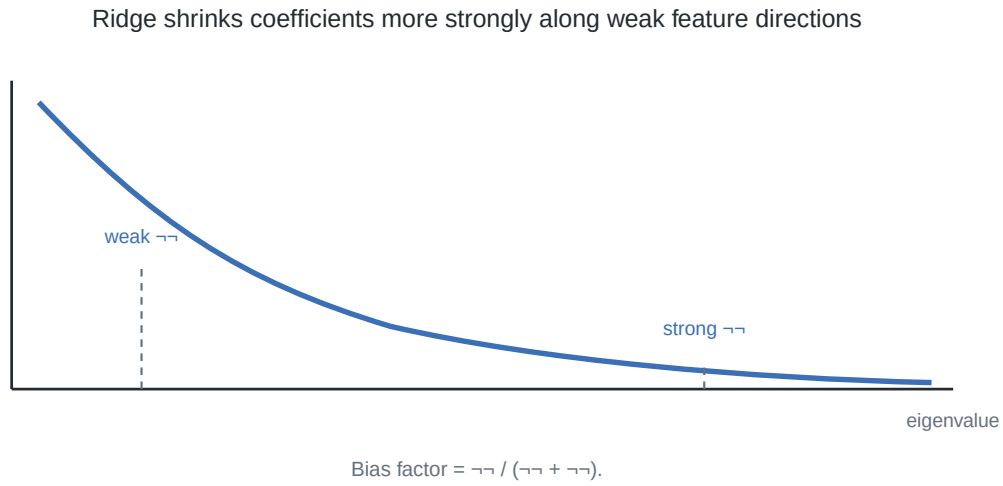


Figure 4 12. Ridge shrinkage as a function of feature eigenvalue

Figure 4.12. Ridge shrinkage depends on the feature eigenvalue. For fixed regularization, ridge shrinks coefficients more strongly along weak feature directions. The worst-conditioned task can

align with the smallest covariance eigenvalue.

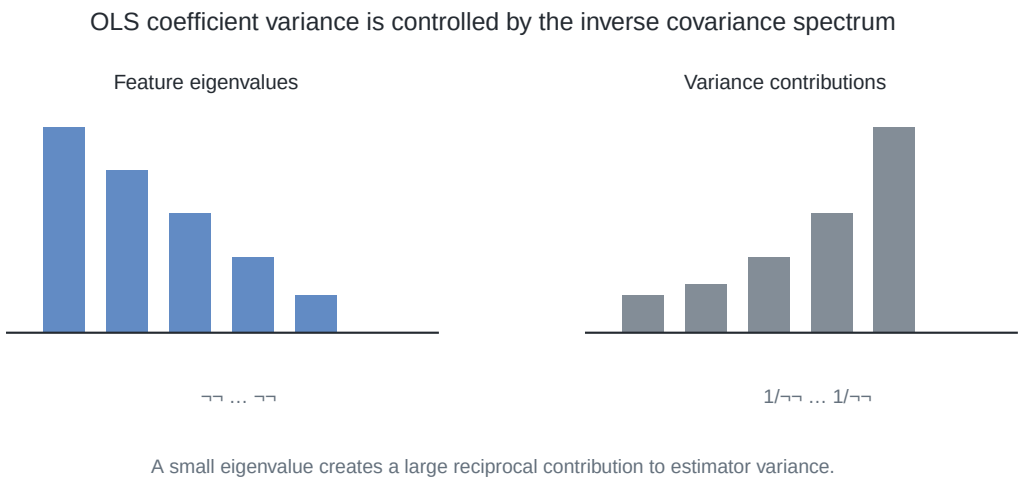


Figure 4 13. OLS variance contributions from inverse covariance eigenvalues

Figure 4.13. OLS variance is controlled by the inverse spectrum. Under full-rank design and homoskedastic noise, coefficient variance is proportional to the inverse covariance spectrum. Small eigenvalues therefore contribute disproportionately.

4.8. Nonlinear downstream risk and Fisher information

The covariance spectrum is only a second-order description of an embedding distribution. It does not determine how samples are arranged locally. This matters for nonlinear predictors that estimate a target by examining nearby embeddings. Their predictions depend not only on the target function, but also on where training samples are concentrated.

The key quantity is the score of the embedding density,

$$\[s_p(z) = \nabla \log p(z),\]$$

and the global statistic that measures its expected squared magnitude,

$$\[J(p) = \mathbb{E}_{Z\sim p}\left[\|\nabla \log p(Z)\|_2^2\right] = \int_{\mathbb{R}^K} \|\nabla \log p(z)\|_2^2 p(z)\, dz.\]$$

LeJEPA calls $J(p)$ the Fisher-information functional of the embedding density. It appears naturally when the bias of local nonparametric estimators is expanded and integrated over query points. The derivation explains why matching only mean and covariance leaves a gap.

4.8.1. A local estimator sees both the target and the sample density

Let $Z \in \mathbb{R}^K$ be a frozen embedding with density $p(z)$. Let the downstream regression function be

$$\eta(z) = \mathbb{E}[Y \mid Z = z].$$

A local estimator predicts $\eta(q)$ at a query point q by averaging labels from training embeddings near q. Two examples are radius-based k-nearest-neighbour regression and Nadaraya–Watson kernel regression.

For a fixed radius r_0, the radius-based estimator is

$$\widehat{\eta}_{\mathrm{kNN}}(q) = \frac{1}{|\mathcal{N}_{r_0}(q)|} \sum_{i \in \mathcal{N}_{r_0}(q)} Y_i, \qquad \mathcal{N}_{r_0}(q) = \{i : \|Z_i - q\|_2 \le r_0\}.$$

The number of neighbours is not fixed. It depends on the local density of embeddings. In a dense region, many training points fall inside the ball; in a sparse region, fewer do.

The Nadaraya–Watson estimator uses a smooth kernel:

$$\widehat{\eta}_h(q) = \frac{\sum_{i=1}^N K_h(q - Z_i) Y_i}{\sum_{i=1}^N K_h(q - Z_i)}, \qquad K_h(u) = h^{-K} K(u/h).$$

The kernel assigns higher weight to nearby samples. The bandwidth h controls the size of the neighbourhood. The denominator normalizes the empirical weights.

Both estimators average labels around the query, but they do so with different weighting profiles. Their bias depends on the shape of the target function and on how training points are distributed around the query.

4.8.2. Deriving the density-gradient term

Consider the large-sample conditional expectation of the kernel estimator:

$$\mathbb{E}[\widehat{\eta}_h(q)] \approx \frac{\int \eta(q+u) p(q+u) K_h(u)\, du}{\int p(q+u) K_h(u)\, du}.$$

The numerator contains the product $\eta(q+u)p(q+u)$. The denominator contains $p(q+u)$. To understand the ratio for small bandwidth, expand both functions around q.

For the regression function,

$$\eta(q+u) = \eta(q) + \nabla\eta(q)^\top u + \frac{1}{2}u^\top \nabla^2\eta(q)u + o(\|u\|^2).$$

For the embedding density,

$$p(q+u) = p(q) + \nabla p(q)^\top u + \frac{1}{2}u^\top \nabla^2 p(q)u + o(\|u\|^2).$$

Assume a normalized, symmetric, radially structured kernel with zero first moment and isotropic second moment:

$$\int K(v)\,dv = 1, \qquad \int vK(v)\,dv = 0, \qquad \int vv^\top K(v)\,dv = \mu_2(K)I_K.$$

The symmetry makes the first-order terms vanish after integration. The second-order terms contribute traces of the Hessians. Multiplying the expansions in the numerator also produces a cross term between the target gradient and the density gradient. After dividing by the denominator and retaining the leading order in h, the pointwise bias is

$$\mathrm{Bias}\left[\widehat{\eta}_h(q)\right] = \frac{h^2\mu_2(K)}{2}\left[\Delta\eta(q) + 2\nabla\eta(q)^\top \nabla \log p(q)\right] + o(h^2).$$

Here $\Delta\eta(q) = \mathrm{tr}(\nabla^2\eta(q))$ is the Laplacian of the downstream regression function.

The expression contains two distinct terms.

Target curvature. The Laplacian $\Delta\eta(q)$ measures the local curvature of the target function. Even if training points are distributed symmetrically around q, averaging nearby target values can differ from the target value at the centre when the function is curved.

Density-gradient coupling. The term $\nabla\eta(q)^\top \nabla \log p(q)$ measures the alignment between the direction in which the target changes and the direction in which embedding density increases relative to its local value. If nearby points are concentrated more heavily on one side of the query, the local average is pulled toward that side. When the target function changes along the same direction, the resulting average is systematically shifted.

The same density-gradient coupling appears for radius-based k-NN. Its leading bias is

$$\mathrm{Bias}\left[\widehat{\eta}_{\mathrm{kNN}}(q)\right] = \frac{r_0^2}{2(K+2)}\left[\Delta\eta(q) + 2\nabla\eta(q)^\top \nabla \log p(q)\right] + o(r_0^2).$$

The coefficients differ because a uniform ball and a smooth kernel have different second moments.

The dependence on the local score $\nabla \log p(q)$ is common to both estimators. The k-NN expansion assumes a smooth embedding density with bounded derivatives through order three, finite covariance and finite Fisher information, together with a twice continuously differentiable target function. [Balestriero and LeCun, 2025]

4.8.3. From a pointwise bias to Fisher information

A downstream representation is evaluated over a distribution of query points, not at one fixed q. Let queries follow the embedding density p. The integrated squared bias is

$$\mathrm{ISB} = \int \mathrm{Bias}(q)^2 p(q)\, dq.$$

The density-dependent part of the pointwise bias contains

$$\nabla \eta(q)^\top \nabla \log p(q).$$

To expose its contribution, consider the assumptions used in the k-NN analysis of LeJEPA. The density is sufficiently smooth, has bounded derivatives through the required order, finite covariance and finite Fisher information. The regression function is twice continuously differentiable. In addition, the downstream gradient is assumed to have mean zero and an isotropic second-moment matrix:

$$\mathbb{E}[\nabla \eta(Z)] = 0, \qquad \mathbb{E}[\nabla \eta(Z) \nabla \eta(Z)^\top] = \tau_g^2 I_K,$$

with $\tau_g^2 > 0$. Queries are drawn from p, and the embedding covariance is subject to a fixed scalar constraint.

Under these conditions, averaging the squared gradient coupling over downstream functions gives

$$\begin{aligned} \mathbb{E}_\eta \mathbb{E}_{Z\sim p}\left[\left(\nabla \eta(Z)^\top \nabla \log p(Z)\right)^2\right] &= \tau_g^2 \mathbb{E}_{Z\sim p}\left[\|\nabla \log p(Z)\|_2^2\right] \\ &= \tau_g^2 J(p). \end{aligned}$$

The functional

$$\boxed{J(p) = \int \|\nabla \log p(z)\|_2^2 p(z)\, dz}$$

is the expected squared score of the embedding density. It is a property of the density itself, not of

one coordinate variance or one covariance entry.

The full k-NN expansion also contains a curvature term and a cross term involving the target Hessian. The score-gradient term contributes $\tau_g^2 J(p)$. The curvature term is independent of the embedding density under the stated derivative prior. The cross term vanishes only under an additional orthogonality or zero-covariance condition between the Laplacian and gradient of the target function. Without that extra condition, it is a finite term of the same asymptotic order as the Fisher-information term, not a smaller-order remainder. The paper therefore writes the result as

$$\mathbb{E}_\eta[\mathrm{ISB}_{\mathrm{kNN}}] = \left(\frac{r_0^2}{K+2}\right)^2 \left\{\tau_g^2 J(p) + O(1)\right\} + o(r_0^4).$$

The expression isolates the part that varies with the embedding density. It does not say that the entire integrated bias equals the Fisher-information term for every downstream-function prior. The minimization over p follows because the remaining terms are bounded and do not provide a competing density-dependent contribution under the paper's assumptions. (Balestriero and LeCun, 2025)

For Nadaraya–Watson regression, the paper considers a smoothness class with bounded target gradient and Laplacian. If

$$\|\nabla \eta(q)\|_2 \le L, \qquad |\Delta \eta(q)| \le B,$$

then the integrated squared bias is bounded by

$$\mathrm{ISB}_{\mathrm{kernel}} \le \left(\frac{h^2 \mu_2(K)}{2}\right)^2 \left[2B^2 + 8L^2 J(p)\right] + o(h^4).$$

The kernel result additionally assumes a normalized, symmetric and isotropic kernel with finite second moment and finite roughness, a sufficiently light-tailed embedding density, and the usual small-bandwidth asymptotic regime. In particular, the pointwise expansion is stated where $p(q) > 0$, with $h \to 0$ and $Nh^K \to \infty$. The theorem bounds worst-case integrated squared bias over a smoothness class of target functions. These conditions delimit the result. [Balestriero and LeCun, 2025]

The pointwise variance also depends on the density:

$$\mathrm{Var}[\widehat{\eta}_h(q)] \approx \frac{R(K)}{Nh^K}\frac{v(q)}{p(q)}, \qquad R(K) = \int K(u)^2\, du,$$

where $v(q) = \mathrm{Var}(Y \mid Z = q)$. A query in a sparse part of the embedding distribution has fewer nearby training samples and therefore higher pointwise variance. But the paper evaluates the integrated variance under the same density used to sample queries. Multiplying by $p(q)$ and integrating cancels the density:

$$\begin{aligned}\mathcal{V}(h; p) &= \int \mathrm{Var}[\hat{\eta}_h(q)] p(q)\, dq \\ &\approx \frac{R(K)}{N h^K} \int v(q)\, dq.\end{aligned}$$

Under this evaluation measure, the leading integrated variance is independent of \(p\). The density-dependent optimization therefore enters through the integrated squared bias bound, while the leading integrated variance does not distinguish candidate embedding densities. This is a more precise statement than saying that every pointwise variance term is independent of the density.

The Fisher-information functional appears because the local estimators are biased by nonuniform sample density. The representation controls that density. For the estimator families considered, the density score is therefore part of the downstream prediction error.

4.8.4. Why an isotropic Gaussian minimizes the functional

The next step is a variational statement about probability densities. Let p be a mean-zero density on $\mathbb{R}^K$, with positive-definite covariance matrix

$$\Sigma = \mathbb{E}[ZZ^\top].$$

Under the regularity conditions needed for integration by parts and finite Fisher information, define the score

$$s_p(z) = \nabla \log p(z).$$

Because $s_p(z)p(z) = \nabla p(z)$, integration by parts gives

$$\mathbb{E}[Z s_p(Z)^\top] = \int z \nabla p(z)^\top dz = -I_K.$$

Now consider the positive-semidefinite matrix

$$\mathbb{E}\left[\left(s_p(Z) + \Sigma^{-1} Z\right)\left(s_p(Z) + \Sigma^{-1} Z\right)^\top\right].$$

Expanding the expectation and using $\mathbb{E}[s_p(Z)s_p(Z)^\top]$ for the Fisher-information matrix, together with $\mathbb{E}[ZZ^\top$=\Sigma], yields

$$\mathbb{E}[s_p(Z)s_p(Z)^\top] - \Sigma^{-1} \succeq 0.$$

Taking traces gives

$$J(p) \geq \operatorname{tr}(\Sigma^{-1}).$$

Equality requires

$$s_p(z) = -\Sigma^{-1}z$$

almost everywhere. Integrating this score identity gives a Gaussian density with covariance Σ:

$$p(z) = \mathcal{N}(0, \Sigma).$$

Thus, for a fixed covariance matrix, the Gaussian uniquely minimizes the Fisher-information functional.

A second optimization determines the covariance shape. Write the positive eigenvalues of Σ as $\lambda_1, \ldots, \lambda_K$. Then

$$\operatorname{tr}(\Sigma^{-1}) = \sum_{k=1}^{K} \frac{1}{\lambda_k}.$$

Under a fixed trace constraint

$$\sum_{k=1}^{K} \lambda_k = c,$$

strict convexity of $u \mapsto 1/u$ gives

$$\sum_{k=1}^{K} \frac{1}{\lambda_k} \geq \frac{K^2}{c},$$

with equality only when $\lambda_k = c/K$ for every k. The covariance must therefore be isotropic:

$$\Sigma = \frac{c}{K} I_K.$$

Combining the two steps gives the unique minimizer under fixed total variance:

$$p^*(z) = \mathcal{N}\left(0, \frac{c}{K} I_K\right).$$

LeJEPA also considers other scalar covariance constraints, including determinant, Frobenius norm and spectral radius. For the specific constraints analysed in the paper, the minimizer remains an isotropic Gaussian with the scale selected to satisfy the constraint.

4.8.5. What the result does and does not say

The argument has a clear chain:

1. Local prediction bias depends on the gradient of the downstream function and the score of the embedding density.
2. Under the stated smoothness and gradient assumptions, integrated bias contains the Fisher-information functional $J(p)$.
3. For a fixed covariance, the Gaussian uniquely minimizes $J(p)$.
4. Under a fixed scalar covariance constraint, isotropic covariance minimizes the Gaussian value $\mathrm{tr}(\Sigma^{-1})$.

This explains the Gaussian target in LeJEPA. It is not a theorem that Gaussian latents are best for every task, every estimator, every control problem or every conditional dynamics model. The result is derived for specified downstream estimator families and risk criteria. Its value is that it gives a principled target under those assumptions, rather than choosing a distribution only because it seems convenient.

It is also important to distinguish isotropy from Gaussianity. A distribution can have covariance proportional to the identity and still be non-Gaussian. Such a distribution matches second-order geometry but may have a different score field and a larger value of $J(p)$. Covariance regularization alone cannot enforce the density property that appears in the local-estimation risk.

The next section turns this theoretical target into a practical objective. SIGReg does not estimate a full high-dimensional density. It compares one-dimensional projections with the corresponding Gaussian projections. The distinction between the all-directions theorem and the finite stochastic sketch used during training is essential.

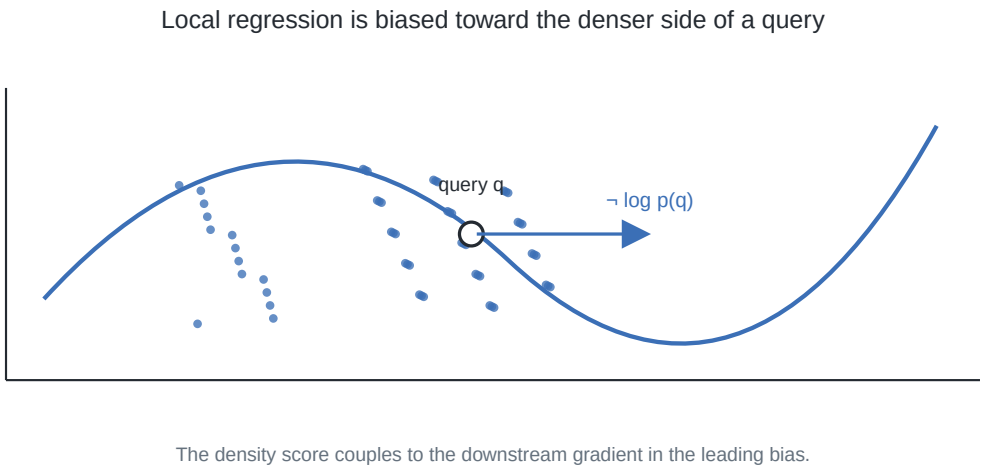


Figure 4 14. Local-regression query in an asymmetric sample density

Figure 4.14. Local regression is affected by density asymmetry. An asymmetric neighbourhood pulls a local estimate toward the denser side. The embedding-density score enters the leading local-regression bias.

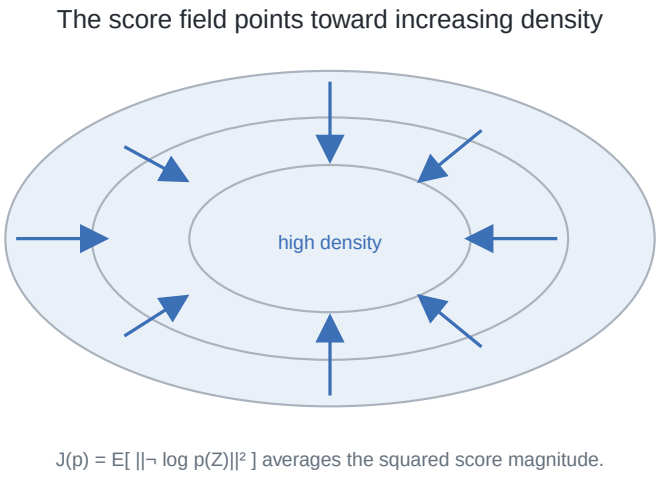


Figure 4 15. Fisher information score field over a density

Figure 4.15. Fisher information measures the squared score field. The score vector points toward increasing density. Its expected squared norm defines the Fisher-information functional used in the nonlinear-risk argument.

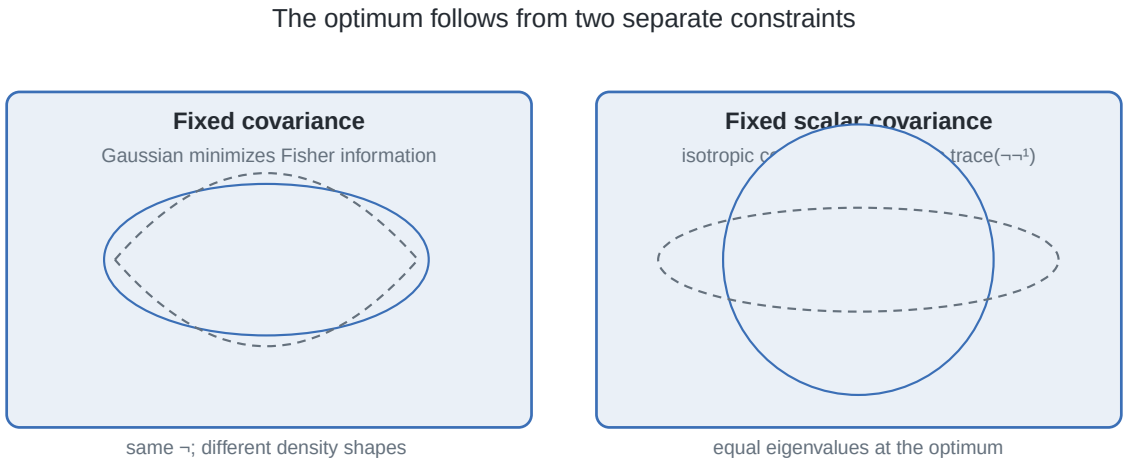


Figure 4 16. Gaussian and isotropic covariance optimality conditions

Figure 4.16. Gaussianity and isotropy arise from separate minimizations. At fixed covariance, the Gaussian minimizes Fisher information. Under a fixed scalar covariance constraint, isotropy minimizes the trace of the inverse covariance.

4.9. Distribution matching by projection: SIGReg

The previous section identified a target distribution through downstream prediction risk. Under the assumptions analysed there, the embedding distribution that minimizes the relevant risk criteria is an isotropic Gaussian. A training objective must now answer a computational question: how can a neural encoder be encouraged to match a high-dimensional distribution without estimating its density explicitly?

SIGReg, introduced in LeJEPA, replaces a multivariate goodness-of-fit problem with a collection of one-dimensional tests. The mathematical link is the Cramér–Wold theorem. The practical statistic is based on empirical characteristic functions and the Epps–Pulley test.

4.9.1. Why moments do not define the distribution

Suppose two embedding distributions have the same mean and covariance. They need not have the same shape. In one dimension, a standard normal random variable and a variable taking values -1 and $+1$ with equal probability both have mean zero and variance one. Their characteristic functions differ:

$$\varphi_{\text{normal}}(t) = e^{-t^2/2}, \qquad \varphi_{\text{two point}}(t) = \cos(t).$$

The first two moments therefore do not identify the distribution. Matching a finite number of

additional moments still does not identify a general probability distribution. For a particular target family, moment constraints may be useful; they are not a general certificate of distributional equality.

This is the limitation of covariance-based regularization when the theoretical target is a Gaussian density. Covariance controls second-order structure. It does not control all higher-order shape.

4.9.2. The characteristic function

For a scalar random variable U, its characteristic function is

$$\varphi_U(t) = \mathbb{E}[e^{itU}], \qquad t \in \mathbb{R}.$$

The characteristic function exists for every probability distribution because the complex exponential has unit magnitude. It uniquely determines the distribution of U.

For a standard normal random variable,

$$\varphi_0(t) = e^{-t^2/2}.$$

Given scalar samples $u_1, \dots, u_N$, the empirical characteristic function is

$$\widehat{\varphi}_N(t) = \frac{1}{N} \sum_{n=1}^{N} e^{itu_n}.$$

It is a finite average of complex exponentials. Unlike high-order moments, it does not require raising samples to increasingly large powers. It is differentiable with respect to each sample, and its sums can be aggregated across distributed workers. Its integrand is bounded, which avoids the polynomial growth that can make high-order moment objectives difficult to optimize.

A characteristic-function statistic still requires a frequency weighting and numerical integration. The choice of weight determines how discrepancies at different frequencies contribute to the result.

4.9.3. Cramér–Wold: why projections can identify a distribution

Let $Z \in \mathbb{R}^K$ be an embedding vector and let $a \in \mathbb{S}^{K-1}$ be a unit direction. The one-dimensional projection is

$$U_a = a^\top Z.$$

The Cramér–Wold theorem states that two probability distributions on $\mathbb{R}^K$ are equal if and only if all their one-dimensional linear projections have the same distribution:

$$a^\top X \overset{d}{=} a^\top Y \quad \text{for every } a \in \mathbb{S}^{K-1} \quad \iff \quad X \overset{d}{=} Y.$$

For an isotropic Gaussian target,

$$Z \sim \mathcal{N}(0, I_K) \quad \implies \quad a^\top Z \sim \mathcal{N}(0,1)$$

for every unit vector a. A high-dimensional Gaussian-matching problem can therefore be examined through scalar normality tests along directions.

The all-directions qualification is essential. Matching a finite set of projections is not, in general, sufficient to identify an arbitrary multivariate distribution exactly. A finite set may miss a structured discrepancy that lies between sampled directions. Increasing the number and diversity of projections improves the sketch, but the finite training objective remains a stochastic approximation to the all-directions condition.

The practical advantage is that a scalar test can be evaluated without estimating a full K-dimensional density. The projection matrix is the computational bridge between the embedding batch and these one-dimensional statistics.

4.9.4. A directional statistical test

Let P_θ denote the distribution of learned embeddings and $Q = \mathcal{N}(0, I_K)$ the target. For a unit direction a, the projected distributions are

$$P_\theta^{(a)} = (a^\top)_\# P_\theta, \qquad Q^{(a)} = (a^\top)_\# Q.$$

A one-dimensional goodness-of-fit statistic T compares samples from $P_\theta^{(a)}$ with the standard-normal target. Repeating the comparison over a set of directions

$$\mathcal{A} = \{a_1, \dots, a_M\}$$

produces directional evidence about the multivariate distribution.

The formal union-intersection test in LeJEPA aggregates directional statistics through a maximum:

$$T_{\mathcal{A}} = \max_{a \in \mathcal{A}} T\left(\{a^\top z_n\}_{n=1}^N\right).$$

The maximum is natural for a statistical test: a sufficiently large discrepancy in any tested direction is evidence against the null. The training regularizer takes a different form. SIGReg averages the directional statistics:

$$\mathcal{L}_{\text{SIGReg}} = \frac{1}{M} \sum_{m=1}^{M} T\left(\{a_m^\top z_n\}_{n=1}^{N}\right).$$

The average distributes gradient contributions across sampled directions. A maximum can concentrate the optimization signal on the current worst direction, leaving the other directions with no direct contribution at that step. The maximum and the average therefore serve different roles: the former is a test statistic, the latter a practical training objective. (Balestriero and LeCun, 2025)

4.9.5. The Epps–Pulley statistic

LeJEPA uses the Epps–Pulley test to compare empirical and target characteristic functions. For a projected sample $u_1, \ldots, u_N$, define

$$\widehat{\varphi}_N(t) = \frac{1}{N} \sum_{n=1}^{N} e^{itu_n}.$$

The Epps–Pulley statistic is

$$T_{\text{EP}} = N \int_{-\infty}^{\infty} \left|\widehat{\varphi}_N(t) - \varphi_0(t)\right|^2 w(t)\, dt, \qquad \varphi_0(t) = e^{-t^2/2}.$$

The paper writes the frequency window as

$$w(t) = e^{-t^2/\sigma^2}.$$

Its displayed Algorithm 1 evaluates the integral on 17 points over $[-5, 5]$, constructs the standard-normal characteristic function using $\exp(-t^2/2)$, and uses that same sampled Gaussian vector to weight the squared empirical-to-target discrepancy. The mathematical roles remain distinct: the target characteristic function specifies the null distribution, while $w(t)$ specifies frequency weighting. The paper's equation and algorithm should therefore be quoted at their respective levels rather than inferring a bandwidth setting that is not explicit in the implementation listing.

The statistic compares the empirical characteristic function with the target over a range of frequencies. It is not a mean-and-variance penalty. Deviations in the shape of the projected distribution can affect the characteristic function even when the first two moments match.

The implementation approximates the integral numerically. Its pseudocode samples 17 points on the interval $[-5, 5]$, evaluates the Gaussian characteristic function $e^{-t^2/2}$, multiplies the squared empirical-to-target discrepancy by that Gaussian vector, and applies trapezoidal quadrature. The paper notes that symmetry of the integrand can be used to double the effective number of knots without an equivalent increase in computation. This is an implementation prescription, not a mathematical requirement of the Epps–Pulley test. The paper's general test definition writes the Gaussian window as $w(t) = e^{-t^2/\sigma^2}$, commonly with $\sigma = 1$; the displayed code uses $e^{-t^2/2}$ as its numerical window. The equation and code should therefore be read in their own stated conventions rather than silently identifying their window parameterizations. (Balestriero and LeCun, 2025)

4.9.6. From one projection to a minibatch objective

For a batch of embeddings $z_1, \ldots, z_N$, sample unit directions by drawing Gaussian vectors and normalizing them:

$$a_m = \frac{g_m}{\|g_m\|_2}, \qquad g_m \sim \mathcal{N}(0, I_K).$$

For each direction, project every embedding:

$$u_{n,m} = a_m^\top z_n.$$

Compute the one-dimensional Epps–Pulley statistic for the N projected samples, then average over directions:

$$\mathcal{L}_{\text{SIGReg}}(Z) = \frac{1}{M} \sum_{m=1}^{M} T_{\text{EP}}(u_{1,m}, \ldots, u_{N,m}).$$

The directions are resampled during training. The reference implementation seeds the direction sampler from the global training step so that distributed workers use synchronized projections. Each minibatch therefore sees a new sketch of the same embedding distribution. The cumulative number of directions sampled over training can be much larger than the number evaluated in any one minibatch.

This resampling has an important consequence. A small number of directions in one step does not mean the model is constrained to those same directions throughout training. LeJEPA reports that resampling as few as 16 directions per minibatch can outperform optimizing against a fixed set containing thousands of directions, because the sampled directions accumulate over training. This is an empirical property of the optimization setup, not an exact finite-step Cramér–Wold guarantee.

(Balestriero and LeCun, 2025)

The projection and characteristic-function operations can be written as matrix multiplication followed by complex-exponential evaluations and numerical integration. With the embedding dimension, number of slices and number of quadrature points fixed, the reference implementation is linear in minibatch size N. More generally, the work depends on N, embedding dimension K, number of directions M, and number of integration points. The paper's $\mathcal{O}(N)$ statement refers to scaling in batch size for fixed sketch parameters; it should not be read as independence from projection count or feature dimension.

For distributed training, each worker computes local sums of complex exponentials and combines them with an all-reduce operation. This makes the empirical characteristic function compatible with data-parallel training without gathering all projected samples on one device.

4.9.7. Stability and finite-batch effects

The empirical characteristic function is an average of bounded complex values. The Epps–Pulley construction therefore avoids the polynomial growth of high-order moments. LeJEPA derives bounds on the statistic's gradient and curvature with respect to projected samples. By the chain rule, this supplies a controlled contribution to the encoder gradient for the statistic under analysis.

This is a statement about the Epps–Pulley statistic and its inputs. It does not imply that the gradient of the complete neural network objective is bounded independently of the network architecture, parameterization or all optimization details.

A finite minibatch also introduces bias in the statistic and its gradient. The paper derives an $\mathcal{O}(1/N)$ term and reports that it is small in the tested settings, including minibatches as small as 16. U-statistic debiasing or sample splitting could provide unbiased alternatives, but those are not part of the reference training procedure. The training loss is thus a finite-sample estimate, not an exact population test. (Balestriero and LeCun, 2025)

There is a related statistical caution. In classical hypothesis testing, failing to reject a null hypothesis does not prove that the null is true. It says only that the observed sample did not supply sufficient evidence against it at the selected test level. Training minimizes a regularizer rather than making a one-shot accept/reject decision, but the logical distinction remains: a small finite-sample projection statistic is evidence of approximate matching under the sampled directions and frequencies, not a certificate of exact population Gaussianity.

4.9.8. Embedding SIGReg inside a predictive objective

SIGReg specifies a distributional constraint. It does not replace the task that tells the encoder which views should be related. LeJEPA combines the two.

For each observation, let $z_{n,v}$ be the embedding of view v. Let the first V_g views be global views, and define their mean

$$\mu_n = \frac{1}{V_g} \sum_{v=1}^{V_g} z_{n,v}.$$

A view-agreement term draws every view embedding toward this global-view centre:

$$\mathcal{L}_{\text{pred}} = \frac{1}{NV} \sum_{n=1}^{N} \sum_{v=1}^{V} \|\mu_n - z_{n,v}\|_2^2.$$

SIGReg is applied to the embedding distribution of the views. The total objective is

$$\mathcal{L}_{\text{LeJEPA}} = (1-\alpha)\mathcal{L}_{\text{pred}} + \alpha \mathcal{L}_{\text{SIGReg}},$$

where α is the single trade-off parameter between view agreement and distribution matching.

This construction differs from an EMA teacher-predictor system. In the base LeJEPA recipe, the mean of the global-view embeddings provides the centre toward which all views are drawn. The method does not require an EMA target encoder, a stop-gradient target branch or a separate predictor network. SIGReg is applied separately to the batch of embeddings from each view and then averaged over the V views. The complete objective can therefore be written as

$$\mathcal{L}_{\text{LeJEPA}} = (1-\alpha)\mathcal{L}_{\text{pred}} + \frac{\alpha}{V} \sum_{v=1}^{V} \text{SIGReg}\left(\{z_{n,v}\}_{n=1}^{N}\right).$$

Here $\mathcal{L}_{\text{pred}}$ averages the squared distance between every view and the per-sample mean of the global-view embeddings. The outer coefficient α is the single trade-off hyperparameter in the method's stated objective. SIGReg does not regularize one pooled collection formed by mixing all views into a single batch; each view distribution is tested separately. This distinction is relevant because crop scales and augmentation distributions can differ between global and local views. (Balestriero and LeCun, 2025)

4.9.9. What SIGReg makes possible

The logical contribution of SIGReg is a practical route from a high-dimensional distributional target to differentiable one-dimensional tests:

1. A downstream-risk analysis motivates a target distribution.

2. Cramér–Wold shows that all one-dimensional projections identify the multivariate distribution.
3. Random unit directions produce a stochastic sketch of those projections.
4. The empirical characteristic function represents each projected distribution.
5. Epps–Pulley measures its discrepancy from the Gaussian target.
6. Numerical quadrature and batch aggregation make the statistic usable in gradient-based training.

Each step introduces an approximation or assumption. The Gaussian target is justified by the risk analysis, not by Cramér–Wold. Cramér–Wold is an all-directions result, not a finite-sample guarantee. Random projections reduce computation but do not remove the need to examine the number of directions and batch size. Epps–Pulley provides a stable statistic for the projected distributions, but the numerical integral is still approximated.

SIGReg therefore controls more than the first two moments, but it does so by sampling and testing one-dimensional views of the embedding distribution. The next section considers a different decomposition of the same distributional problem: control the centre and scale explicitly, then compare shape after the scale has been removed.

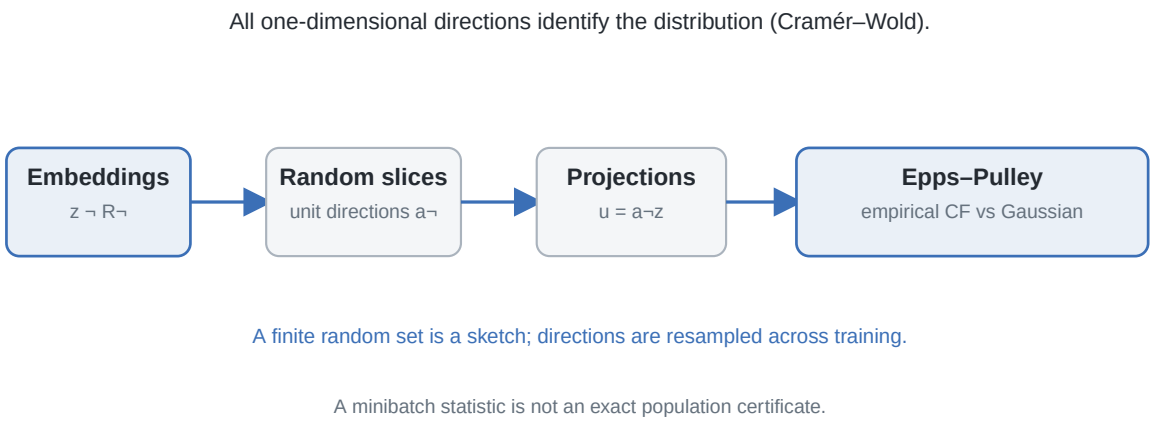


Figure 4 17. Cramér Wold projections and SIGReg sketch

Figure 4.17. All directions identify a distribution; sampled directions form a sketch. Cramér–Wold characterizes a distribution through all one-dimensional projections. SIGReg uses sampled directions in each minibatch and accumulates coverage across training.

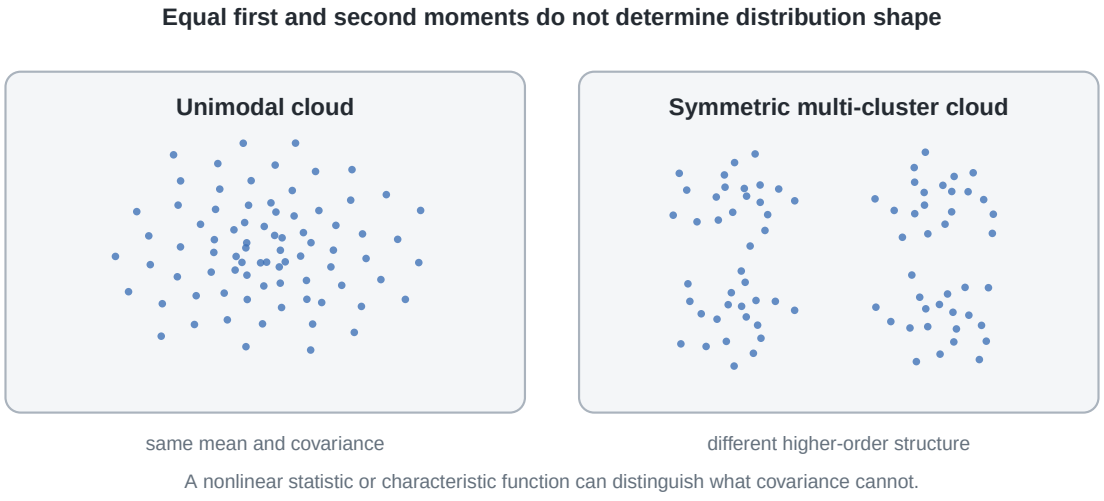


Figure 4 18. Two distributions with equal mean and covariance but different shapes

Figure 4.18. Matching the first two moments does not fix distribution shape. Two distributions can share a mean and covariance yet have different shapes. A projected characteristic function can reveal differences that covariance alone cannot detect.

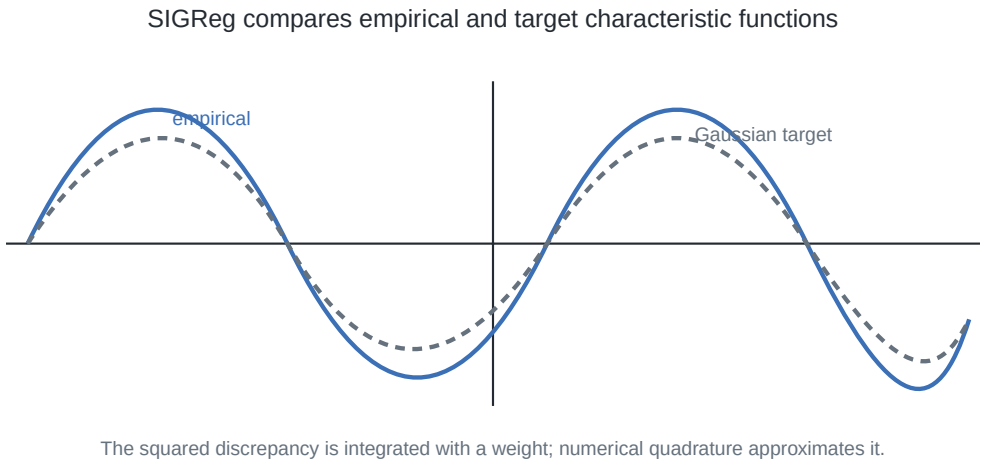


Figure 4 19. Empirical and Gaussian characteristic functions

Figure 4.19. The Epps–Pulley statistic compares characteristic functions. SIGReg compares empirical and Gaussian characteristic functions across frequency and integrates the squared discrepancy with a weight. A finite quadrature grid approximates the integral.

4.10. Separating scale and shape: VISReg

SIGReg compares projected distributions with a Gaussian target through characteristic functions. VISReg addresses the same distributional problem with a different decomposition. It separates the

scale of the embedding coordinates from the shape of the normalized batch, and it allows the three regularization components to be reweighted independently.

The purpose of this construction is not merely to add another anti-collapse penalty. It gives the optimizer two distinct jobs: maintain usable coordinate scale, and shape the normalized embedding distribution toward an isotropic Gaussian.

4.10.1. Three different properties of a batch

Let the embeddings of a batch be $Z \in \mathbb{R}^{N \times D}$, with N samples and D feature dimensions. Define the batch mean and centered embeddings:

$$\mu = \frac{1}{N} \sum_{n=1}^{N} z_n, \qquad \widehat{Z} = Z - \mu.$$

Three distinct properties can now be considered.

Centre. Is the embedding cloud centred at the origin? A nonzero mean shifts every projection, even if the covariance and higher-order shape are otherwise acceptable.

Scale. How much variation does each coordinate have? If a representation contracts toward the origin, its standard deviations become small. If some coordinates expand excessively, they can dominate the embedding geometry.

Shape. After the mean and scale are accounted for, does the distribution resemble the desired shape? A batch can have unit coordinate-wise variance and identity covariance while remaining non-Gaussian.

These properties should not be treated as one statistic. A variance term does not identify the full distribution. A distributional shape test that normalizes away scale does not, by itself, require the unnormalized embedding cloud to have unit variance.

4.10.2. The scale term

Let $\sigma_j(\widehat{Z})$ denote the batch standard deviation of coordinate j. VISReg uses the scale objective

$$\mathcal{L}_{\text{scale}} = \frac{1}{D} \sum_{j=1}^{D} \left(1 - \sigma_j(\widehat{Z})\right)^2.$$

The target standard deviation is one in each coordinate. The term penalizes both contraction below unit scale and expansion above it. Its minimum is attained when every coordinate has unit batch

standard deviation.

The scalar derivative for coordinate j is

$$\frac{\partial \mathcal{L}_{\text{scale}}}{\partial \sigma_j} = -\frac{2}{D}(1 - \sigma_j).$$

At zero standard deviation, this derivative remains finite and nonzero. The paper contrasts this scale signal with SIGReg's characteristic-function gradient near collapsed embeddings, which diminishes in the collapse regime. The authors report controlled gradient comparisons in which the VISReg scale term supplies a corrective signal under the tested setup. This is not a universal lower bound on every encoder-parameter gradient: the chain rule through the batch standard deviation and the network still depends on the representation, architecture and numerical implementation. (Wu et al., 2026)

4.10.3. Removing scale from the shape comparison

VISReg normalizes the centered batch coordinate-wise:

$$\tilde{Z} = \frac{\hat{Z}}{\mathrm{sg}(\sigma(\hat{Z})) + \epsilon},$$

where $\epsilon > 0$ stabilizes the denominator and sg denotes stop-gradient.

The normalization removes coordinate scale before the shape comparison. The stop-gradient means that the shape term does not differentiate through the standard-deviation denominator. The scale objective remains the direct mechanism for adjusting the unnormalized coordinate spread, while the shape statistic compares normalized samples.

This gradient boundary is not the same as an EMA target update. An EMA makes a target encoder follow the online encoder slowly. Here, stop-gradient is applied to the scale statistic used in normalization. It changes which derivatives the shape objective contributes. The two mechanisms should not be described as the same kind of teacher-student asymmetry.

The normalization also does not imply that the scale and shape objectives are interchangeable. The scale term acts on the unnormalized centered coordinates. The shape term acts on the normalized batch. Their separate coefficients allow the training objective to assign different importance to the two requirements.

4.10.4. Sliced Wasserstein distance

The shape term compares projected normalized embeddings with projections of a standard

Gaussian. The one-dimensional sorting operation is piecewise differentiable with respect to the projected values except at ties, where the ordering changes; the loss does not require a differentiable relaxation of a high-dimensional sorting operation. For a unit direction $w_k \in \mathbb{S}^{D-1}$, form the scalar samples

$$u_k = \tilde{Z} w_k \in \mathbb{R}^N.$$

Let $u_{k,(1)} \le \cdots \le u_{k,(N)}$ denote the sorted projected values. Define the standard-normal reference quantiles by

$$r_i = \frac{i}{N+1}, \qquad q_i = \Phi^{-1}(r_i), \qquad i = 1, \ldots, N,$$

where Φ is the standard-normal cumulative distribution function.

For two equally weighted empirical distributions on the real line with the same number of samples, the optimal quadratic transport coupling matches their order statistics. Thus

$$W_2^2(\hat{P}, \hat{Q}) = \frac{1}{N} \sum_{i=1}^{N} \left(u_{(i)} - q_i\right)^2.$$

The factor $1/N$ is part of the empirical one-dimensional Wasserstein distance. The paper's displayed shape objective is

$$\mathcal{L}_{\text{shape}} = \frac{1}{K_s} \sum_{k=1}^{K_s} \left\| \operatorname{sort}(\tilde{Z} w_k) - q \right\|_2^2,$$

where K_s is the number of slices. The accompanying Algorithm 1, however, computes `(p_sorted - target).pow(2).mean()`, which averages over both the N samples and the K_s slices. The code therefore includes a factor $1/N$ relative to the displayed squared-norm expression. This is a normalization discrepancy between the printed objective and the pseudocode, not a difference in the intended transport coupling. It changes the numerical scale of the regularizer with batch size and must be resolved against the exact implementation when reproducing loss weights or comparing reported coefficients.

The paper's normalization equation also shows an ϵ in the denominator, whereas the printed algorithm excerpt divides by the detached standard deviation directly. Implementations should use the stabilizer appropriate to the precision and numerical regime, and document that choice. The conceptual construction is unchanged: standardize each coordinate using a detached scale statistic,

then compare the sorted one-dimensional projections with Gaussian quantiles.

The difference between SIGReg and VISReg lies in the one-dimensional statistic. SIGReg compares empirical characteristic functions with the target characteristic function. VISReg sorts projected samples and compares their empirical quantiles with Gaussian quantiles. Both use random projections, but they do not compute the same discrepancy. (Wu et al., 2026)

4.10.5. Centre, scale and shape in one objective

The centering term is

$$\mathcal{L}_{\text{center}} = \|\mu\|_2^2.$$

VISReg assigns separate weights to its three regularization components:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{scale}}\mathcal{L}_{\text{scale}} + \lambda_{\text{shape}}\mathcal{L}_{\text{shape}} + \lambda_{\text{center}}\mathcal{L}_{\text{center}}.$$

The method also retains a view-agreement signal. Let $z_{n,v}$ denote the embedding of sample n in view v, with V_g global views among V total views. Define the centre of the global-view embeddings as

$$\mu_{g,n} = \frac{1}{V_g}\sum_{v=1}^{V_g} z_{n,v}.$$

A view-invariance term draws each view embedding toward that global-view mean:

$$\mathcal{L}_{\text{pred}} = \frac{1}{NV}\sum_{n=1}^{N}\sum_{v=1}^{V} \|\mu_{g,n} - z_{n,v}\|_2^2.$$

The complete loss is

$$\mathcal{L}_{\text{VISReg}} = (1-\lambda)\mathcal{L}_{\text{pred}} + \lambda\mathcal{L}_{\text{reg}}.$$

There are therefore four loss weights in the formulation: the outer λ, and the three component weights inside $\mathcal{L}_{\text{reg}}$. The default component weights are one in the stated baseline; the authors ablate and reweight them, finding that increasing the shape weight can help on low-quality datasets. They also report different useful outer λ values for small and large datasets. These are optimization choices, not new terms in the objective. (Wu et al., 2026)

4.10.6. Computational scaling and distributed slices

The regularization computation projects the $N \times D$ batch onto K_s random directions and sorts the N scalar values in each slice. Its two dominant operations are

$$\[O(NDK_s) + O(K_s N \log N).\]$$

Because $\log N \ll D$ at the scales considered, the paper treats the effective regularization complexity as $O(NDK_s)$. This is linear in batch size, embedding dimension and number of slices when those are treated as separate scaling parameters.

The number of slices needed for a good approximation can, however, grow with embedding dimension. The authors show empirically that a single-GPU setting often needs $K_s = CD$ for some factor $C > 1$, which makes the effective projection cost approximately $O(CND^2)$. They address this by generating independent slices across GPUs. With M devices, each device can process a fraction of the total slice set, and the partial losses can be aggregated. Their experiments show that increasing the number of devices can compensate for a smaller number of slices per device. The paper therefore argues that K_s can remain fixed per device while scaling the training setup. This is an empirical scaling strategy, not a theorem that a fixed finite set of directions identifies every distribution. (Wu et al., 2026)

4.10.7. What the experiments support

VISReg evaluates its regularizer through ablations, low-quality and long-tailed data, low-rank data, linear evaluation, transfer learning, out-of-distribution performance, dense prediction and image-generation guidance. The paper reports that the shape weight can be increased for low-quality datasets, that VISReg remains comparatively robust at lower slice counts, and that distributing slices across multiple GPUs can compensate for an insufficient per-device projection count.

For downstream evaluation, the authors report that an ImageNet-22K-pretrained VISReg ViT-L/14 reaches an average score of 72.94 across their selected downstream datasets, compared with 72.93 for DINOv2-LVD142M in the reported table. This is a matched aggregate on the listed benchmarks, not evidence that the two methods are equivalent on every task. The paper also reports competitive OOD performance from ImageNet-1K pretraining and strong robustness in long-tailed and low-rank settings. (Wu et al., 2026)

These experiments support the proposed decomposition under the paper's architectures, data and optimization settings. They are not a theorem that every parameterization recovers from every collapsed initialization. A controlled gradient comparison does not prove that every network can recover from every degenerate state. An OOD score on a particular benchmark set does not establish universal superiority or task sufficiency.

4.10.8. The broader design principle

The sequence from covariance penalties to SIGReg and VISReg is not simply a progression of increasingly elaborate anti-collapse tricks.

A variance floor constrains marginal spread. A covariance penalty constrains pairwise linear redundancy. SIGReg compares projected characteristic functions with a Gaussian target. VISReg separates the correction of centre and scale from a sliced-Wasserstein shape comparison. Each objective introduces a different statistical condition.

None of these conditions, in isolation, proves that the representation is sufficient for a physical task. The Gaussian target is motivated by the downstream-risk analysis already developed. The practical objectives approximate distributional constraints on the representation. A separate predictive target is still needed to specify what the representation should preserve.

This distinction becomes decisive when the model is used for interaction. The next section turns from the geometry of passive observations to the information needed to predict the consequences of chosen actions.

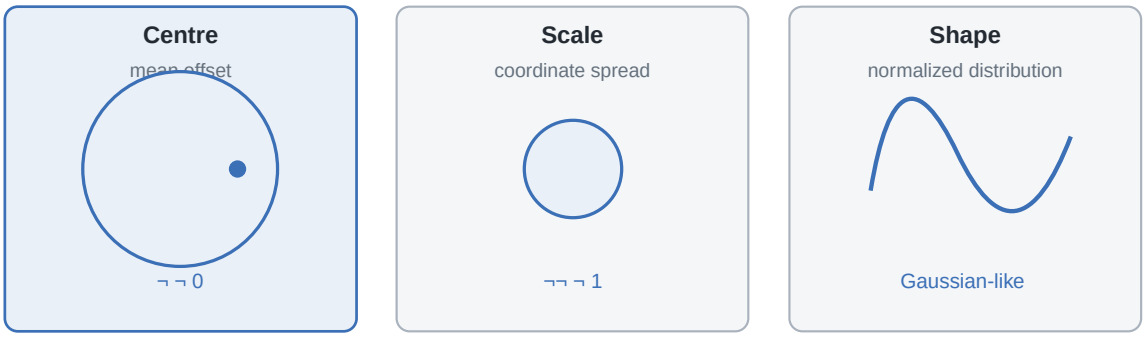


Figure 4 20. VISReg centre scale and shape decomposition

Figure 4.20. VISReg separates centre, scale and shape. VISReg regularizes mean offset, coordinate scale and normalized distribution shape with distinct terms. These components constrain different properties of the embedding batch.

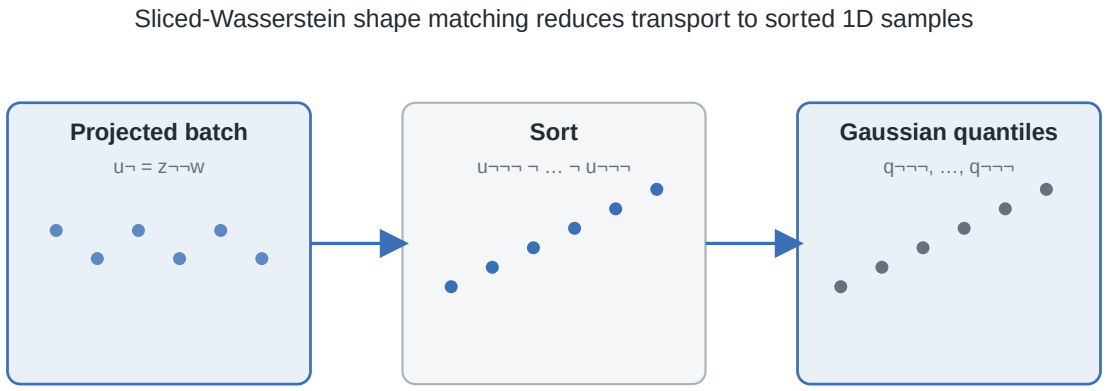


Figure 4 21. Sorted empirical and Gaussian projected samples

Figure 4.21. Sliced-Wasserstein matching is one-dimensional sorting. After projection, empirical samples and Gaussian reference quantiles are sorted and corresponding order statistics are matched. Repeating this operation across directions approximates sliced transport.

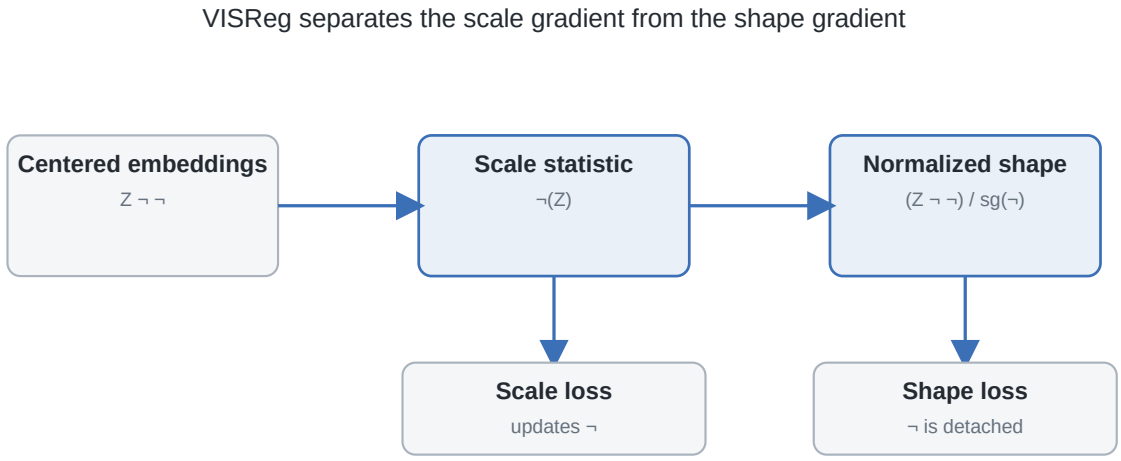


Figure 4 22. VISReg scale and shape gradient paths

Figure 4.22. Stop-gradient separates scale correction from shape normalization. The scale term acts on the unnormalized centered batch, while the shape loss treats the standard-deviation denominator as constant. This separates gradient paths without an EMA target.

4.11. Cross-modal prediction: when the target is language

The image and video examples establish that a predictor can be trained to infer a hidden representation rather than reconstruct the observation itself. They do not establish that the target

representation must belong to the same modality as the input. VL-JEPA makes this distinction explicit. Its visual input is an image or video, but its prediction target is a text embedding. A language query specifies which aspect of the visual evidence should be expressed.

This extension clarifies the generality of prediction in representation space. It does not, by itself, make a vision-language model a model of physical dynamics.

4.11.1. The visual context, query and target are different variables

Let X_V denote the visual input, X_Q a text query, and Y the target text. The query is not the target. It specifies the question being asked; the target is the answer representation against which the prediction is trained.

A minimal mathematical description is

$$\[S_V = f_\theta\(X_V\), \qquad S_Y = e_\psi\(Y\), \qquad \widehat{S}_Y = p_\phi(S_V, X_Q).\]$$

The visual encoder f_θ converts an image or video into visual features. The target encoder e_ψ maps the answer text to a continuous representation. The predictor p_ϕ combines the visual features with the query and estimates a vector in the target encoder's space.

Consider a video showing a person reaching toward a wall. One query asks which object the person is about to touch; another asks what action the person is performing. The frames are identical, but the requested answer changes. The query therefore selects the conditional prediction problem.

This separation is useful because a visual sequence does not determine one universally appropriate description. Captioning, object identification, question answering and retrieval impose different output requirements. A query-conditioned predictor can share visual features across them rather than treating each question as a new perception problem.

4.11.2. Why predict a text embedding?

A token-generative model produces a sequence of discrete symbols. An embedding-predictive model instead maps the target text through an encoder and learns to estimate the resulting vector. The difference is in the object being predicted and the geometry in which errors are measured.

For normalized predicted and target embeddings (ar\{hat s}_i) and (ar s_i), a one-direction batch contrastive objective can be written as

$$\[\mathcal{L}_{\widehat{S}\to S} = -\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(\bar{\hat{s}}_i^{\top}\bar{s}_i/\tau)}{\sum_{j=1}^{B}\exp(\bar{\hat{s}}_i^{\top}\bar{s}_j/\tau)},\]$$

where B is the batch size and (au) is a temperature. The reverse direction exchanges predicted and target embeddings. VL-JEPA averages the two directions, combining an alignment signal for paired examples with a uniformity pressure that discourages all examples from occupying the same representation.

The loss is important to identify correctly. VL-JEPA uses bidirectional InfoNCE in the reported formulation. It does not use SIGReg as its regularizer. The paper discusses non-sample-contrastive alternatives, including VICReg and SIGReg, as possibilities for future investigation rather than as part of its method (Chen et al., 2026).

An embedding target can place semantically related descriptions near one another even when their token sequences differ. For example, `the light turns off'' and` the room becomes dark" may be represented as related descriptions of the same event. Whether they are close is determined by the text encoder and its training, not by the fact that the outputs are vectors. The same geometry can be harmful when two answers differ in a small but consequential detail. Embedding prediction changes the target space; it does not make the target encoder a neutral or universally sufficient representation.

4.11.3. Prediction and decoding are separate operations

The text decoder is not required for the main embedding-prediction loss. It is used when the system must turn a predicted vector into human-readable language. Other tasks can consume the embedding directly.

For retrieval, the predicted feature can be compared with embeddings of candidate texts. For classification, candidate label descriptions can be encoded and ranked against it. Neither operation requires generating a sentence token by token. Captioning and open-ended question answering do require readable language, so a decoder is needed for those outputs.

The published VL-JEPA instantiation uses a frozen V-JEPA 2 ViT-L visual encoder. Its predictor is initialized from Transformer layers of Llama 3.2, while the target text encoder is initialized from EmbeddingGemma. Projection heads map predicted and target features into a shared 1,536-dimensional space. Visual tokens and query tokens are jointly attended by the predictor. These are implementation choices in the paper, not defining requirements of cross-modal prediction (Chen et al., 2026).

Training is staged. An initial large-scale phase learns visual-text alignment from image-text and video-text pairs without task queries. A second, query-conditioned supervised fine-tuning phase introduces tasks such as visual question answering while retaining the pretrained alignment used for classification and retrieval. The two phases expose the model to different query distributions and should not be conflated (Chen et al., 2026).

The embedding/decoder separation also permits selective decoding. A system can update a stream of semantic embeddings as video frames arrive and invoke the text decoder only when the representation changes enough to warrant a new description. On the paper's EgoExo4D evaluation, selective decoding at 0.35 Hz matched the reported performance of uniform decoding at 1 Hz, using approximately 2.85 times fewer decoding operations. This is a result for that protocol, not a general latency guarantee (Chen et al., 2026).

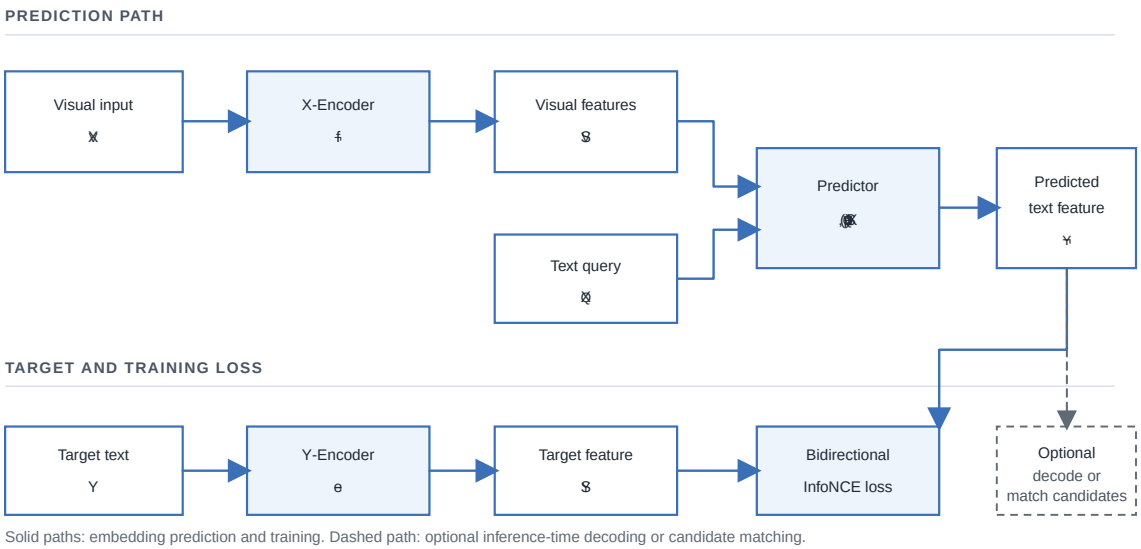


Figure 4 23. VL-JEPA visual context, query and text-target pathways

Figure 4.23. Cross-modal prediction separates the query from the target. The predictor combines visual features with a text query to estimate a target-text embedding. During training, that estimate is compared with the encoded target; at inference, decoding is optional when an embedding-level decision is sufficient.

> **Description.** The visual pathway supplies image or video features, while a separate text query specifies the requested semantic output. A target-text encoder supplies the training representation. The predicted feature can be decoded when readable language is required, but the decoder is not part of the embedding prediction itself.

4.11.4. A language query is not a physical action

VL-JEPA evaluates semantic tasks that include action anticipation and next-step forecasting. Their outputs are descriptions or labels inferred from video context. These evaluations show that a visual representation can support useful predictions about likely events or human actions (Chen et al., 2026). They do not establish that the model can compare candidate robot commands or predict the

physical consequences of a selected control.

The distinction can be expressed by comparing the two conditional computations:

$$\widehat{S}_Y = p_\phi(f_\theta(X_V), X_Q), \qquad \widehat{S}_{t+1} = F_\omega(S_t, A_t).$$

The first predicts a language-derived representation from visual evidence and a query. The second predicts a future state under a specified action. A prompt such as ``What would happen if the switch were pushed?" asks for a hypothetical semantic answer, but the presence of hypothetical wording does not establish an intervention model. That requires action-conditioned data, an explicit control interface, appropriate action coverage, and evaluation of predicted state changes.

The target need not share the input modality. The conditioning variable, however, must still represent the quantity whose effect the model is meant to estimate. This is the boundary between cross-modal prediction and controlled dynamics. The next section examines the separate data and training requirements that arise when actions become explicit inputs to a world model.

4.12. V-JEPA 2 and V-JEPA 2.1: from video pretraining to controlled prediction

The move from a video encoder to a model that can be used for action selection is not a single architectural modification. It changes the training data, the predictor’s inputs, the target distribution and the evaluation protocol. V-JEPA 2 makes this separation unusually clear: large-scale action-free video pretraining produces a visual representation, after which a new predictor is trained on robot trajectories with explicit action and end-effector-state inputs. V-JEPA 2.1 later changes the representation objective to improve dense spatial features. Its robot-planning experiments still require a separately trained action-conditioned predictor.

These stages should not be compressed into the claim that ``V-JEPA 2 learns to control robots" or that dense features automatically create a world model. The useful lesson is how each stage changes the information available to the predictor and what additional evidence is needed before a model can be used for planning. (Assran et al., 2025; Mur-Labadia et al., 2026)

4.12.1. Scaling the action-free representation

V-JEPA 2 begins with an action-free video representation model. Its pretraining data combines large video sources with images. The paper describes the resulting VideoMix22M mixture as 22 million samples and more than one million hours of video, with ImageNet images included to increase visual coverage. An image is converted into a video sample by repeating it across the temporal dimension. This allows images and videos to be processed by a shared video encoder while preserving a consistent input format.

The encoder is trained with a masked-feature prediction objective. A video is patchified into

spatiotemporal tokens, a subset is removed from the context input, and the online encoder processes the visible tokens. Learnable mask tokens carrying the positions of missing patches are concatenated with the visible embeddings and passed to a predictor. The predictor outputs features at the target positions. An EMA encoder processes the unmasked input and supplies the feature targets. The loss is an L_1 regression loss on the masked tokens.

The high-level objective retains the structure of V-JEPA:

$$\mathcal{L}_{\text{masked}} = \frac{1}{|M|} \sum_{i \in M} \| P_\phi(E_\theta(x), \Delta_y)_i - \text{sg}(E_{\bar{\theta}}(y)_i) \|_1,$$

where M is the set of masked positions, x is the masked context view, y is the unmasked target view, and Δ_y contains the positions of the missing tokens. The stop-gradient operator detaches target features in the loss; the EMA rule updates the target encoder's parameters. The formula is deliberately written at the level of the masked-token operation. The complete recipe also specifies data mixtures, temporal sampling, augmentations, architecture, optimization and model scale.

The distinction between masked prediction and action-conditioned prediction remains unchanged at this scale. The pretraining objective asks the encoder-predictor pair to infer missing video features from visible video context. It does not provide the model with a candidate intervention and ask what would happen under that intervention. An action-free video model can support motion understanding and human-action anticipation, but those evaluations are not evidence that it can predict the consequences of a robot's chosen control command.

The paper reports strong results on visual understanding and temporal anticipation, including 77.3% top-1 accuracy on Something-Something v2 and 39.7 recall-at-5 on Epic-Kitchens-100 for the reported V-JEPA 2 setting. These are task-specific results obtained with the evaluation protocols described in the paper. They do not measure the accuracy of an action-conditioned transition model.

4.12.2. Why video features alone do not identify action effects

Suppose a model observes a sequence $x_{1:t}$ and predicts a future feature z_{t+1}. From passive data, it can estimate a conditional relation of the form

$$p(z_{t+1} \mid x_{1:t}).$$

The distribution includes the actions that happened to be taken in the recorded trajectories, whether those actions are visible or not. It is not the same object as

$$p(z_{t+1} \mid x_{1:t}, a_t),$$

where a_t is a specified action whose consequence is being evaluated.

To see the difference, consider a robot arm approaching a cup. The observed videos may mostly contain successful grasps. A passive predictor can learn that a particular hand configuration is often followed by the cup moving with the gripper. But it has not necessarily learned how the cup would move if the arm stopped, moved sideways, opened the gripper or pushed the cup. These alternatives may be rare or absent in the dataset. A model that is not given the candidate action cannot directly distinguish them as interventions.

This is not an argument that passive video is useless for learning dynamics. It can provide visual features, motion regularities and strong priors about the physical environment. It is an argument about identification: the training objective must expose the predictor to the variable whose effect it is meant to estimate. If control is the target use, the action must be represented in the model input and aligned with the observed transition.

4.12.3. V-JEPA 2-AC: freeze the encoder, learn a controlled predictor

V-JEPA 2-AC is a post-training stage built on top of the pretrained V-JEPA 2 encoder. The encoder is frozen. A new autoregressive predictor is trained to estimate future video features conditioned on past video features, control actions and robot end-effector states.

The paper studies a tabletop Franka Panda arm with a fixed external camera. Each sampled clip contains 16 frames at four frames per second. The corresponding end-effector state is a seven-dimensional vector containing three position coordinates, three orientation coordinates and a gripper-state coordinate. Actions are constructed from changes in end-effector state between adjacent frames. In this setup the model receives both the visual feature sequence and proprioceptive information about the robot.

The predictor is a transformer with approximately 300 million parameters and 24 layers. Its attention is block-causal: a token at a given time can attend to the visual features, action and end-effector state at that time and to information from previous time steps, but not to future inputs. This makes the predictor autoregressive with respect to the controlled trajectory. The input types are projected separately into the predictor’s hidden dimension before being combined.

The training data are drawn from the raw DROID robot dataset. The paper uses roughly 62 hours of unlabeled robot video after discarding clips shorter than four seconds, leaving approximately 23,000 trajectories. ``Unlabeled" here has a specific meaning. The model does not receive task labels, reward values, or success/failure annotations for these demonstrations. It does receive the video and the end-effector state signals required to construct actions and condition the transition predictor.

Let $z_k = E(x_k)$ be the frozen visual feature map at time k, s_k the end-effector state, and a_k the

change in end-effector state between adjacent frames. The predictor receives the history

$$\[(a_t, s_t, z_t)_{t \le k}\]$$

and predicts the next visual feature map. A teacher-forcing loss compares the prediction at each step with the encoded feature of the observed next frame:

$$\[\mathcal{L}_{\mathrm{TF}} = \frac{1}{T} \sum_{k=1}^{T} \left\| P_\phi((a_t, s_t, z_t)_{t \le k}) - z_{k+1} \right\|_1.\]$$

Teacher forcing supplies the observed preceding feature at every training step. At inference time, however, the model must consume its own predicted features when rolling forward. This creates a training-inference mismatch: a predictor trained only with observed histories can accumulate error when its own estimates become part of the next input.

V-JEPA 2-AC therefore adds a short rollout loss. The predictor is run autoregressively for two steps, and its final predicted representation is compared with the corresponding encoded target:

$$\[\mathcal{L}_{\mathrm{rollout}} = \left\| P_\phi(a_{1:2}; s_1, z_1) - z_3 \right\|_1.\]$$

The complete objective is the sum of teacher-forcing and rollout terms:

$$\[\mathcal{L}_{\mathrm{AC}} = \mathcal{L}_{\mathrm{TF}} + \mathcal{L}_{\mathrm{rollout}}.\]$$

The two-step rollout is not a guarantee of long-horizon accuracy. It trains the predictor through one recurrent step beyond the teacher-forced transition, making the model experience a limited version of its own prediction errors. Longer rollouts would increase both the difficulty of optimization and the chance that errors in early predictions distort later inputs.

4.12.4. From the predictor to a planning loop

Once the model predicts future feature maps under proposed actions, a planner can compare candidate action sequences. Let z_k be the current visual representation, s_k the current end-effector state, and z_g the representation of a goal image. For a candidate action sequence $\hat{a}_{1:T}$, the model predicts a future representation. The paper uses a goal-conditioned energy of the form

$$\[\mathcal{E}(\hat{a}_{1:T}; z_k, s_k, z_g) = \left\| P_\phi(\hat{a}_{1:T}; s_k, z_k) - z_g \right\|_1.\]$$

The planner searches for an action sequence that reduces the discrepancy between the imagined future and the encoded goal. V-JEPA 2 uses the Cross-Entropy Method (CEM) to optimize the

action sequence. At each iteration it samples candidate trajectories from a distribution over actions, evaluates their predicted feature-space outcomes, retains high-scoring candidates and updates the sampling distribution around them.

The system then executes only the first action, receives a new camera observation and robot state, and plans again. This is receding-horizon or model-predictive control. It prevents the controller from blindly executing an entire open-loop sequence when fresh observations are available.

The complete loop therefore contains at least five distinct elements:

1. An encoder maps current observations into a feature representation.
2. An action-conditioned predictor estimates future features under candidate controls.
3. A goal representation specifies the desired visual outcome.
4. A planner searches candidate action sequences using the model and a goal-conditioned energy.
5. The robot executes an initial action and the loop repeats with a new observation.

The prediction model is not the planner, and the energy is not the prediction loss used to train the encoder. The L_1 feature-prediction loss teaches the predictor to match observed futures during training. The goal-conditioned energy ranks candidate action sequences during planning. The distinction is the same one established at the start of the chapter between prediction discrepancy, compatibility score and task cost.

4.12.5. What the physical planning result demonstrates

The V-JEPA 2 paper deploys the system zero-shot on Franka arms in two different laboratories. It evaluates reaching, grasping and pick-and-place tasks using image goals. The reported results are measured over a small number of task trials and several object or initial-position variations. The paper's claim is specifically that the pretrained video representation, when combined with robot-trajectory post-training and model-predictive control, supports these tasks in new environments without collecting environment-specific robot data there or adding task-specific training or reward labels for the deployment tasks.

This is significant evidence for the complete system. It is not evidence that the action-free encoder alone controls the robot. The action-conditioned predictor was trained on DROID robot trajectories, and the planner uses explicit visual goals. Nor does the experiment establish general-purpose control across arbitrary robots or environments.

The paper also identifies limitations. The model is sensitive to camera placement because action coordinates are defined relative to the robot base while the visual input comes from a camera. If the base is not visible, inferring the relation between the camera frame and action coordinates can be underdetermined. Long-horizon planning is limited by two different effects: autoregressive

representation errors accumulate, and the number of candidate action trajectories grows rapidly with the planning horizon. A longer planning horizon can be necessary for non-greedy tasks, but it increases both model and search difficulty.

These limits are valuable because they reveal where the representation-to-control transition remains incomplete. A model may predict short-horizon features well but become unreliable when rolled forward. A planner may find actions that minimize the learned goal distance while exploiting errors in the predictor. A representation may ignore a visually small detail that determines whether a gripper actually closes around an object. Physical planning must be evaluated at the level of the full closed loop.

4.12.6. V-JEPA 2.1: why masked-token prediction is not enough for dense features

The next development addresses a different limitation. V-JEPA 2's masked-token objective supervises the target features at masked positions. The visible context tokens are inputs to the predictor, but they do not receive the same direct prediction loss. The V-JEPA 2.1 paper argues that this can encourage context tokens to aggregate information useful for predicting global masked targets without preserving sufficiently local information at each visible position.

The distinction is between **global information useful for prediction** and **local information accessible at a particular token**. A representation can contain enough information to predict a hidden region while individual visible tokens are not ideal descriptors of the local object part or spatial location they correspond to. This is not necessarily a failure for tasks evaluated on pooled features. It becomes important for dense tasks such as depth estimation, semantic segmentation, object correspondence and localization, where a feature at one position should retain information about the scene at that position.

V-JEPA 2.1 adds a context loss to the original masked-token prediction loss. The context loss supervises visible tokens, with a weight determined by their nearest masked token in spatiotemporal block coordinates. If $d_{\min}(i, M)$ is the minimum distance, measured in number of blocks, from visible token i to the masked set, the paper uses

$$\lambda_i = \frac{\lambda}{\sqrt{d_{\min}(i, M)}}.$$

Visible tokens nearest to a masked region therefore receive the strongest context supervision. This choice encourages local continuity across the boundary between visible and masked regions, rather than imposing an equal identity-like target on every context token. The total dense prediction objective can be written schematically as

$$\mathcal{L}_{\text{dense}} = \mathcal{L}_{\text{predict}} + \mathcal{L}_{\text{context}},$$

with

$$\mathcal{L}_{\text{context}} = \frac{1}{|C|} \sum_{i \in C} \lambda_i \| P_\phi(E_\theta(x), \Delta_y)_i - \text{sg}(E_{\bar{\theta}}(y)_i) \|_1 .$$

Here C is the set of context positions and λ_i is a position-dependent weight. The target encoder processes the unmasked input. The original masked-token loss remains in place; the new term extends self-supervision to visible positions. The weighting is intended to encourage local consistency without making the context objective equivalent to an unconstrained identity mapping over all visible tokens.

This change creates a trade-off. In the paper's ablation, adding context loss improves dense prediction substantially, but initially reduces global recognition performance. On the reported ADE20K segmentation and NYUv2 depth evaluations, the context term improves dense metrics; at the same time, ImageNet and Something-Something classification accuracy fall. The paper's next component addresses that loss.

4.12.7. Deep self-supervision and multiple representation levels

V-JEPA 2.1 takes outputs from three intermediate encoder blocks in addition to the final block. The four normalized representations are concatenated along the channel dimension and passed through a lightweight MLP that fuses them and reduces their dimensionality. The predictor produces four corresponding output levels. Both the masked-token prediction loss and the weighted context loss are applied at all four levels, with the unmasked target encoder providing the multi-level target features.

The motivation is that a final-layer objective can place most of the learning pressure on the representation after many attention and mixing operations. Earlier blocks may not receive an equally direct signal to retain spatial detail. Applying losses at several depths creates training signals throughout the encoder. It also gives the model a way to balance local structure with global semantics instead of relying on the final representation to satisfy both demands alone.

The ablation is instructive. Context loss alone raises dense-task performance but harms global recognition. Adding multi-level prediction recovers much of the recognition performance while preserving the dense improvement. The reported progression then adds a larger and more diverse image mixture, modality-specific tokenizers, model scaling and a high-resolution cooldown phase. These ingredients are not all the same kind of intervention: the context loss changes what is supervised, deep self-supervision changes where the signal enters the encoder, tokenizers change the input interface, and data/model scaling changes capacity and coverage.

For a reader, the most important point is not to memorize the sequence of recipe modifications. It

is to understand the underlying tension. A representation trained to predict a hidden target may learn global context useful for that target. A representation used for dense prediction must also preserve spatially localized information at visible positions. Dense supervision changes the objective so that visible tokens themselves have a target to match.

4.12.8. Dense representations are not action conditioning

V-JEPA 2.1 is an image-and-video representation-learning recipe. Its dense prediction loss improves spatially and temporally structured features. It does not, by itself, add candidate actions to the visual prediction objective.

The V-JEPA 2.1 robot experiments make this distinction explicit. The authors train a frame-causal action-conditioned predictor following the V-JEPA 2-AC protocol, use the V-JEPA 2.1 video encoder to compute features, and train the approximately 300-million-parameter predictor on the raw DROID trajectories with teacher-forcing and two-step rollout losses. The model is then deployed zero-shot on a tabletop Franka Panda arm using image-goal model-predictive control. Thus, the representation encoder changes, but the action-conditioned predictor and planning interface remain separate trained components.

The reported grasp results require careful comparison. Under the matched planning configuration of 800 candidate samples, 10 CEM iterations, a one-step horizon and a three-second budget, the reported cup-grasp success rate rises from 60% with V-JEPA 2 features to 70% with V-JEPA 2.1 features. That is a ten-percentage-point increase in the matched table. The paper also reports an 80% V-JEPA 2.1 grasp result using 300 samples, 15 iterations, an eight-step horizon and a fourteen-second budget. The latter is a different planning configuration and should not be presented as a matched comparison with the V-JEPA 2 row.

The appropriate conclusion is limited but useful: in the reported setup, improved dense visual features can improve the spatial information available to a separately trained action-conditioned predictor and planner. The result does not show that dense feature prediction alone creates a controller, or that the same improvement will hold under all planning budgets, robots or tasks.

4.12.9. Three independent design questions

The progression from V-JEPA 2 to V-JEPA 2.1 separates three questions that are often collapsed.

What representation should be learned from passive video? The action-free pretraining objective determines which visual and temporal distinctions are rewarded. Masked-token prediction, dense context supervision, intermediate-layer losses, tokenizer design and data coverage all affect this representation.

How should the model predict under a chosen action? This requires an action-conditioned

predictor whose inputs include aligned observations, control commands and, when relevant, proprioceptive state. The training trajectories must cover the state-action combinations over which the model will be used.

How should the system choose and execute actions? A planner needs a goal or cost, a candidate action parameterization, a search or optimization procedure, constraints and a feedback policy. Its performance depends on model accuracy, search budget and the consequences of model error.

Changing the encoder addresses the first question. Adding an action-conditioned predictor addresses the second. Choosing a cost and planning algorithm addresses the third. A result at one level does not automatically establish the others.

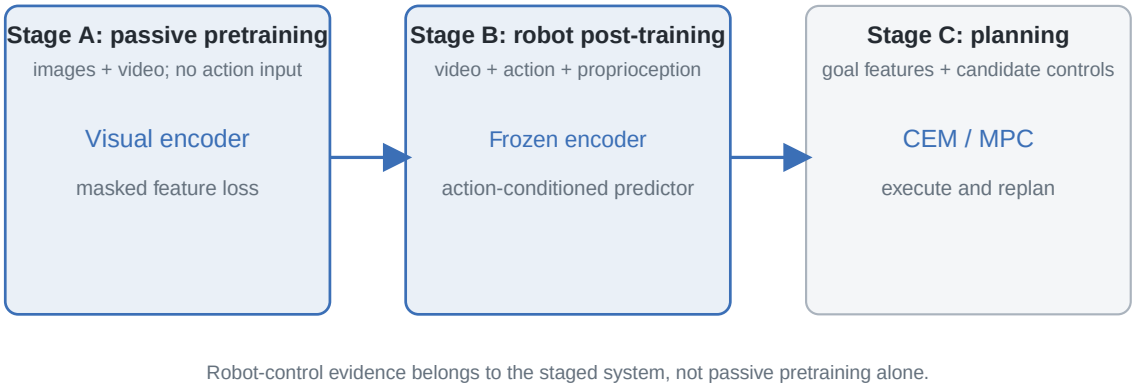


Figure 4 24. Passive visual pretraining followed by action conditioned robot post training

Figure 4.24. V-JEPA 2 separates passive pretraining from robot post-training. The visual encoder is pretrained on passive image and video data, then a separate action-conditioned predictor is trained on robot observations, actions and state. Planning evaluates candidate controls using the resulting model.

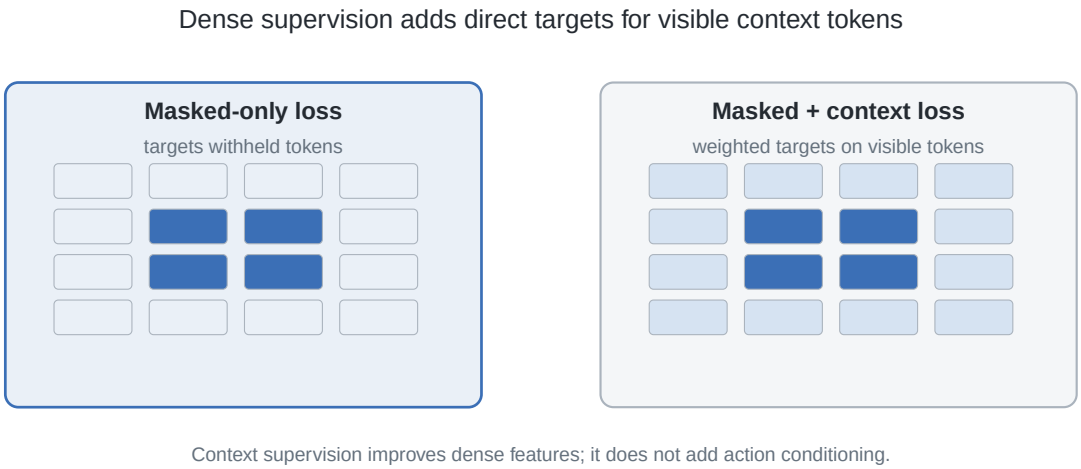


Figure 4 25. Masked only prediction and dense visible context supervision

Figure 4.25. V-JEPA 2.1 adds direct supervision to visible context tokens. The base objective supervises masked tokens; V-JEPA 2.1 adds distance-weighted targets on visible tokens and uses multi-level supervision. This improves dense features without making the visual encoder action-conditioned.

This progression provides a concrete example of how representation learning becomes part of a world-model system. The next section formalizes the distinction between passive prediction and prediction under an intervention, then examines the assumptions required before a learned transition model can be trusted for planning.

4.13. From passive prediction to action-conditioned prediction

A video contains changes in the world, but it does not necessarily reveal which changes would follow from a particular action. A model may predict what tends to happen after an observed history and still be unable to answer what would happen if an agent deliberately chose a different control input.

The distinction is the difference between predicting an observation sequence and predicting the consequences of an intervention.

4.13.1. The observational prediction problem

Let H_t denote the available observation history up to time t. A model trained only on passive video can learn a conditional distribution of the form

$$\[P(O_{t+1} \mid H_t).\]$$

This distribution averages over the actions, policies and external influences present in the collected data. If the video was generated by a behaviour policy π, then schematically

$$\[P(O_{t+1} \mid H_t) = \int P(O_{t+1} \mid H_t, A_t = a)\, P_\pi(A_t = a \mid H_t)\, da.\]$$

The conditional future is therefore tied to the action distribution of the data-collection process. It does not directly specify what would happen under an arbitrary alternative action.

For example, a video may show a cup moving to the right. The movement could have been caused by a robot pushing it, a person moving it, gravity acting on a tilted surface, or an external disturbance. The observed frames alone may not distinguish these explanations. A model that learns the usual visual continuation has not necessarily learned the effect of a robot command.

This is not a criticism of passive video learning. It is a statement about the information available in the training problem. Passive video can teach appearance, motion regularities, object persistence and many other useful features. It cannot be assumed to identify the consequences of controls that were not represented or recorded.

4.13.2. The controlled prediction problem

An action-conditioned model receives an explicit action variable. Its observational training distribution can be written as

$$\[P(O_{t+1} \mid H_t, A_t).\]$$

This conditional is not automatically an interventional distribution. The quantity needed to answer what would happen if the system deliberately set the action to a is

$$\[P(O_{t+1} \mid H_t, \mathrm{do}(A_t = a)).\]$$

The two distributions coincide only under additional assumptions about the state, data-collection policy and action support. In particular, the recorded history must contain the variables that jointly influence action selection and the next observation, and the dataset must include adequate support for the actions being evaluated. A latent predictive model can be written as

$$\[\widehat{Z}_{t+1} = g_\phi(Z_{\le t}, A_t, U_t),\]$$

where $Z_{\le t}$ is a history of visual representations, A_t is the action, and U_t denotes additional state information such as joint positions or end-effector configuration. For a sequence of controls,

$$\[\widehat{Z}_{t+1:t+H} = g_\phi(Z_{\le t}, A_{t:t+H-1}, U_{\le t}).\]$$

The action is not a label attached to the predicted future after the fact. It is an input that distinguishes the candidate intervention whose consequence is being estimated.

The difference can be written directly:

$$P(Z_{t+1} \mid Z_{\leq t}) \qquad \text{versus} \qquad P(Z_{t+1} \mid Z_{\leq t}, A_t).$$

The second conditional is the model's observational target when actions are recorded. It can support an estimate of the interventional quantity only when the data and modelling assumptions justify that identification. The first conditional marginalizes over the action distribution induced by the data-collection policy. Neither expression alone guarantees counterfactual validity.

4.13.3. Recording an action is necessary but not sufficient

Adding an action vector to a neural network does not automatically make the learned predictor causal or reliable. The action must correspond to the transition it is meant to explain. This creates several requirements.

Temporal alignment. The camera observation, control command and robot state must be synchronized. A command may be held over several frames, applied with delay, or executed at a different rate from the video. A one-frame misalignment changes which visual consequence is associated with an action.

Action semantics. The input must describe the command at the level used by the environment. Joint torques, joint-position targets, end-effector displacements and temporally extended action chunks define different transition problems. A model trained on one interface should not be assumed to understand another.

State information. Visually similar frames can correspond to different robot configurations. Proprioception can disambiguate joint angles, gripper state or end-effector pose that may be hidden by occlusion or perspective.

Coverage. The data must contain enough variation in actions and states to support the predictions the planner will request. If a model has never observed a particular action in a relevant configuration, it must extrapolate. A low training loss does not establish that such extrapolation is valid.

Unobserved influences. Even with recorded actions, an unobserved variable may affect both the action selection and the resulting transition. Conditioning on the recorded action does not by itself remove this confounding. The reliability of counterfactual predictions depends on the data-collection process, state variables, action support and modelling assumptions.

The correct claim is therefore narrower than ``adding actions produces a causal world model."

Recording actions makes the conditioning variable explicit. Estimating intervention effects additionally requires suitable state information, temporal alignment, action coverage and assumptions that control confounding. Whether a learned predictor generalizes those effects is an empirical and statistical question. (Assran et al., 2025)

4.13.4. A concrete training pipeline

The separation between passive representation learning and controlled prediction is not only conceptual. It determines how a training pipeline is organized. A visual encoder can first be trained on large image and video collections, where action labels are absent or unnecessary. The encoder can then be frozen while a second predictor is trained on aligned observation, action and state trajectories. The two stages optimize different conditional problems and should be evaluated separately.

V-JEPA 2 provides a detailed instance of this design. Its action-conditioned post-training, training data, prediction losses and image-goal planning interface are developed in Section 12. The general lesson is that a broad visual representation can supply the observation features, while a smaller action-conditioned model learns how those features change under a particular control interface. The second stage does not retroactively make the first-stage objective action-conditioned.

This modular design is useful when a large visual encoder is expensive to retrain or when the available robot data are much smaller than the passive video corpus. It also introduces an interface constraint: the action-conditioned predictor can only exploit information retained by the frozen encoder and the additional state inputs supplied to it. If an essential variable is absent from both, post-training cannot recover it from the input representation alone.

4.13.5. The target representation is part of the control interface

Suppose two physical states x_1 and x_2 map to the same latent representation,

$$f_\theta(x_1) = f_\theta(x_2).$$

If the same action a leads to different future distributions,

$$P(Y \mid x_1, a) \neq P(Y \mid x_2, a),$$

then no predictor receiving only this latent and the action can distinguish the two cases. The encoder has merged histories that the controlled prediction problem requires to remain separate.

This is the action-conditioned version of predictive sufficiency. A representation need not reconstruct every aspect of the physical world, but it must preserve distinctions that change the outcome distribution under relevant actions. A variable can be visually subtle and still be essential

for control. Examples include contact state, gripper opening, object velocity, friction condition or the position of an obstacle near the end of a planned trajectory.

A target encoder trained for broad visual invariance may discard exactly the small detail that makes a manipulation action succeed or fail. This is why a strong frozen representation benchmark is not enough to establish a reliable action-conditioned world model.

4.13.6. The transition model must be tested under interventions

A controlled predictive model should be evaluated on more than feature similarity along recorded trajectories. The tests should distinguish whether it predicts the consequences of different actions from the same or comparable starting states.

Useful evaluations include:

- holding the initial observation fixed and comparing predictions under alternative actions;
- predicting beyond the immediate next frame or feature step;
- testing action-state combinations not used during training;
- comparing predicted and observed changes in robot and object state;
- separating the visual encoder’s feature quality from the action-conditioned predictor’s transition accuracy;
- checking whether the planner requests actions outside the training support.

These tests are still not universal certificates of causal correctness. They reveal whether the model supports the specific interventions and environments that the agent is expected to handle.

The transition from passive video to action-conditioned prediction is therefore a change in the learning problem, not merely an architectural option. (Assran et al., 2025) The next section examines what an agent can do once it has such a predictor, and why planning introduces new failure modes even when the model performs well on held-out transitions.

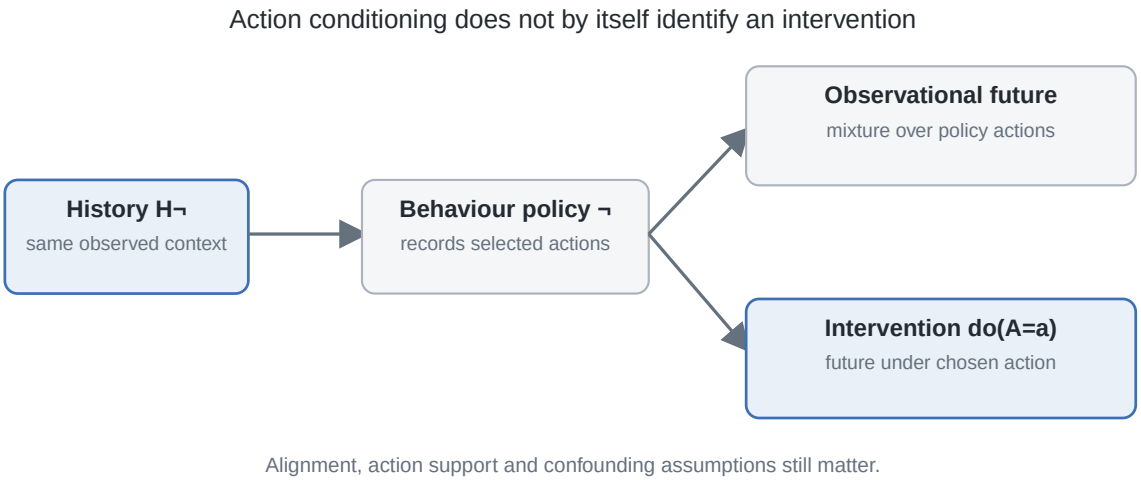


Figure 4 26. Observed behavior policy outcomes versus intervention

Figure 4.26. Observed action conditioning is not automatically an intervention. A behaviour policy produces observed actions and outcomes, while an intervention fixes an action to query a different future distribution. Identifying the latter requires alignment, action support and assumptions about confounding.

4.14. Planning with a learned predictive model

An action-conditioned predictor estimates how a representation may change under a specified control input. Planning adds another problem: choose a sequence of controls so that the predicted outcome satisfies an objective.

The predictor and the planner therefore have different responsibilities. The predictor estimates consequences. The planner compares those consequences and searches over possible actions. Their interaction creates capabilities that neither component has alone, but it also creates failure modes that are absent from ordinary representation evaluation.

4.14.1. From a one-step prediction to a candidate trajectory

Let z_t denote the current latent state, a_t a control input, and g_ϕ an action-conditioned transition predictor. A deterministic one-step model has the form

$$\hat{z}_{t+1} = g_\phi(z_t, a_t).$$

A model with observation history or proprioception can include those variables as additional inputs. For a candidate action sequence

$$\mathbf{a}_{t:t+H-1} = (a_t, a_{t+1}, \dots, a_{t+H-1}),$$

the model is iterated to produce a predicted latent trajectory:

$$\hat{z}_{t+k+1} = g_\phi(\hat{z}_{t+k}, a_{t+k}), \qquad k = 0, \dots, H-1.$$

This is a model rollout. It is not a prediction of the real environment at every intermediate step. It is the trajectory implied by repeated application of the learned transition function.

A planner evaluates the candidate sequence using a cost. A simple goal-conditioned objective is

$$J(\mathbf{a}_{t:t+H-1}) = d(\hat{z}_{t+H}, z_g) + \sum_{k=0}^{H-1} c_a(a_{t+k}),$$

where z_g is a goal representation, d measures terminal mismatch, and c_a penalizes action magnitude or another control preference. A more general objective can accumulate state costs:

$$J(\mathbf{a}) = \sum_{k=1}^{H} c_z(\hat{z}_{t+k}) + \sum_{k=0}^{H-1} c_a(a_{t+k}) + c_T(\hat{z}_{t+H}).$$

The components have different meanings. The transition model predicts latent states. The goal or cost function evaluates them. The action penalty expresses a control preference. The planner searches over the sequence.

A low training prediction loss does not specify any of these planning choices. A latent predictor cannot decide by itself whether a future is desirable.

4.14.2. Image goals and latent distance

A common goal-conditioned setup supplies a target image o_g. The current observation and goal image are encoded into representations,

$$z_t = f_\theta(o_t), \qquad z_g = f_\theta(o_g).$$

For each candidate action sequence, the transition model predicts a future representation $\hat{z}_{t+H}$. The planner can then minimize a goal discrepancy such as

$$d(\hat{z}_{t+H}, z_g).$$

This avoids requiring the model to decode every predicted state into an image before the planner can compare it with the goal. It can also make the objective insensitive to appearance variation that the encoder has learned to ignore.

But latent distance is only a proxy for physical success. If the encoder discards gripper aperture, contact state or a small obstacle, a low latent distance can correspond to an unsafe or unsuccessful physical state. The goal representation and the transition representation must preserve the variables needed to distinguish success from failure.

A visual goal can also be ambiguous. The same image may be reached through different object configurations, contact conditions or robot postures. Whether those states are interchangeable depends on the task. A goal encoder does not automatically resolve this ambiguity.

4.14.3. Searching over action sequences

The action space may be continuous and the sequence horizon long. Directly evaluating every possible sequence is usually infeasible. Planning therefore relies on an optimization or search procedure.

One option is gradient-based optimization. If the transition model and cost are differentiable, gradients can be propagated through the imagined rollout to the candidate action variables:

$$\nabla_{\mathbf{a}} J(\mathbf{a}).$$

The optimizer changes the action sequence to reduce predicted cost. This is possible because the learned model is a differentiable computation graph. It does not imply that gradients pass through the physical environment. The agent optimizes actions inside its model, executes a selected action in the environment, and then obtains a new observation.

A derivative-free alternative is the cross-entropy method (CEM). It maintains a distribution over candidate action sequences:

$$\mathbf{a}^{(i)} \sim q_\psi(\mathbf{a}), \qquad i = 1, \dots, N.$$

Each sampled sequence is rolled through the learned model and assigned a cost. The method selects an elite set with low predicted cost and refits the sampling distribution to those candidates. Repeating this procedure concentrates samples around sequences that the model predicts to be promising.

CEM does not require gradients through the predictor, though it still depends on the predictor's accuracy. It can also search over actions whose parameterization is inconvenient for direct gradient descent. Its effectiveness depends on the sample count, number of iterations, distribution family, horizon and action bounds.

Neither gradient descent nor CEM is intrinsically a guarantee of good physical behaviour. They optimize the objective supplied to them under the model they are given. If that objective or model

is wrong, a more effective optimizer may find a more convincing failure.

4.14.4. Receding-horizon feedback

A planner normally does not execute an entire optimized sequence open loop. It executes the first action or a short prefix, obtains a new observation, estimates the current state again, and replans.

At time t, solve

$$\mathbf{a}^*_{t:t+H-1} = \arg\min_{\mathbf{a}_{t:t+H-1}} J_t(\mathbf{a}_{t:t+H-1}).$$

Execute a^*_t, observe the environment at $t+1$, update the representation and solve a new optimization problem. This is the receding-horizon principle used in model-predictive control.

Replanning limits how long the system commits to a single predicted trajectory. It can compensate for disturbances and moderate model errors because the next plan starts from a fresh observation. It does not remove systematic bias. If the model repeatedly underestimates a collision risk or ignores a hidden state, replanning can repeatedly choose the same unsafe action.

Feedback is therefore a robustness mechanism, not a proof of safety.

4.14.5. Why planning can fail when prediction looks good

Prediction error is usually measured on trajectories sampled from a data distribution. A planner does not sample actions according to that same distribution. It actively searches for sequences that minimize the model's estimated cost. This creates a form of distribution shift.

Consider a model that is accurate on ordinary actions but slightly wrong in a particular region of the action space. A planner searches many candidates and may discover a sequence that exploits the error. The predicted trajectory appears to reach the goal cheaply, so the optimizer prefers it. In the real environment, the sequence fails.

This is often called model exploitation or model bias exploitation. The planner is not necessarily behaving irrationally. It is solving the optimization problem it was given. The problem is that the model's predictions are trusted outside the region where they have been validated.

Several effects make this more likely:

- **Horizon growth.** Small one-step errors compound when the model is iterated over many steps.
- **Action distribution shift.** The optimizer may choose actions that were rare or absent in the training trajectories.

- **Representation aliasing.** Two physical states that respond differently to an action may have similar latent representations.
- **Unmodelled state.** Proprioception, contact, friction or other hidden variables may be omitted.
- **Cost mismatch.** A goal-distance objective may reward a state that violates a safety or feasibility constraint.
- **Search pressure.** Evaluating many candidate sequences increases the chance of finding a sequence whose predicted value is optimistic because of model error.

A planning result must therefore report the model, training data, action representation, horizon, planner, objective, search budget, environment and execution protocol. Reporting only a representation benchmark or a one-step prediction score is insufficient.

4.14.6. Multiple futures and uncertainty

A deterministic transition model maps one context and action to one predicted future. Real environments may have multiple possible outcomes because of stochastic dynamics, partial observability, other agents or model uncertainty.

A more complete model represents a conditional distribution,

$$p_\phi(z_{t+1:t+H} \mid z_{\le t}, \mathbf{a}_{t:t+H-1}),$$

or a set of alternative hypotheses. A latent variable can index possible futures, but it must be structured so that it selects meaningful alternatives rather than simply copying the target.

A planner using uncertain predictions has to decide how to evaluate risk. It might minimize expected cost,

$$J_{\text{mean}}(\mathbf{a}) = \mathbb{E}_{Z \sim p_\phi(\cdot \mid \mathbf{a})}[C(Z)],$$

or it might penalize high-cost tails, worst-case outcomes or violations of a chance constraint. These choices produce different behaviour. Expected cost can prefer a sequence with a small probability of catastrophic failure if most outcomes are cheap. A risk-sensitive objective can prefer a more conservative action.

The uncertainty estimate itself must be evaluated. A stochastic predictor does not guarantee calibrated probabilities. An ensemble does not automatically capture all uncertainty. A multi-hypothesis model can still omit a mode. The planner’s risk criterion is only as reliable as the predictive distribution it receives.

4.14.7. Safety is not a latent distance

Suppose a planner minimizes only the distance between a predicted latent and a goal latent. Unless the representation and objective explicitly preserve and penalize unsafe states, the optimizer has no reason to avoid them.

A manipulation system may need explicit constraints on collision, force, velocity, joint limits and gripper state. A navigation system may need obstacle constraints and a margin around forbidden regions. These constraints can be imposed in the cost, in the action parameterization, through a separate safety filter, or by a combination of mechanisms.

The role of a world model is to predict consequences that the planner can evaluate. It does not decide by itself which consequences are acceptable. Nor does a low energy or a low feature distance imply a safe state.

4.14.8. What a physical planning result demonstrates

A successful physical planning experiment demonstrates that a complete system worked under the reported conditions. The system includes at least:

1. an observation encoder and state representation;
2. a predictor conditioned on relevant actions and state variables;
3. training trajectories that support the requested predictions;
4. a goal representation and task cost;
5. a planner and a specified optimization budget;
6. an execution interface and feedback loop;
7. an environment and a measured success criterion.

The V-JEPA 2-AC experiments are an example of this system-level evidence. The visual encoder is pretrained from passive data, the action-conditioned predictor is post-trained on robot trajectories, and an image-goal planner searches candidate action sequences. The reported physical outcome belongs to that complete configuration. (Assran et al., 2025) It should not be attributed to the visual encoder alone.

A responsible evaluation should distinguish simulated from physical results, familiar from unfamiliar environments, one-step from multi-step training, and matched from unmatched planner budgets. Success rates should be read with the number of trials, task definition and uncertainty across trials. A single aggregate score cannot certify safety or generalization to all objects, robots and environments.

A predictive model becomes useful for action only when it can be queried with interventions and

integrated into an objective, search procedure and feedback loop. The final section examines the larger architecture proposed to organize these capabilities, including memory, multiple levels of prediction and the distinction between fast reactive behaviour and deliberative planning.

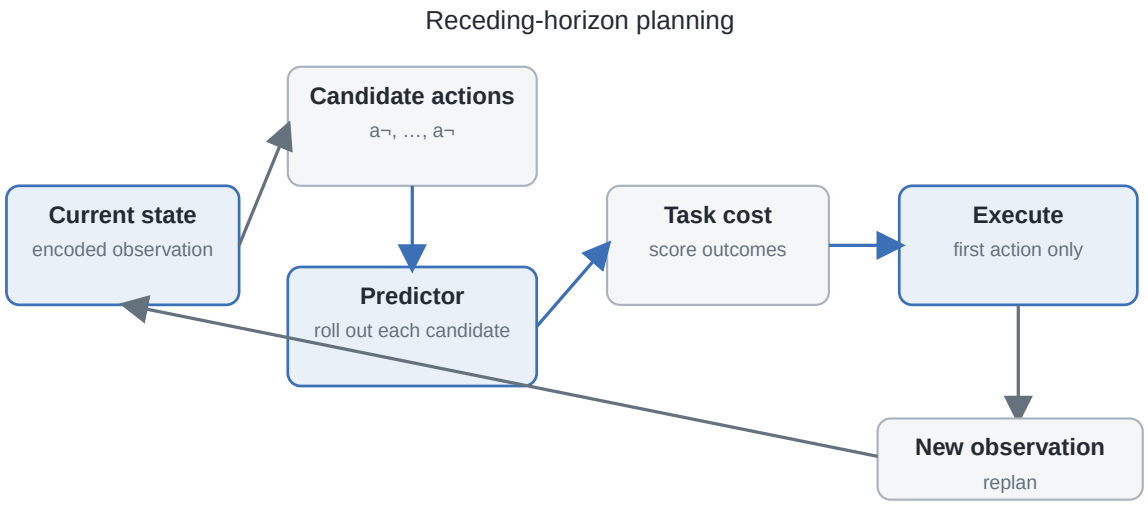


Figure 4 27. Receding horizon planning loop

Figure 4.27. A planner executes the first action and replans. Candidate action sequences are evaluated through imagined rollouts, but only the first action is executed. A fresh observation closes the feedback loop and starts the next optimization.

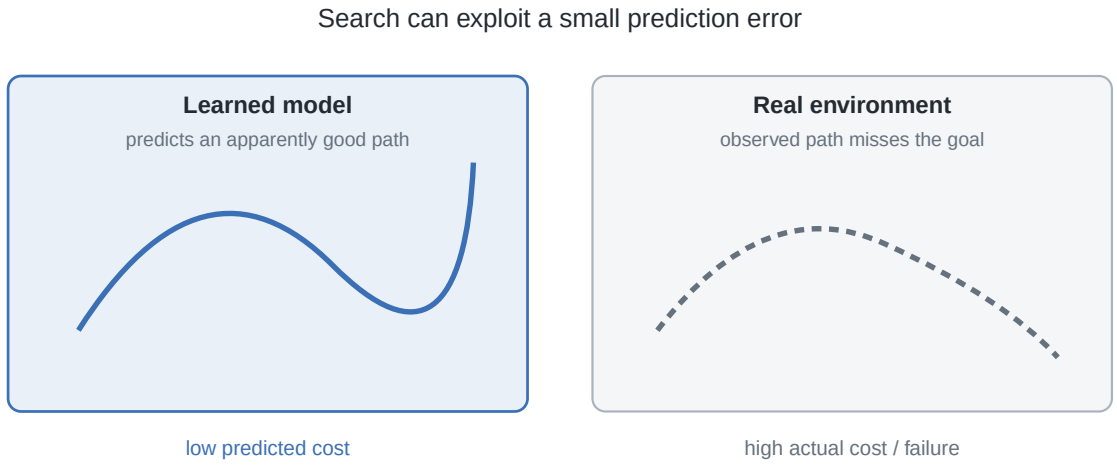


Figure 4 28. Predicted trajectory versus real environment failure

Figure 4.28. Search can exploit prediction error. An optimizer may find a sequence with deceptively low model-predicted cost that fails in the real environment. More search can intensify this failure by selecting optimistic model errors.

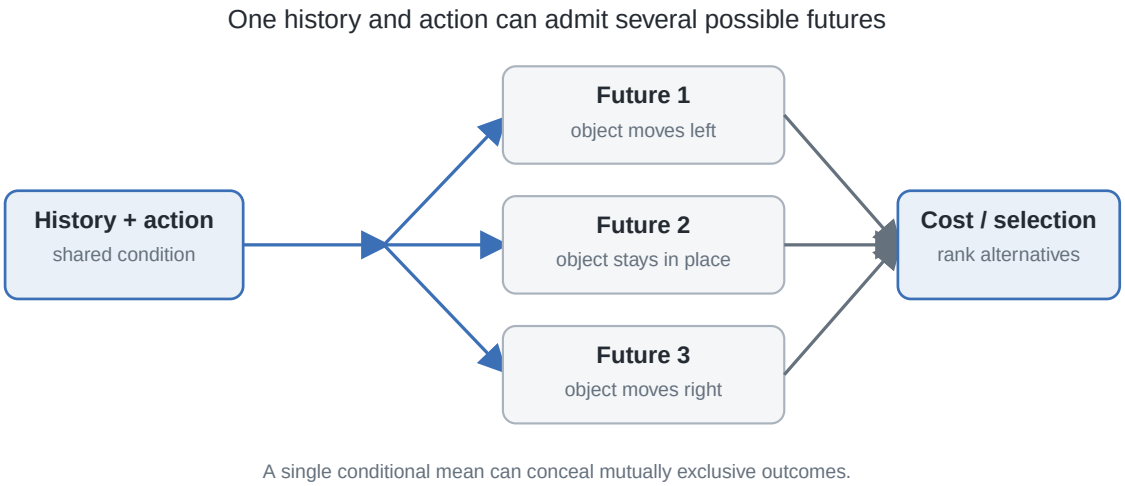


Figure 4 29. Multiple future outcomes from one context and action

Figure 4.29. A single deterministic prediction can hide multiple futures. One context and action can admit several plausible outcomes. A distribution or latent selector can preserve these alternatives instead of collapsing them into one averaged feature.

4.15. Memory, hierarchy and an agent around the world model

An action-conditioned predictor supplies a model of how candidate actions may change a represented state. It still does not specify which actions should be tried, what outcomes count as desirable, how information from earlier observations should be recovered, or how behaviour should change when the task changes.

These responsibilities belong to the agent that uses the world model. LeCun's *A Path Towards Autonomous Machine Intelligence* proposes an architecture that makes them explicit. (LeCun, 2022) The paper is a research position and design proposal, not a report of a fully validated system implementing every component. Its value here is to clarify what must surround a predictive model if it is to support autonomous behaviour.

4.15.1. Six modules, six different responsibilities

The proposed architecture has six top-level modules:

1. **Perception** maps sensory observations into an internal state representation.
2. **World Model** predicts possible future states under imagined action sequences.
3. **Cost** assigns scalar energy to current and predicted states.
4. **Actor** proposes actions and action sequences.

5. **Short-Term Memory (STM)** stores and retrieves relevant past, current and predicted states, together with associated intrinsic costs.
6. **Configurator** configures the other modules for the task at hand.

The hierarchy inside Cost matters. The proposal places two components within the Cost module:

- **Intrinsic Cost** computes immediate intrinsic energy from a state. It represents basic drives or intrinsic quantities that are not learned in the same way as the critic.
- **Trainable Critic** predicts future intrinsic cost from states and experience.

Intrinsic Cost and the Critic are not additional top-level peers beside Perception, World Model, Cost, Actor, STM and Configurator. They are components of Cost.

This architecture separates several scalar functions that might otherwise be conflated. A JEPA prediction loss measures mismatch between a predicted and target representation. A compatibility energy scores whether a candidate configuration is compatible with a model. An agent-level cost evaluates whether a predicted state or trajectory is desirable. A critic estimates future cost. These scalars may interact, but they do not have the same meaning.

4.15.2. The information flow through the agent

At a high level, the agent operates as a closed loop.

An observation reaches Perception, which produces a state representation. The World Model uses that state, relevant memory and candidate actions to predict possible futures. Cost evaluates the predicted states. Actor uses those evaluations to select an action or action sequence. The selected action is sent to the environment, which produces a new observation. The loop repeats.

STM is not merely a record of every past frame. It provides an associative mechanism for retrieving information relevant to a current query. The World Model can use stored states when predicting or completing a state estimate, and the Critic can use retrieved experience and associated intrinsic costs to estimate future cost.

Configurator modulates the active modules according to the task. It can change what Perception emphasizes, what the World Model predicts, which costs matter and how the Actor behaves. The proposal treats this as an executive function, not simply as an extra task embedding appended to a fixed network.

The physical environment sits outside the learned computational graph. Gradients can be propagated through differentiable Perception, World Model, Cost and Actor components when optimizing imagined trajectories. The agent then executes an action in the real environment and receives a new observation. The architecture should not be interpreted as backpropagating gradients through the robot or the physical world itself.

4.15.3. Mode 1: reactive behaviour

Mode 1 selects an action directly from the current state, possibly using STM:

$$\[a_t = A_\psi(s_t, m_t),\]$$

where s_t is the perceived state and m_t denotes retrieved memory. The policy does not deliberately search through multi-step imagined action sequences for every decision.

Mode 1 can be computationally inexpensive at inference time. It can still be sophisticated, memory-dependent and task-configured. Its defining property is that action selection is reactive rather than the result of a fresh optimization over candidate trajectories through the World Model and Cost.

The World Model need not stop learning while the agent operates in Mode 1. It can continue to predict the next state, compare that prediction with the new observation, and use the observed transition as a learning signal. Reactive action selection and predictive learning are separate operations.

4.15.4. Mode 2: deliberate planning through imagined futures

Mode 2 uses the World Model as an internal simulator. A typical planning cycle is:

1. Perception estimates the current state and Cost evaluates its immediate energy.
2. Actor proposes an initial action sequence.
3. World Model predicts one or more possible state sequences under those actions.
4. Cost evaluates the predicted states or trajectory.
5. Actor modifies the proposed actions to reduce the predicted cost.
6. The system executes the first action or a short prefix.
7. A new observation updates the state estimate and the planning cycle is repeated.

For a deterministic model and an additive cost, the sequence objective can be written schematically as

$$\[J(\mathbf{a}) = \sum_{k=1}^{H} C(\hat{s}_{t+k}) + \sum_{k=0}^{H-1} c_a(a_{t+k}).\]$$

When the learned model and cost are differentiable, the system can optimize candidate actions by gradients through the imagined rollout. For discrete actions or non-differentiable objectives, it can use other search or planning procedures. The model is used to evaluate candidates internally; only

a selected action is executed in the world.

Mode 2 is closely related to model-predictive control. Planning considers a finite horizon, but execution is followed by fresh observation and replanning. The distinction between an imagined action sequence and an executed action sequence is fundamental. The model may simulate many alternatives, but the physical environment experiences only the action that the Actor sends to it.

The proposal does not prescribe one universally correct optimizer. Gradient-based optimization, dynamic programming, tree search and sampling-based methods make different assumptions. Their effectiveness depends on the model, cost, action parameterization and available compute.

4.15.5. Distilling Mode 2 into Mode 1

Repeated deliberative search can be expensive. The proposal therefore allows the outputs of Mode 2 to train a reactive policy.

Let $\tilde{a}_t$ be an action selected by Mode-2 planning for state s_t. A policy A_ψ can be trained to imitate these planner-generated actions:

$$\mathcal{L}_{\text{distill}}(\psi) = \mathbb{E}_{s_t}\left[D(\tilde{a}_t, A_\psi(s_t))\right],$$

where D is an action-space loss appropriate to the action representation. The resulting policy approximates the planner's decision with a direct state-to-action mapping.

This is a form of amortized inference. Mode 2 computes a relatively expensive answer for sampled states. Mode 1 learns a function that approximates those answers quickly. The learned policy can later operate reactively, or it can provide an initialization for another Mode-2 search.

The distillation target is not necessarily an expert action from an external teacher. It is an action that the agent's own model and cost considered desirable. Consequently, the distilled policy inherits the teacher planner's errors. If the World Model is biased or Cost is misaligned, policy distillation can make the same error faster and more consistently.

4.15.6. Short-Term Memory as associative state retrieval

A partially observed system may encounter the same physical configuration more than once without receiving identical observations. Relevant information may be absent from the current camera view, hidden behind an object, or distributed across observations separated in time.

An associative memory stores information in a form that can be retrieved by content. A query derived from the current state can retrieve a related past state and its associated value or intrinsic cost. The retrieved information can support state completion, prediction or evaluation.

This differs from simply appending a fixed number of recent frames to the current input. A

temporal window retrieves information by adjacency in time. Associative memory retrieves information by similarity or a learned relation to the query. A past state can be relevant even if it is not the immediately preceding state.

The proposed memory stores information about past, current and predicted states, together with associated intrinsic costs. The World Model can query and update this memory. The Critic can retrieve past states and their observed costs while learning to estimate future intrinsic cost.

The paper leaves important implementation questions open: capacity, retrieval, write policy, key and value design, update rules and interference between stored states. These are research choices, not solved consequences of naming a module STM. The mathematical treatment of temporal state and recurrent memory belongs elsewhere in the book; the point here is the role associative retrieval plays in a complete predictive agent.

4.15.7. Hierarchical prediction and H-JEPA

A single predictor operating at one temporal resolution may have to represent both fine sensory changes and abstract long-term outcomes. A hierarchy separates these responsibilities.

Let $s_t^{(0)}$ denote a fine-scale state representation. Higher levels $s_t^{(1)}, s_t^{(2)}, \dots$ describe increasingly abstract states or events over longer intervals. A level-k predictor may estimate an abstract target over horizon Δ_k:

$$\hat{s}_{t+\Delta_k}^{(k)} = F_k(s_{\le t}^{(k)}, u_{t:t+\Delta_k}).$$

The variable u at a high level need not be a physical motor command. It can encode a subgoal or a constraint that a lower-level model must satisfy. A high-level predictor may specify that an object should be moved to a region. A lower-level predictor then determines which sequence of intermediate states and physical actions can realize that subgoal.

This changes the planning problem. A high-level model can reason over a smaller number of abstract transitions, while lower-level models expand a subgoal into detailed predictions and actions. It can reduce the effective horizon and search complexity. However, abstraction is only useful if it preserves the constraints needed for execution. If a high-level representation discards a contact condition or obstacle, a lower-level controller may be unable to realize the proposed subgoal.

LeCun's H-JEPA discussion is a proposed direction for organizing prediction across levels and time scales. (LeCun, 2022) It should not be described as a single canonical architecture with a settled training algorithm. The specific representations, interfaces, temporal resolutions and methods for training levels remain design questions.

4.15.8. Multiple futures and latent variables

A deterministic predictor produces one future for a given context and action sequence. Yet the same action can lead to different outcomes because the system is partially observed, the environment is stochastic, or other agents intervene.

A latent variable z can index alternative futures:

$$\hat{s}_{t+1:t+H} = g_\phi(s_{\leq t}, \mathbf{a}, z).$$

Different values of z can select different plausible continuations. In an energy-based formulation, the latent variable can select a configuration that reduces incompatibility between context and target. The latent should represent a restricted alternative, not an unrestricted copy of the target. Otherwise the model can transmit the answer through z and make prediction trivial.

A hierarchy can also represent uncertainty at several levels. A high-level model may retain alternative subgoals, while lower-level predictors estimate the detailed transitions associated with each subgoal. The agent can use Cost to rank these possibilities and avoid expanding every possible trajectory indefinitely.

This is a computational problem as well as a statistical one. If a model branches into many futures at each time step, the number of candidate trajectories can grow exponentially with horizon. The system must use structure, learned priors, costs or search procedures to allocate computation to the futures that matter.

The proposal does not prescribe one complete solution to uncertainty. Latent-variable models, ensembles, stochastic predictors and explicit distributions each represent different assumptions. A stochastic output is not automatically calibrated, and multiple hypotheses are not useful if they omit the dangerous or relevant mode.

4.15.9. The Configurator remains an open problem

The Configurator receives information from the other modules and changes how the system is organized for a task. LeCun describes it as executive control: it pre-configures Perception, World Model, Cost and Actor for the task, modulating the parameters and attention circuits of the modules it feeds. It can prime Perception to extract task-relevant information, configure the World Model for the current situation, and configure Cost to express the current goal. The Actor's policy is also described as configurable. The paper does not provide a finished algorithm for learning the Configurator.

This role is broader than selecting a task-specific output head. It is intended to configure how several components function together. Yet the proposal does not provide a finished learning algorithm for discovering these configurations or prove that such modulation will be stable. The

design raises difficult questions about what a task description must contain, how modules should communicate, and how configuration should change as the environment changes.

The same restraint applies to the full architecture. The six-module design makes the responsibilities of an autonomous agent explicit, but it is not empirical evidence that one integrated implementation has solved them. The chapter's contribution is to show why a predictive representation is only one component in that system.

4.15.10. The role of a JEPA inside an agent

The architecture closes the argument developed throughout the chapter.

A representation-learning objective defines which views should agree and which target features should be predictable. A distributional constraint controls the geometry of the embedding space. An action-conditioned model predicts the consequences of candidate interventions. A cost or energy ranks the predicted states. An Actor searches or selects actions. Memory supplies information that is absent from the current observation. Hierarchy allows prediction and control to operate at different temporal scales. A Configurator adapts these mechanisms to a task.

No single feature-prediction loss provides all of these functions. The useful question is not whether a JEPA is itself an autonomous agent. It is what predictive information it supplies to an agent, and what other structures are required before that information can support reliable action.

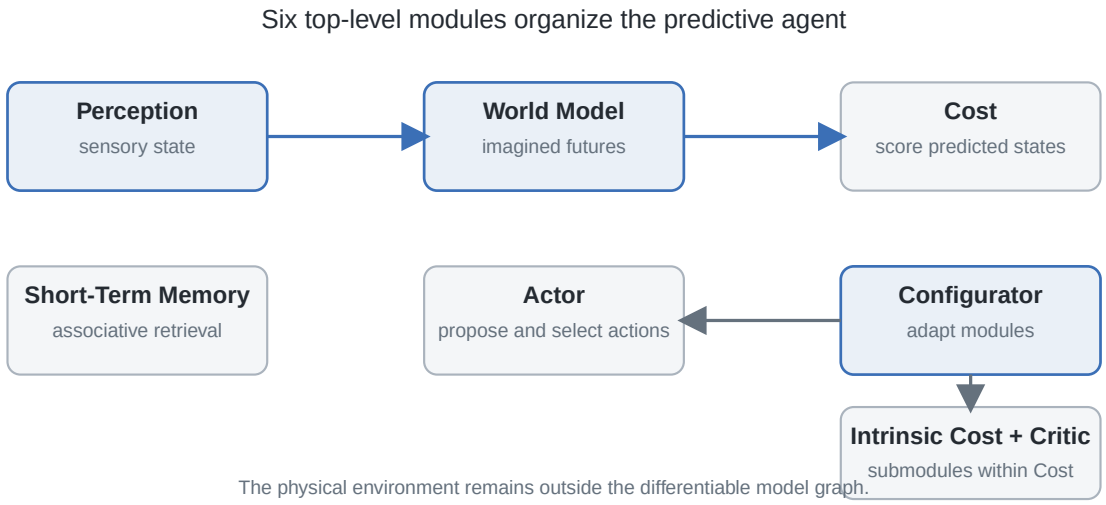


Figure 4 30. Six module predictive agent architecture

Figure 4.30. The proposed agent has six top-level modules. Perception, World Model, Cost, Actor, Short-Term Memory and Configurator are the six top-level modules. Intrinsic Cost and Trainable Critic are subcomponents of Cost; the physical environment remains outside the model graph.

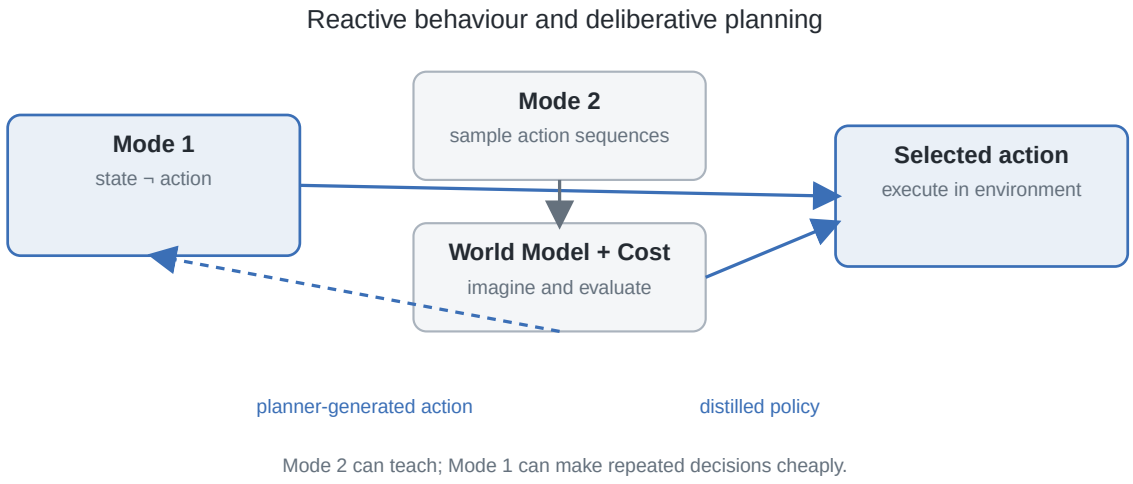


Figure 4 31. Reactive Mode 1 and deliberative Mode 2

Figure 4.31. Mode 1 is reactive; Mode 2 searches imagined futures. Mode 1 maps state directly to action, while Mode 2 uses the world model and cost to evaluate candidate trajectories. Planner-generated actions can serve as training targets for a cheaper reactive policy.

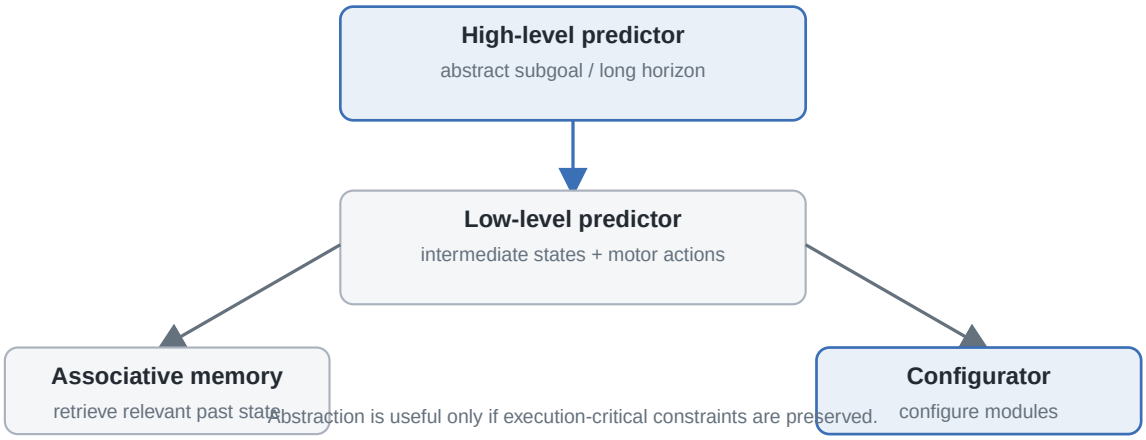


Figure 4 32. High level and low level predictive hierarchy

Figure 4.32. Hierarchical prediction separates time scales. A high-level predictor proposes an abstract subgoal and a lower-level predictor refines it into intermediate states and actions. Memory and configuration can carry relevant context across the hierarchy.

4.16. Reading a JEPA training step

The equations of a predictive objective become easier to reason about when each variable is connected to an operation on tensors. This section traces one training step through an image-based

JEPA. The purpose is not to prescribe a complete training recipe for every domain. It is to make the data paths, gradient paths and mask semantics explicit.

I-JEPA provides a useful example because its architecture separates a context encoder, a target encoder and a predictor, and because it predicts several target blocks from one context block. The same structure can then be compared with video feature prediction and with predictor-free multi-view matching.

4.16.1. From an image to patch representations

Let an image batch have shape

$$\[X \in \mathbb{R}^{B \times C \times H \times W}.\]$$

A patch embedding layer divides each image into N patches and maps every patch to a D -dimensional token:

$$\[T = \mathrm{PatchEmbed}(X) \in \mathbb{R}^{B \times N \times D}.\]$$

The transformer encoder operates on these tokens together with positional information. Before any masking, the image can therefore be viewed as a grid of feature locations. A mask does not simply delete arbitrary values from an already-computed feature vector. It defines which image patches are supplied to the context pathway and which positions are requested as targets.

Let M_c be the context mask and $M_1, \ldots, M_M$ the target masks. For an image-based task, these masks are spatial. In a video model, they become spatiotemporal and index locations across both position and time.

A basic shape check is useful. If an image is divided into a grid of $h_p \times w_p$ patches, then $N = h_p w_p$. A target block containing n_i patch locations should produce a target tensor with n_i tokens per image. The predictor must return exactly the same target locations and feature dimension. A model that predicts the right number of tokens in the wrong spatial order has not solved the intended task.

4.16.2. Constructing context and target blocks

I-JEPA samples one relatively large context block and several target blocks. The target blocks are sampled independently of the context. They can overlap the context before masking is resolved, so any overlapping target locations are removed from the context block. The remaining context is spatially distributed and informative, but does not reveal the target tokens directly.

The paper uses target blocks of a sufficiently large spatial scale to encourage semantic prediction

rather than trivial local interpolation. It also uses a context block that covers most of the image while retaining some spatially distributed missing regions. These choices define the difficulty of the task. A very small target may be predictable from local texture. A target that is too large or too distant may be ambiguous even when the context contains all useful information.

The masks are therefore part of the model's inductive bias. They determine which conditional relationship is repeatedly presented during training.

For one image, the context encoder receives the visible context positions:

$$\[T_c = T[M_c].\]$$

The notation $T[M]$ denotes selection of the tokens indexed by mask M, not multiplication by a binary matrix. The context encoder processes only these visible tokens:

$$\[S_c = f_\theta(T_c).\]$$

If the context contains N_c patches, then

$$\[S_c \in \mathbb{R}^{B \times N_c \times D}.\]$$

The context pathway does not need to run attention over the missing patches. This is one reason representation prediction can be less computationally expensive than a full-resolution reconstruction objective.

4.16.3. The target encoder and target selection

The target branch defines the feature vectors that the predictor must match. It is not enough to say that the target branch ``encodes the target crop." In the official I-JEPA implementation, the target encoder processes the full image and produces a spatial feature map. The target positions are then selected from this feature map. The target encoder can therefore use image context outside an individual target block when constructing its reference features. This is a specific information path, not an implementation detail that can be omitted from an architecture diagram. (Assran et al., 2023; official I-JEPA implementation)

Write the target encoder as

$$\[S_t = f_{\bar{\theta}}(T) \in \mathbb{R}^{B \times N \times D}.\]$$

For target block i, select its target locations:

$$\[S_t^{(i)} = S_t[M_i] \in \mathbb{R}^{B \times N_i \times D}.\]$$

The target encoder parameters $\bar{\theta}$ do not receive the prediction-loss gradient. They are updated by an exponential moving average of the online/context encoder:

$$\bar{\theta}_{t+1} = m_t \bar{\theta}_t + (1 - m_t)\theta_{t+1}.$$

The target branch is thus a slowly moving reference. It is neither a fixed pretrained network nor an independent network optimized by the same gradient as the context encoder.

The implementation may apply a normalization layer to the target features before or after selecting the requested positions, depending on the exact method and code version. The order matters because normalization statistics can couple tokens or dimensions. When describing a particular implementation, state the actual order rather than drawing a generic target crop through the encoder.

4.16.4. A target position is a query

The predictor cannot infer which output locations are requested from the context features alone. It receives a query describing the target positions. I-JEPA constructs this query with learned mask tokens and positional embeddings.

For target block i, let

$$Q_i = \{q_j : j \in M_i\}$$

be the sequence of query tokens. Each token contains information about the position it represents. The predictor receives both the context features and the query:

$$\widehat{S}_t^{(i)} = g_\phi(S_c, Q_i).$$

Its output has the same shape as the selected target:

$$\widehat{S}_t^{(i)} \in \mathbb{R}^{B \times N_i \times D}.$$

This is not an ordinary decoder that reconstructs pixels. The query asks for features at specified positions, and the predictor returns vectors in the target encoder’s feature space.

Positional information is essential. Without it, the predictor would be asked to produce a set of target features without knowing which location each feature belongs to. A permutation-invariant set of outputs would not preserve the intended spatial correspondence. The target mask is therefore both a sampling device and a query specification.

4.16.5. The loss and its gradient paths

For M target blocks, I-JEPA averages a squared feature discrepancy over the requested target positions:

$$\mathcal{L}_{\text{I-JEPA}} = \frac{1}{M} \sum_{i=1}^{M} D\left(\widehat{S}_t^{(i)}, S_t^{(i)}\right),$$

where the paper equation averages over target blocks and sums squared Euclidean discrepancies over target positions. An implementation may add batch averaging or another explicit normalization; those choices change the loss scale and should be reported separately from the displayed paper equation.

The gradient flows through the predictor and the context encoder. It does not flow through the target encoder. The target encoder follows the online encoder through the EMA update after the optimizer step.

The training step can be represented schematically as the following pseudocode:

```
target_map = target_encoder(patchify(image_batch))
context = context_encoder(select(patchify(image_batch), context_mask))

loss = 0
for mask in target_masks:
    query = mask_tokens_with_positions(mask)
    prediction = predictor(context, query)
    target = select(target_map, mask)
    loss = loss + feature_distance(prediction, target)

loss = loss / number_of_target_blocks
optimizer.zero_grad()
loss.backward()
optimizer.step()
update_target_encoder_ema()
```

This pseudocode omits batching details and the implementation's exact normalization, but it makes the computational graph explicit. In particular, the target map is computed from the image rather than from a crop that contains only the requested target block. The predictor receives positional queries, and the EMA update is separate from ordinary gradient descent.

A figure showing only an encoder, a predictor and a target encoder hides the most important part of the method. The reader should be able to trace which pixels reach each branch, which positions are selected, where positional queries enter, which tensors are compared and which parameters receive gradients.

4.16.6. What changes for video

V-JEPA retains the feature-prediction principle but replaces spatial image patches with spatiotemporal tubelets or tokens. A video is patchified across time, height and width. Its context

mask can remove tubes or spatiotemporal blocks, and its target can include positions distributed across the clip.

The original V-JEPA objective uses an ℓ_1 feature regression loss and an EMA target encoder with stop-gradient. This differs from the squared ℓ_2 discrepancy used in the I-JEPA formulation. The predictor is conditioned on the requested target locations, and the masking strategy determines whether the target is a spatially hidden region, a temporally withheld region or a combination. (Bardes et al., 2024)

The original V-JEPA paper studies several masking regimes, including random tubes, causal multi-block masks that restrict context to a video prefix, and multi-block masks that can draw context and targets across the clip. (Bardes et al., 2024) The causal-prefix setup asks a different question from a non-causal mask over the entire video. The latter can use visible frames both before and after a target time, so it is not equivalent to predicting a future from the past. A reader should never infer temporal causality from the fact that a model was trained on video.

V-JEPA 2 retains masked feature prediction for visual pretraining and then introduces a separate action-conditioned post-training stage. That stage changes the input information and the transition problem, as discussed in the following sections. (Assran et al., 2025)

4.16.7. What changes for predictor-free view matching

LeJEPA illustrates a different case. It constructs global and local views of an observation and moves their embeddings toward the mean of the global-view embeddings. The mean acts as a shared centre. SIGReg regularizes the distribution of the embeddings.

There is no separate predictor that receives a spatial or temporal query and produces a target block. The training views are intended to represent related content, and the method uses direct view agreement. The distributional regularizer constrains the embedding batch while the prediction term defines which views should be close. (Balestriero and LeCun, 2025)

This is not an exception to the JEPA principle. It demonstrates why the predictor should be tied to the information structure of the task. When a target requires a location-specific or action-conditioned prediction, a predictor is useful. When the task is symmetric agreement across related views, a direct loss may suffice.

4.16.8. An implementation audit

Before trusting a feature-prediction training run, check the data and gradient paths rather than looking only at the scalar loss.

Mask leakage. Verify that target positions are not passed to the context encoder as visible inputs. If target blocks overlap the sampled context, resolve the overlap before constructing the context

sequence.

Target correspondence. Check that predicted token j is compared with the target feature at the same requested spatial or spatiotemporal position. A mismatch in flattening order can create a numerically valid but semantically incorrect objective.

Query information. Confirm that the predictor receives a position-bearing query for every requested target location. A mask token without its location cannot specify which feature is to be predicted.

Gradient routing. Confirm that the target branch is detached from the prediction loss when the method calls for an EMA target, and that the EMA update is performed after the online optimizer step. An accidental gradient through the target branch changes the objective.

Target updates. Log the online and target parameter distance. A target that is not updated, or is copied directly from the online encoder, does not implement the intended moving-reference dynamics.

Feature normalization. Record where normalization is applied and whether it is per token, per feature dimension or over the batch. Changing the location of a normalization layer can change both the target geometry and the meaning of the loss.

Prediction scale. Report whether the feature discrepancy is summed or averaged over patches, dimensions, blocks and batch items. The numerical value of the loss changes with these conventions, even when the underlying tensor differences are identical.

Temporal semantics. For video, record whether context is causal, whether the target lies in the future or anywhere in the clip, and how the temporal stride affects the prediction horizon. A random spatiotemporal mask is not automatically a future-prediction task.

Action alignment. For a controlled predictor, verify that every action is paired with the state transition it caused or was intended to cause. A one-step shift in action indexing changes the conditional prediction problem.

A low loss is meaningful only after these checks. Otherwise, the model may be solving a different task from the one the architecture diagram suggests.

The worked example provides the operational interpretation of the central equations: a JEPA predicts features at requested locations in a target representation space. The mask distribution defines what is withheld, the target encoder defines what counts as a correct feature, the predictor maps context to the query, and the gradient path determines which parameters learn from the discrepancy. The next section returns to the complete argument and the limits of what such a training objective establishes.

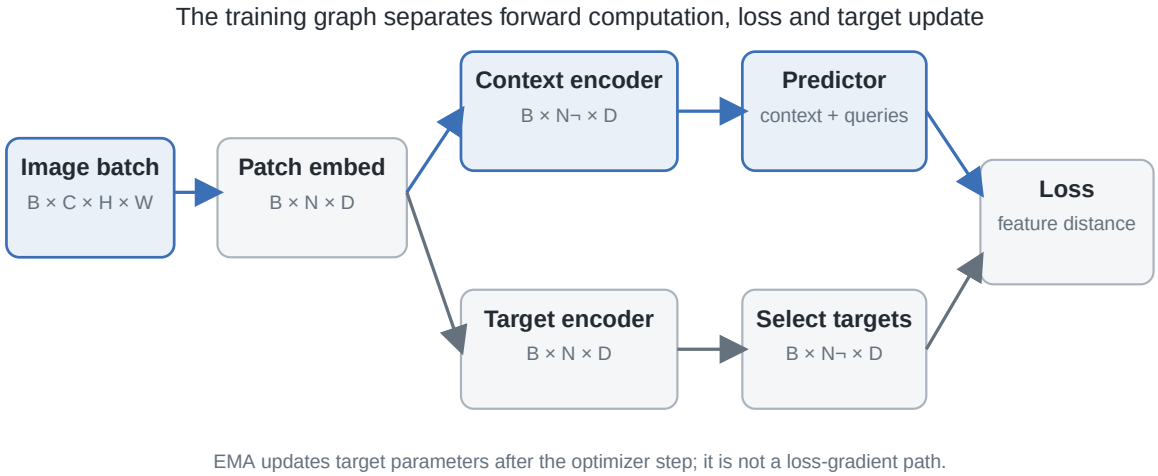


Figure 4 33. I JEPA training step and EMA target update

Figure 4.33. One I-JEPA training step from patch tokens to target update. The context and target branches process different inputs; position queries enter the predictor, and the loss compares predicted features with selected target features. The EMA update changes target parameters after the online optimizer step.

4.17. Synthesis and exercises

4.17.1. The argument of the chapter

A joint-embedding predictive architecture learns a relation between a context and a target in a representation space. Its apparent simplicity hides several questions. What information is available to the predictor? Which target is being requested? What differences does the target encoder preserve? Which representations can minimize the loss? What statistical properties make the resulting features useful to downstream predictors? What additional model and decision machinery is required before the predictions can guide action?

The chapter developed these questions in sequence.

A compatibility energy assigns a score to a context and candidate target. A JEPA can define that compatibility through a discrepancy between a predicted representation and a target representation. The distinction between a score and a probability remains important: a normalized density requires a partition function and a base measure. A task cost is different again because it expresses what an agent should prefer.

The prediction problem is determined by the context, target, query and sampling distribution. A target may be spatially hidden, temporally withheld, action-conditioned or cross-modal. The predictor is justified by the information structure of the problem, not required by the name JEPA.

A deterministic squared-error predictor returns the conditional mean of the target representation, which can average across distinct possible futures.

An unconstrained agreement objective admits a constant representation as a global minimum. Preventing this collapse is necessary, but nonzero variance does not establish that a representation preserves every task-relevant variable. A useful state is sufficient only relative to a target and an action domain:

$$P(Y \mid X, A) = P(Y \mid Z, A).$$

The distribution of embeddings also matters to downstream estimation. Under the assumptions of the linear-probe analysis, covariance anisotropy can increase ridge bias for a task aligned with a weak feature direction and increase OLS coefficient variance at fixed total variance. For local nonlinear probes, the score of the embedding density enters the leading bias. Under the paper's smoothness, gradient and covariance assumptions, the integrated bias depends on the Fisher-information functional

$$J(p) = \int \|\nabla \log p(z)\|_2^2 p(z)\, dz.$$

For the estimator families and scalar covariance constraints analysed by LeJEPA, an isotropic Gaussian uniquely minimizes the relevant risk criteria. This is a conditional mathematical result, not a universal claim about every downstream task.

SIGReg turns Gaussian distribution matching into a collection of projected one-dimensional tests. The Cramér–Wold result concerns all directions; training uses a finite stochastic sketch and an approximate numerical integral. VISReg uses a different one-dimensional statistic and explicitly separates centre, scale and shape. Both methods constrain embedding geometry, but neither establishes that the learned variables are causally complete or sufficient for physical control.

Action-conditioned prediction changes the information supplied to the model. Passive video estimates what tends to happen in observed trajectories. A controlled model is trained to predict the future given a specified action and relevant state. Planning then searches over action sequences, evaluates predicted outcomes with a task cost, executes a selected action and replans from new observations. Its reliability depends on the representation, transition model, data coverage, cost, optimizer, constraints and feedback loop.

Finally, a world model operates inside a larger agent. Perception estimates state; World Model imagines futures; Cost evaluates them; Actor selects actions; Short-Term Memory retrieves experience; Configurator adapts the modules to the task. Mode 1 provides reactive behaviour, while Mode 2 reasons through imagined trajectories. Hierarchical prediction, memory and multiple

futures are proposed mechanisms for extending this process across time scales and uncertainty.

The chapter's central claim is therefore limited but useful: predicting representations can supply a compact internal model of selected relations in the world. Whether those relations support action depends on what the representation preserves, what the predictor is conditioned on, and how an agent uses its predictions.

4.17.2. What has and has not been established

The main mathematical conclusions can be separated from the engineering and empirical claims.

Result	**What it establishes**	**What it does not establish**
Constant embeddings minimize unconstrained agreement loss	Agreement alone does not require information	Every training run will collapse
Predictive sufficiency $P(Y \mid X, A) = P(Y \mid Z, A)$	A task-relative condition for retaining predictive information	Universal physical state sufficiency
Ridge and OLS covariance analysis	Anisotropy can worsen specified linear-estimator bias or variance	Every task performs worse in every anisotropic representation
Fisher-information analysis	Local-estimator bias is linked to the embedding density score under stated assumptions	Gaussianity is optimal for every nonlinear model or control task
Cramér–Wold	All one-dimensional projections identify a multivariate distribution	A finite set of projections certifies equality
Epps–Pulley/SIGReg	A differentiable practical statistic for projected Gaussian matching	Exact population Gaussianity from one minibatch
VISReg scale and shape decomposition	Separate terms can target separate batch properties	Universal gradient recovery or no-collapse guarantees
Action-conditioned prediction	Actions are explicit inputs to the transition estimate	Causal identification without suitable data and assumptions

Result	What it establishes	What it does not establish
Planning success in an experiment	A complete model-planner-environment system worked under a reported protocol	General reliability or safety in arbitrary environments
LeCun's architecture	A proposed organization of agent capabilities	A validated end-to-end autonomous system

4.17.3. Exercises

Energy and probability

Let $E(x, y)$ be a compatibility energy. Derive the conditional Gibbs density associated with this energy. Identify the partition function and explain why minimizing energy over y does not require computing it. Give an example of a task cost that should not be interpreted as a probability density.

Latent optimization versus marginalization

Compare

$$\inf_z E(x, y, z) \quad \text{and} \quad -\log \int e^{-E(x,y,z)}\, dz.$$

Construct a case with two equal low-energy latent configurations and a second case with one low-energy configuration. Explain how minimization and marginalization treat these cases differently.

A predictor or direct view agreement?

Consider two training views X_1, X_2 of the same image. Propose a setting in which direct embedding agreement is appropriate. Then consider a robot observation history H_t, an action A_t, and a future visual target Y_{t+1}. Explain why the second setting has an information asymmetry that can justify a predictor.

Conditional-mean prediction

Let $H \mid X = x$ take the values -2 and $+2$ with equal probability. Derive the deterministic squared-error predictor and show that its value is not one of the possible target outcomes. Explain one reason why this can be harmful for a planner.

Collapse

For the objective

$$\mathcal{L} = \mathbb{E}\|f_\theta(X) - f_\theta(X')\|_2^2,$$

show that a constant representation is a global minimizer. Does adding an EMA target encoder change this algebraic fact about a constant solution? Explain the difference between a property of the objective and a statement about optimization dynamics.

Predictive equivalence

Construct two histories x_1, x_2 that have similar current images but lead to different future distributions under the same action. Define a representation that maps them to the same vector. Show why the predictive-sufficiency condition fails.

Ridge bias

Let $X^\top X = Q\Lambda Q^\top$. Starting from the ridge estimator, derive its coefficient bias in the eigenbasis. Under a fixed sum of eigenvalues, construct the downstream coefficient vector that makes an anisotropic covariance more biased than an isotropic covariance.

OLS variance

Under a full-rank design and homoskedastic noise, derive the conditional covariance of the OLS estimator. Use Jensen's inequality to show that the trace of this covariance is minimized by equal eigenvalues when the total feature variance is fixed. State the assumptions required.

Local regression bias

Starting with the Nadaraya–Watson estimator, explain why the leading bias contains both the Laplacian of the target function and the dot product of the target gradient with the density score. Give an intuitive example in which the sample density is high on one side of a query point.

Fisher-information lower bound

Let p be a mean-zero density with covariance $\Sigma \succ 0$. Use integration by parts to show

$$\mathbb{E}[Z\nabla \log p(Z)^\top] = -I.$$

Then use positive semidefiniteness of an outer-product expectation to derive

$$J(p) \geq \operatorname{tr}(\Sigma^{-1}).$$

State the equality condition and identify the density that achieves it.

Projection matching

Construct two distributions with equal means and covariances but different shapes. Explain why a finite number of moments may fail to distinguish them. Then state precisely what Cramér–Wold requires and why a finite random projection set is a sketch.

SIGReg versus VISReg

Write the conceptual one-dimensional statistic used by SIGReg and the sorted-sample transport statistic used by VISReg. Identify the role of normalization, random directions, numerical integration and sorting. Explain which parts of each method are stochastic approximations.

Passive and controlled prediction

Suppose a camera records an object moving while a robot controller acts in the background. Give two data-generating mechanisms that produce similar passive videos but different consequences under a new robot action. Specify which variables should enter an action-conditioned model and which alignment information the dataset must retain.

Planning and model exploitation

Construct a simple example in which a planner discovers an action sequence that receives a low predicted goal cost but fails in the real environment. Explain how more candidate samples can make this problem worse. Propose two mitigations and identify their limitations.

Mode 1 and Mode 2

Describe a task for which a Mode-2 planner can generate useful action targets and a Mode-1 policy can later execute them cheaply. Write a distillation loss. Explain why the reactive policy may inherit systematic errors from the planner.

Hierarchical prediction

Design a two-level predictor for a robot moving an object to a shelf. Specify an abstract high-level target, a lower-level physical state and the interface between them. Identify a state variable that must not be discarded by the high-level representation.

Memory

Give an example in which a state that is not visible in the current observation is useful for prediction. Explain how associative retrieval differs from simply concatenating the most recent fixed number of frames.

Evidence design

Design an evaluation that separates frozen representation quality, action-conditioned prediction, multi-step rollout and physical planning. For each stage, state the metric, data split and failure mode it is intended to detect.

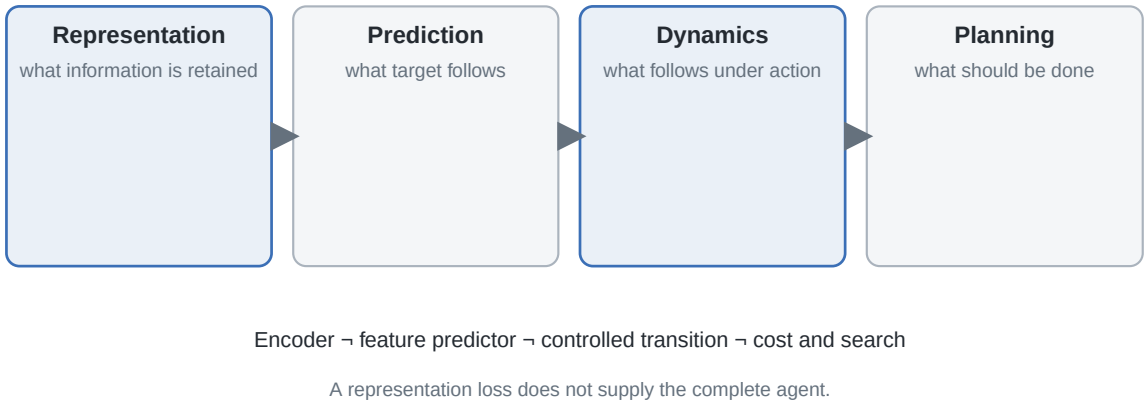


Figure 4 34. Representation prediction dynamics and planning interfaces

Figure 4.34. Representation, prediction, dynamics and planning are separate interfaces. The representation determines what information is retained, the prediction objective specifies a target, the transition model estimates consequences under actions, and the planner selects desirable actions. No single feature loss supplies the complete agent.

4.18. References

This file identifies the exact versions supplied for Chapter 4 and the primary sources used in the text. Paper-specific claims should be cited to the version that supports them. Where a method is a stage within a larger paper rather than a separate publication, it is cited to that paper rather than treated as an independent work.

Primary papers supplied for the chapter

4.19. I-JEPA

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., and Ballas, N. (2023). ``Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:2301.08243, version 3.

Supplied file: `rough-draft/chapter4/papers_pdfs/2301.08243v3.pdf`.

Primary source: https://arxiv.org/abs/2301.08243

Official implementation: https://github.com/facebookresearch/ijepa

Use in this chapter: spatial context-to-target prediction, multiple target blocks, feature-space prediction, target-encoder pathway and the limits of image-only representation learning.

4.20. Original V-JEPA

Bardes, A. et al. (2024). ``Revisiting Feature Prediction for Learning Visual Representations from Video." arXiv:2404.08471, version 1.

Supplied file: `rough-draft/chapter4/papers_pdfs/2404.08471v1.pdf`.

Primary source: https://arxiv.org/abs/2404.08471

Official project and implementation sources are maintained by Meta FAIR through the V-JEPA research releases.

Use in this chapter: spatiotemporal feature masking and prediction, target/context information flow, video representation learning, and the distinction between masked video prediction and action-conditioned dynamics.

4.21. V-JEPA 2 and V-JEPA 2-AC

Assran, M. et al. (2025). ``V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning." arXiv:2506.09985, version 1.

Supplied file: `rough-draft/chapter4/papers_pdfs/2506.09985v1.pdf`.

Primary source: https://arxiv.org/abs/2506.09985

Official implementation: https://github.com/facebookresearch/vjepa2

Use in this chapter: passive video pretraining, the separate action-conditioned post-training stage, robot trajectory data, proprioceptive inputs, image-goal planning and the scope of the reported physical deployments. V-JEPA 2-AC is treated as a model/stage within this paper, not as an unrelated publication.

4.22. LeJEPA and SIGReg

Balestriero, R. and LeCun, Y. (2025). ``LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics." arXiv:2511.08544, version 3, 14 November 2025.

Supplied file: `rough-draft/chapter4/papers_pdfs/2511.08544v3 (1).pdf`.

Primary source: https://arxiv.org/abs/2511.08544

Official project/code pointer: https://github.com/rbalestr-lab/lejepa

Use in this chapter: the conditions under which a predictor is justified, multi-view agreement, linear and nonlinear probe risk, the Fisher-information functional, Gaussian optimality under stated assumptions, Cramér–Wold projections, the Epps–Pulley statistic and SIGReg.

4.23. VL-JEPA

Chen, D., Shukor, M., Moutakanni, T., Chung, W., Yu, L., Kasarla, T., Bolourchi, A., LeCun, Y., and Fung, P. (2025/2026). ``VL-JEPA: Joint Embedding Predictive Architecture for Vision-Language." arXiv:2512.10942, version 2.

Supplied file: `rough-draft/chapter4/papers_pdfs/2512.10942v2.pdf`.

Primary source: https://arxiv.org/abs/2512.10942

Use in this chapter: one cross-modal example showing that the target representation need not be visual. VL-JEPA predicts a text embedding from visual context and can decode language when a text output is required. It is not treated as an action-conditioned physical dynamics model.

4.24. V-JEPA 2.1

Mur-Labadia, L. et al. (2026). ``V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning." arXiv:2603.14482, version 3.

Supplied file: `rough-draft/chapter4/papers_pdfs/2603.14482v3_compressed.pdf`.

Primary source: https://arxiv.org/abs/2603.14482

Source license: arXiv identifies version 3 as CC BY-NC-ND 4.0. Treat the paper's figures as non-commercial, no-derivatives material; independently drawn explanatory figures are preferable.

Official implementation: https://github.com/facebookresearch/vjepa2

Use in this chapter: dense visible- and masked-token supervision, deep self-supervision and the distinction between dense representation learning and action-conditioned robot prediction. Any planning comparison must retain the predictor and planner settings from the paper.

4.25. VISReg

Wu, H., Balestriero, R., and Levine, M. (2026). ``Variance-Invariance-Sketching Regularization for JEPA Training." arXiv:2606.02572, version 1.

Supplied file: `rough-draft/chapter4/papers_pdfs/2606.02572v1 (1).pdf`.

Primary source: https://arxiv.org/abs/2606.02572

Source license: arXiv identifies the source with CC BY-NC 4.0 terms. Check the precise reuse conditions before adapting or reproducing any paper material; use original explanatory artwork.

Project/source pointer: https://haiyuwu.github.io/visreg/

Use in this chapter: explicit separation of centre, scale and shape; stop-gradient in shape normalization; sliced-Wasserstein projections; scale-gradient analysis; compute dependence on batch size, embedding dimension and number of slices; and the paper's bounded experimental claims.

4.26. Autonomous machine intelligence

LeCun, Y. (2022). *A Path Towards Autonomous Machine Intelligence*. Version 0.9.2, 27 June 2022. OpenReview: BZ5a1r-kVsf.

Supplied file: `rough-draft/chapter4/papers_pdfs/10356_a_path_towards_autonomous_mach.pdf`.

Primary source: https://openreview.net/forum?id=BZ5a1r-kVsf

Use in this chapter: energy-based compatibility, latent variables for alternative futures, the six-module agent proposal, Mode 1 and Mode 2, Mode-2-to-Mode-1 distillation, associative short-term memory, hierarchical prediction and the Configurator. The document is identified as a position paper and research proposal.

Additional background citations used in Chapter 4

These papers are cited briefly for mechanisms already introduced in Chapter 3. The Chapter 4 discussion is limited to the role each mechanism plays in the collapse argument.

- Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. (2020). ``A Simple Framework for Contrastive Learning of Visual Representations." *Proceedings of the 37th International Conference on Machine Learning (ICML)*. arXiv:2002.05709.
- Caron, M., Misra, I., Mairal, J., Goyal, P., Bojanowski, P., and Joulin, A. (2020). ``Unsupervised Learning of Visual Features by Contrasting Cluster Assignments." *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2006.09882.
- Grill, J.-B. et al. (2020). ``Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning." *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2006.07733.

- Chen, X. and He, K. (2021). ``Exploring Simple Siamese Representation Learning." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:2011.10566.
- Zbontar, J. et al. (2021). ``Barlow Twins: Self-Supervised Learning via Redundancy Reduction." *Proceedings of the 38th International Conference on Machine Learning (ICML)*. arXiv:2103.03230.
- Bardes, A., Ponce, J., and LeCun, Y. (2022). ``VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning." *International Conference on Learning Representations (ICLR)*. arXiv:2105.04906.